Full-Stack Developer Revision Guide

Basics to Advanced — beginner-friendly explanations with coding examples

JavaScript (ES2024+) • TypeScript

React (18 / 19) • Node.js & Express

Python • Django & Django REST Framework

SQL (PostgreSQL / MySQL)

Table of Contents

Part 0 — Foundations: Read This First

- 0.1 How to Use This Guide
- 0.2 What Is Programming?
- 0.3 How the Web Works
- 0.4 The Three Languages of the Browser
- 0.5 Frontend, Backend, Database: Where Each Technology Fits
- 0.6 Essential Tools
- 0.7 Core Computer-Science Ideas You Will Meet
- 0.8 Glossary of Common Terms

Part 1 — JavaScript: Basics to Advanced

- 1.1 How JavaScript Runs
- 1.2 Variables, Scope and Hoisting
- 1.3 Data Types
- 1.4 Functions
- 1.5 this Binding Rules
- 1.6 Closures
- 1.7 Objects and Prototypes
- 1.8 Classes (ES6+)
- 1.9 Arrays — Essential Methods
- 1.10 Modern Syntax (ES6 → ES2024)
- 1.11 Map, Set, WeakMap, WeakSet
- 1.12 Iterators and Generators
- 1.13 The Event Loop
- 1.14 Promises and Async/Await
- 1.15 Error Handling
- 1.16 Modules
- 1.17 Memory Management and Performance
- 1.18 Browser and DOM Essentials
- 1.19 Functional Programming Concepts
- 1.20 Design Patterns in JavaScript
- 1.21 Deep Clone and Deep Equal (Interview Implementations)
- 1.22 Quick Revision — Tricky Output Questions

Part 2 — TypeScript: Basics to Advanced

- 2.1 Why TypeScript
- 2.2 Key tsconfig.json Options
- 2.3 Basic Types
- 2.4 Interfaces vs Type Aliases
- 2.5 Functions
- 2.6 Narrowing and Type Guards
- 2.7 Generics
- 2.8 Built-in Utility Types
- 2.9 Advanced Types
- 2.10 Classes in TypeScript
- 2.11 Enums vs Union Literals
- 2.12 Decorators (TS 5 Standard Decorators)
- 2.13 Declaration Files and Module Augmentation

2.14 TypeScript with React
2.15 TypeScript with Node/Express
2.16 Type Challenges (Practice)
2.17 Variance, any vs unknown vs never — Summary

Part 3 — React: Basics to Advanced

3.1 Core Ideas
3.2 Components and Props
3.3 Hooks — Complete Reference
3.4 Custom Hooks
3.5 Stale Closures — The Most Common Hook Bug
3.6 Performance Optimization
3.7 Advanced Component Patterns
3.8 Error Boundaries
3.9 Class Component Lifecycle (for legacy code)
3.10 State Management
3.11 Routing (React Router v6/v7)
3.12 Forms — React Hook Form + Zod
3.13 Rendering Strategies and Server Components
3.14 Testing React
3.15 Interview Questions Checklist

Part 4 — Node.js: Basics to Advanced

4.1 Architecture
4.2 The Node.js Event Loop Phases
4.3 Modules
4.4 Core Modules
4.5 EventEmitter
4.6 Buffers and Streams
4.7 HTTP Server from Scratch
4.8 Express.js In Depth
4.9 Authentication and Authorization
4.10 Databases from Node
4.11 Caching with Redis
4.12 Scaling Node.js
4.13 Background Jobs and Queues
4.14 Real-time: WebSockets / Socket.IO
4.15 Error Handling and Process Management
4.16 Configuration and Logging
4.17 Security Checklist
4.18 Performance
4.19 Testing Node
4.20 REST API Design Quick Reference

Part 5 — Python: Basics to Advanced

5.1 How Python Runs
5.2 Core Data Types
5.3 Control Flow and Comprehensions
5.4 Functions
5.5 Decorators
5.6 Iterators and Generators
5.7 Object-Oriented Python
5.8 Context Managers

5.9 Exceptions
5.10 Type Hints
5.11 Concurrency: Threads, Processes, Asyncio
5.12 Modules, Packages and Environments
5.13 Memory Model and Internals
5.14 Metaclasses (Advanced)
5.15 Testing with pytest
5.16 Common Interview Snippets

Part 6 — Django & Django REST Framework

6.1 Architecture: MTV and the Request Lifecycle
6.2 Settings Essentials
6.3 Models
6.4 ORM — QuerySets
6.5 Transactions and Concurrency
6.6 Migrations
6.7 Views
6.8 Templates and Forms (Quick)
6.9 Middleware
6.10 Signals
6.11 Authentication and Permissions
6.12 Django REST Framework (DRF)
6.13 Caching
6.14 Celery (Background Tasks)
6.15 Async Django
6.16 Testing Django
6.17 Security Checklist
6.18 Performance and Deployment
6.19 Service Layer Pattern (Clean Django)
6.20 Interview Questions Checklist

Part 7 — SQL: Basics to Advanced

7.1 SQL Sub-languages
7.2 Data Types (Common)
7.3 DDL and Constraints
7.4 Basic DML and Queries
7.5 Logical Order of Execution
7.6 Aggregation: GROUP BY and HAVING
7.7 Joins
7.8 Set Operations
7.9 Subqueries
7.10 Common Table Expressions (CTEs)
7.11 Window Functions
7.12 Classic SQL Interview Problems
7.13 Indexes
7.14 Query Optimization and EXPLAIN
7.15 Transactions and ACID
7.16 Normalization
7.17 Views, Materialized Views, Functions, Triggers
7.18 Working with JSON, Strings and Dates (PostgreSQL)
7.19 Security, Roles and SQL Injection
7.20 Scaling Databases

7.21 Quick Reference — Postgres vs MySQL

Part 0 — Foundations: Read This First

This guide assumes you are starting from zero. This first part explains the big picture: what programming is, how the web works, and how all the technologies in this guide fit together. Every later section begins with a shaded **"Understand the concept"** box written in plain language. Read that box first, then study the code examples beneath it.

0.1 How to Use This Guide

1. **Read the concept box** for a section. Make sure you can explain the idea out loud in your own words.
2. **Type the code examples yourself** instead of copying them. Change values and see what happens. You learn by breaking things.
3. **Run JavaScript** in your browser console (press `F12` and open the *Console* tab) or with Node.js (`node file.js`). **Run Python** with `python file.py` or the interactive shell `python`. **Run SQL** in a free online playground (search "PostgreSQL online") or a local PostgreSQL install.
4. **Recommended order for a complete beginner:** Part 0 → JavaScript → TypeScript → React → Node.js → SQL → Python → Django. JavaScript comes first because React, Node and TypeScript all build on it. SQL is easier to learn before Django, because Django generates SQL for you.
5. Don't try to memorise everything on the first pass. Go through once for understanding, then again for detail.

0.2 What Is Programming?

A computer can only follow very precise, very small instructions. **Programming** means writing those instructions in a **programming language**, which is a strict, human-readable notation that a tool then translates into something the machine can execute.

Every program, however large, is built from a handful of ideas:

Idea	Plain meaning	Example
Value / data	A piece of information	`42`, `"Anish"`, `true`
Variable	A labelled box that holds a value	`age = 30`
Operator	Combines values	`+`, `-`, `==`, `>`
Condition	Do something only if a rule is true	`if age >= 18 ...`
Loop	Repeat something	"for every user in the list..."
Function	A named, reusable recipe of steps	`calculateTax(salary)`
Data structure	A way to organise many values	list `[1,2,3]`, dictionary `{name: "A"}`
Input / Output	Data coming in (keyboard, network, file) and going out (screen, response)	reading a form, showing a page

Compiled vs interpreted languages. A *compiled* language (C, Go) is translated fully into machine code before it runs. An *interpreted* language (Python, JavaScript) is read and executed by another program, called the **interpreter** or **engine**, at run time. Modern JavaScript and Python engines mix both approaches for speed.

Static vs dynamic typing. In a *statically typed* language (TypeScript, Java), the kind of value a variable holds is checked *before* the program runs. In a *dynamically typed* language (JavaScript, Python), it is only known *while* the program runs. Static typing catches mistakes earlier; dynamic typing is quicker to write.

0.3 How the Web Works

When you type `https://shop.com/products` into a browser:

1. **DNS lookup:** the browser asks a DNS server, "what is the IP address of `shop.com`?" (a bit like looking up a phone number in a contacts list).
2. **Connection:** the browser opens a TCP connection to that address. With **HTTPS** it is encrypted using TLS.
3. **HTTP request:** the browser sends a text message like `GET /products`.
4. **Server:** a program on the server (written with Node/Express or Python/Django, for example) receives the request, maybe reads a **database**, and builds a **response**.
5. **HTTP response:** the server replies with a **status code** (`200 OK`, `404 Not Found`), **headers** (metadata) and a **body** (HTML, JSON, an image...).
6. **Rendering:** the browser turns HTML + CSS + JavaScript into the page you see.



Client vs server. The *client* is the program asking for something (usually the browser or a mobile app). The *server* is the program answering. One server handles requests from thousands of clients.

HTTP methods express intent: `GET` (read), `POST` (create), `PUT` / `PATCH` (update), `DELETE` (remove).

JSON (JavaScript Object Notation) is the most common format for exchanging data between client and server:

```
{ "id": 7, "name": "Laptop", "price": 999.99, "tags": ["electronics", "sale"] }
```

An API (Application Programming Interface) is a defined set of requests a program accepts. A **REST API** exposes data as URLs (`/api/products/7`) and uses HTTP methods to act on them.

0.4 The Three Languages of the Browser

- **HTML** describes *structure and content*: headings, paragraphs, buttons, forms.
- **CSS** describes *appearance*: colours, layout, fonts, spacing.
- **JavaScript** describes *behaviour*: what happens when you click, fetching data, updating the page without a reload.

```
<button id="hello">Say hello</button>              <!-- HTML: a button -->
<style> #hello { background: navy; color: white; } </style>   <!-- CSS: styling -->
<script>
  document.getElementById('hello').addEventListener('click', () => alert('Hello!')); // JS:
behaviour
</script>
```

The **DOM** (Document Object Model) is the browser's in-memory tree representation of the HTML. JavaScript changes the page by changing the DOM.

0.5 Frontend, Backend, Database: Where Each Technology Fits

Layer	Job	Technologies in this guide
Frontend (runs in the browser)	Shows the UI, reacts to the user	JavaScript, TypeScript, **React**
Backend (runs on a server)	Business rules, security, talking to the database, APIs	**Node.js + Express**, **Python + Django / DRF**
Database	Stores data permanently and lets you query it	**SQL** (PostgreSQL, MySQL)

A typical **full-stack** feature, such as "show my orders", touches all three layers. A React component calls `GET /api/orders`. A Django or Express view checks who you are, runs a SQL query, and returns JSON. React then displays the list.

- **JavaScript** is the language of the browser. **Node.js** lets the same language run on servers.
- **TypeScript** is JavaScript plus *types*, which catch mistakes before the code runs.
- **React** is a JavaScript *library* for building user interfaces out of reusable *components*.
- **Python** is a general-purpose language known for readability. **Django** is a Python *framework* for building web backends quickly.
- **SQL** (Structured Query Language) is the language used to ask relational databases for data.

Library vs framework. You call a *library* when you need it (React, lodash). A *framework* calls *your* code and dictates the structure of your project (Django, Next.js). People describe this as "inversion of control".

0.6 Essential Tools

- **Terminal / command line:** a text interface to run programs (`cd folder`, `ls`, `node app.js`, `python manage.py runserver`).
- **Code editor:** VS Code is the most common choice.
- **Package managers: npm** for JavaScript and **pip** for Python. They download libraries other people wrote (`npm install react`, `pip install django`).
- **Git:** version control. It records snapshots ("commits") of your code so you can go back, collaborate and review changes. **GitHub** hosts Git repositories online.
- **Browser DevTools (F12):** Console (run JS, see errors), Elements (inspect HTML/CSS), Network (see every HTTP request).
- **Debugger:** pause code on a line (a *breakpoint*) and inspect variables. Use `debugger;` in JS or `breakpoint()` in Python.

0.7 Core Computer-Science Ideas You Will Meet

Memory: stack and heap. Programs store short-lived values for the function that is currently running on the *stack* (fast, small, automatically cleaned up). Objects that live longer or grow (lists, objects) go on the *heap*. Variables often hold a *reference* (an address) to something on the heap. That is why two variables can point to the *same* object.

Synchronous vs asynchronous. *Synchronous* code runs one step after another, and each step waits for the previous one to finish. *Asynchronous* code starts a slow task (a network request, reading a file) and continues with other work, handling the result later. Think of ordering coffee: you don't stand frozen at the counter. You get a buzzer and sit down, and the buzzer is the "callback". JavaScript, Node.js and Python's `asyncio` all rely on this idea.

Big-O notation describes how the work grows as the input grows:

Notation	Name	Example
O(1)	Constant	Look up a key in a dictionary
O(log n)	Logarithmic	Binary search, database index lookup
O(n)	Linear	Loop over a list once
O(n log n)	Linearithmic	Good sorting algorithms
O(n²)	Quadratic	A loop inside a loop over the same list

Errors and exceptions. When something goes wrong at run time (dividing by zero, a missing file), the program *throws/raises* an exception. You can *catch* it and handle it gracefully, or let it crash the program.

Security basics. Never trust input from users. Validate it and escape it. Store passwords only as *hashes* (one-way scrambled values). Keep secrets (API keys, passwords) out of your code, in *environment variables*.

0.8 Glossary of Common Terms

Term	Meaning
Syntax	The grammar rules of a language
Runtime	The environment where code executes (browser, Node.js, Python interpreter)
Compile / transpile	Translate code into another form (TypeScript → JavaScript)
Bug	A mistake that causes wrong behaviour
Refactor	Restructure code without changing what it does
Dependency	A library your project needs
Environment variable	A configuration value supplied from outside the code
Endpoint	A specific URL + method of an API, e.g. `POST /api/users`
Authentication	Proving *who* you are (login)
Authorization	Deciding *what* you are allowed to do (permissions)
CRUD	Create, Read, Update, Delete: the four basic data operations
ORM	Object-Relational Mapper: lets you use database rows as objects in code (Django ORM, Prisma)
Cache	A fast temporary copy of data to avoid repeating slow work
Deployment	Putting your application on a server so others can use it
Unit test	Code that automatically checks a small piece of other code

Part 1 — JavaScript: Basics to Advanced

UNDERSTAND THE CONCEPT

What is JavaScript? JavaScript (JS) is the programming language that web browsers understand. It was created in 1995 to make web pages interactive, for example to react to clicks or check a form before sending it. Today it also runs on servers (Node.js), phones (React Native) and desktops (Electron). It is the most widely used programming language in the world.

Key characteristics, in plain words:

- **Single-threaded:** it does one thing at a time. It still handles many tasks "at once" through asynchronous callbacks (explained in 1.13).
- **Dynamically typed:** a variable can hold a number now and a string later. The language does not check types in advance.
- **Functions are values:** you can store a function in a variable, pass it to another function, or return it from a function. Many advanced patterns build on this.
- **Prototype-based objects:** objects can inherit directly from other objects (explained in 1.7).

Your first program. Open the browser console (F12) and type:

```
let name = 'World';                 // store a value in a variable
console.log('Hello, ' + name);      // print to the console → Hello, World

function square(n) {                // define a reusable function
  return n * n;
}
console.log(square(5));             // 25

if (square(5) > 20) {               // a condition
  console.log('big number');
}

for (let i = 1; i <= 3; i++) {      // a loop: runs with i = 1, 2, 3
  console.log('Count', i);
}
```

1.1 How JavaScript Runs

UNDERSTAND THE CONCEPT

JavaScript code is plain text. To run it, a program called a **JavaScript engine** reads and executes it. Chrome and Node.js use the **V8** engine, Firefox uses SpiderMonkey and Safari uses JavaScriptCore.

The engine works roughly like this:

1. **Parsing:** it reads your text and checks the grammar. A typo here gives a `SyntaxError` before anything runs.
2. **Interpreting:** it quickly converts the code into simple instructions (*bytecode*) and starts running them.
3. **Optimising (JIT, "Just-In-Time" compiling):** while the code runs, the engine notices which functions are used often ("hot") and compiles them into fast machine code. If its assumptions turn out wrong (say a function that always got numbers suddenly gets a string), it throws the fast version away. That is called *de-optimising*.

The **host environment** is whatever is running the engine. The *browser* adds things like `document` (the page), `window` and `alert`. *Node.js* adds things like `fs` (files) and `http` (servers). The core language (variables, functions, arrays) is the same everywhere.

An **execution context** is the "workspace" the engine creates whenever it runs code. One is created for the whole file (global context) and a new one every time a function is called. It remembers the variables that exist, where to look for outer variables, and what `this` means. These contexts are stacked on the **call stack**. When a function is called its context is pushed on top, and when it returns that context is popped off.

JavaScript is a single-threaded, dynamically typed, prototype-based language with first-class functions. It runs inside a **host environment** (browser, Node.js, Deno, Bun) that provides APIs such as the DOM, `fetch`, timers, and file system access.

Engine pipeline (V8): source → parser → AST → Ignition (bytecode interpreter) → TurboFan (optimizing JIT compiler). Hot functions are optimized based on observed types; if types change ("megamorphic" call sites), code is deoptimized. Practical takeaway: keep object shapes consistent (initialize all properties in the constructor, in the same order).

Execution context has three parts:

1. **Variable environment** — `var` declarations and function declarations.
2. **Lexical environment** — `let`, `const`, `class` bindings, plus a reference to the outer environment (this chain is the *scope chain*).
3. **`this` binding**.

Each context is created in two phases: **creation** (hoisting — memory allocated for declarations) and **execution** (code runs line by line).

1.2 Variables, Scope and Hoisting

UNDERSTAND THE CONCEPT

A variable is a named box that stores a value. You create (declare) one with `let`, `const` or the older `var`.

- Use **`const`** by default. The name cannot be pointed at a different value later.
- Use **`let`** when the value must change (a counter, for example).

- Avoid `var`. It has confusing older rules, but you must understand it for interviews and legacy code.

Scope answers the question "where in my code can this variable be seen?"

- **Global scope:** declared outside everything, so visible everywhere.
- **Function scope:** declared inside a function, so visible only inside that function.
- **Block scope:** declared inside `{ }` (an `if`, a `for` loop) with `let` / `const`, so visible only inside those braces.

```
const site = 'shop';            // global
function show() {
  const user = 'A';             // function scope
  if (true) {
    let temp = 1;               // block scope — exists only inside these braces
    console.log(site, user, temp); // all visible here
  }
  // console.log(temp);         // ReferenceError: temp is not defined here
}
```

Inner code can see outer variables, but outer code cannot see inner variables. Picture a house with one-way windows: from a small room you can look out into the hallway, but nobody in the hallway can see into the room. When the engine looks up a variable, it checks the current scope, then the one outside it, and so on until the global scope. This path is called the **scope chain**.

Hoisting. Before running a scope, the engine first scans it for declarations and reserves memory for them. This makes it *look as if* declarations were "lifted" (hoisted) to the top.

- `var` variables are hoisted *and* set to `undefined`, so reading them early gives `undefined` instead of an error.
- `let` / `const` are hoisted but *not initialised*. Reading them before the declaration line throws an error. The zone between the start of the scope and the declaration line is called the **Temporal Dead Zone (TDZ)**.
- **Function declarations** are hoisted completely, so you can call them before the line where they are written.

Keyword	Scope	Hoisted?	Re-declare	Re-assign	TDZ
`var`	Function	Yes, initialized to `undefined`	Yes	Yes	No
`let`	Block	Yes, uninitialized	No	Yes	Yes
`const`	Block	Yes, uninitialized	No	No (binding only)	Yes

```
console.log(a); // undefined   (var hoisted & initialized)
var a = 1;

console.log(b); // ReferenceError: Cannot access 'b' before initialization (TDZ)
let b = 2;

const obj = { x: 1 };
obj.x = 2;      // OK — const freezes the binding, not the value
// obj = {};    // TypeError

// Function declarations are fully hoisted; function expressions are not
hoisted();      // works
```

```
function hoisted() { return 'hi'; }

notHoisted();   // TypeError: notHoisted is not a function
var notHoisted = function () {};
```

Classic loop bug — `var` is function-scoped, so all callbacks share one `i`:

```
for (var i = 0; i < 3; i++) setTimeout(() => console.log(i)); // 3 3 3
for (let j = 0; j < 3; j++) setTimeout(() => console.log(j)); // 0 1 2 (new binding per iteration)
```

1.3 Data Types

UNDERSTAND THE CONCEPT

Every value in JavaScript has a **type**. There are two families:

1. Primitive types are simple, single values. They are *immutable*: you never change a primitive, you replace it with a new one.

Type	Example	Meaning
`number`	`42`, `3.14`, `NaN`, `Infinity`	All numbers (integers and decimals share one type)
`string`	`'hello'`, `"hi"`, `` `hi ${x}` ``	Text
`boolean`	`true`, `false`	Yes/no
`undefined`	`undefined`	"A value was never assigned"
`null`	`null`	"Intentionally empty", set on purpose
`bigint`	`123n`	Integers larger than `Number` can safely hold
`symbol`	`Symbol('id')`	A guaranteed-unique identifier

2. Objects are collections of values (arrays, functions, dates and plain `{}` objects are all objects). A variable holding an object stores a **reference**, like the address of a house, rather than the house itself. Copy the variable and you copy the address, so both variables lead to the **same** object.

```
let a = 5;  let b = a;  b = 10;          // primitives are copied by value
console.log(a);                           // 5 — unaffected

let o1 = { n: 5 };  let o2 = o1;  o2.n = 10;  // objects are shared by reference
console.log(o1.n);                        // 10 — same object!
```

Type coercion means JavaScript automatically converting a value to another type, for example turning `'5'` into `5` when you do `'5' - 1`. It is convenient but surprising, so prefer the **strict equality** operator `===`, which never converts types.

Truthy and falsy: in a condition like `if (x)`, JavaScript converts `x` to true or false. Only eight values are **falsy** (`false`, `0`, `-0`, `0n`, `''`, `null`, `undefined`, `NaN`). Everything else, including `'0'`, `[]` and `{}`, is **truthy**.

Shallow vs deep copy: a *shallow copy* duplicates only the outer object. Nested objects are still shared. A *deep copy* duplicates everything, all the way down.

Primitives (7): `string`, `number`, `bigint`, `boolean`, `undefined`, `symbol`, `null`. Primitives are immutable and compared by value. **Reference type:** `object` (includes arrays, functions, dates, maps, sets...). Compared by reference.

```
typeof null;          // 'object'  (historical bug)
typeof function(){};  // 'function'
typeof 10n;           // 'bigint'
Array.isArray([]);    // true — the reliable array check

0.1 + 0.2 === 0.3;                    // false (IEEE-754 double)
Math.abs(0.1 + 0.2 - 0.3) < Number.EPSILON; // true
Number.MAX_SAFE_INTEGER;              // 9007199254740991 → use BigInt beyond
NaN === NaN;                          // false → use Number.isNaN(x)
Object.is(NaN, NaN);                  // true
Object.is(0, -0);                     // false
```

Type Coercion and Equality

`==` performs coercion; `===` does not. Always prefer `===` except `x == null` (checks both `null` and `undefined`).

```
'5' + 3      // '53'  (+ with a string concatenates)
'5' - 3      // 2     (- coerces to number)
[] + {}      // '[object Object]'
[] == false  // true  ([] → '' → 0, false → 0)
null == 0    // false (null only == undefined)
!!''         // false

// Falsy values: false, 0, -0, 0n, '', null, undefined, NaN — everything else is truthy
```

Copying: Shallow vs Deep

```
const original = { a: 1, nested: { b: 2 } };

const shallow = { ...original };          // or Object.assign({}, original)
shallow.nested.b = 99;                    // also changes original.nested.b!

const deep = structuredClone(original);   // modern deep clone (handles Map, Set, Date, cycles)
// JSON.parse(JSON.stringify(o)) loses functions, undefined, Dates, Maps
```

1.4 Functions

UNDERSTAND THE CONCEPT

A function is a reusable block of code with a name. You define it once and *call* (run) it as many times as you want. It can take **parameters** (inputs) and **return** a value (output).

```
function greet(name) {        // `name` is a parameter
  return 'Hello ' + name;     // `return` sends a value back to the caller
}
const message = greet('Anish'); // 'Anish' is the argument → message = 'Hello Anish'
```

Parameters vs arguments: parameters are the placeholder names in the definition. Arguments are the actual values passed when calling.

Three ways to write a function:

1. **Function declaration:** `function add(a, b) { ... }`. It is hoisted, so you can call it before the line where it is defined.
2. **Function expression:** `const add = function (a, b) { ... };`. The function is stored in a variable.
3. **Arrow function:** `const add = (a, b) => a + b;`. A short syntax, and for one-line bodies the result is returned automatically. Arrow functions do not have their own `this` (see 1.5).

Default parameters give a value when the caller passes nothing. **Rest parameters** (`...nums`) collect any number of arguments into an array. **Spread** (`...array`) does the opposite: it expands an array into separate arguments.

First-class and higher-order functions. In JavaScript, functions are values, just like numbers. A **higher-order function** takes a function as input or returns one. You already use these: `array.map(fn)`, `setTimeout(fn, 1000)` and `button.addEventListener('click', fn)`. A function passed in to be called later is a **callback**.

Pure function: given the same inputs, it always returns the same output and changes nothing outside itself. Pure functions are easy to test and reason about.

```
// Declaration
function add(a, b = 0) { return a + b; }    // default parameter

// Expression & arrow
const mul = (a, b) => a * b;

// Rest & spread
function sum(...nums) { return nums.reduce((acc, n) => acc + n, 0); }
sum(...[1, 2, 3]); // 6

// IIFE
(function () { /* private scope */ })();

// Higher-order function
const withLogging = (fn) => (...args) => {
  console.log('calling with', args);
  return fn(...args);
};
```

Arrow functions vs regular functions:

- No own `this` — they capture `this` lexically from the enclosing scope.
- No `arguments` object, cannot be used with `new`, no `prototype`.
- Ideal for callbacks; avoid as object methods that need `this`.

1.5 `this` Binding Rules

UNDERSTAND THE CONCEPT

`this` is a special keyword that refers to **the object that is "running" the function right now**. Unlike normal variables, its value is **not** fixed where the function is written. It is decided by **how the function is called**.

An analogy: the word "me" depends on who says it. When Anish says "me", it means Anish. When Ravi says the same word, it means Ravi. `this` works the same way.

The rules, in order:

1. Called with `new`, e.g. `new User()`: `this` is the brand-new object being created.
2. Called with `call` / `apply` / `bind`: `this` is whatever object you explicitly pass in.
3. Called as a **method**, `obj.method()`: `this` is the object before the dot (`obj`).
4. Called as a **plain function**, `method()`: `this` is `undefined` in strict mode, or the global object otherwise.
5. **Arrow functions** have no `this` of their own. They use the `this` of the code around them. That is why arrows are handy inside callbacks.

The most common bug is taking a method out of its object, e.g. `const f = user.greet; f();`. There is no longer a "dot" in front, so `this` is lost. `bind` fixes this by creating a copy of the function whose `this` is permanently set.

- `fn.call(obj, a, b)` runs immediately, with arguments listed one by one.
- `fn.apply(obj, [a, b])` runs immediately, with arguments passed as an array.
- `fn.bind(obj)` does **not** run. It returns a new function with `this` locked.

Precedence (highest first):

1. `new` **binding** — `new Foo()` → `this` is the new object.
2. **Explicit** — `call`, `apply`, `bind`.
3. **Implicit** — `obj.method()` → `this` is `obj`.
4. **Default** — standalone call → `undefined` in strict mode, `globalThis` otherwise.
5. Arrow functions ignore all rules and use lexical `this`.

```
const user = {
  name: 'Anish',
  greet() { return `Hi ${this.name}`; },
  greetLater() { setTimeout(() => console.log(this.name), 0); }, // arrow keeps this
};

const g = user.greet;
g();                        // 'Hi undefined' — lost implicit binding
g.call({ name: 'Ravi' });   // 'Hi Ravi'
g.apply({ name: 'Ravi' }, []);
const bound = g.bind(user); // permanently bound
bound();                    // 'Hi Anish'
```

Polyfill for `bind` (common interview question)

```
Function.prototype.myBind = function (ctx, ...preset) {
  const fn = this;
  return function (...later) {
    return fn.apply(ctx, [...preset, ...later]);
  };
};
```

1.6 Closures

UNDERSTAND THE CONCEPT

A closure is a function that remembers the variables from the place where it was created, even after that place has finished running.

Normally, when a function finishes, its local variables are thrown away. But if an inner function is still using them, for example because it was returned or saved as a callback, JavaScript keeps those variables alive for it. Think of it as a "backpack" of variables the inner function carries around.

```
function makeGreeter(greeting) {      // outer function
  return function (name) {            // inner function "closes over" greeting
    return greeting + ', ' + name;
  };
}
const hello = makeGreeter('Hello');   // makeGreeter has finished...
hello('Anish');                       // ...but 'Hello' is remembered → 'Hello, Anish'
```

Why closures matter:

- **Private data:** variables inside the outer function cannot be touched from outside, only through the functions you return (see the counter example below).
- **Function factories:** create customised functions (`makeGreeter('Hi')` , `makeGreeter('Namaste')`).
- **Callbacks and timers** remember the data they need.
- **React hooks** are built on closures. Each render's event handlers "see" the state of that render.

Currying turns a function of several arguments into a chain of one-argument functions: `add(1)(2)(3)` . It is used to pre-fill some arguments.

Debounce waits until the user *stops* doing something (typing in a search box) before running the function once. **Throttle** runs the function at most once per time period (for example, during scrolling). Both use a closure to remember the timer between calls.

A **closure** is a function bundled with references to its surrounding lexical environment. The inner function keeps access to outer variables even after the outer function returns.

```
function createCounter() {
  let count = 0;                 // private state
  return {
    inc: () => ++count,
    dec: () => --count,
    get: () => count,
  };
}
const c = createCounter();
c.inc(); c.inc();
c.get(); // 2

// Memoization with closures
function memoize(fn) {
  const cache = new Map();
```

```
  return function (...args) {
    const key = JSON.stringify(args);
    if (!cache.has(key)) cache.set(key, fn.apply(this, args));
    return cache.get(key);
  };
}

// once()
function once(fn) {
  let called = false, result;
  return (...args) => (called ? result : (called = true, result = fn(...args)));
}
```

Uses: data privacy, factories, memoization, currying, debouncing, React hooks (each render closes over its props/state — the root of the "stale closure" bug).

Currying and Partial Application

```
const curry = (fn) =>
  function curried(...args) {
    return args.length >= fn.length
      ? fn.apply(this, args)
      : (...next) => curried.apply(this, [...args, ...next]);
  };

const add3 = curry((a, b, c) => a + b + c);
add3(1)(2)(3); // 6
add3(1, 2)(3); // 6
```

Debounce and Throttle

```
function debounce(fn, delay) {
  let timer;
  return function (...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn.apply(this, args), delay);
  };
}

function throttle(fn, interval) {
  let last = 0;
  return function (...args) {
    const now = Date.now();
    if (now - last >= interval) {
      last = now;
      fn.apply(this, args);
    }
  };
}
// debounce → search box (fire after the user stops typing)
// throttle → scroll/resize handlers (fire at most every N ms)
```

1.7 Objects and Prototypes

UNDERSTAND THE CONCEPT

An object is a collection of named values called properties. When a property holds a function, it is called a **method**.

```
const user = {
  name: 'Anish',          // property
  age: 30,
  greet() {               // method
    return 'Hi, I am ' + this.name;
  },
};
user.name;          // dot notation → 'Anish'
user['age'];        // bracket notation (useful with variable keys)
user.city = 'Pune'; // add a property
delete user.age;    // remove a property
```

Prototypes are how objects inherit in JavaScript. Every object has a hidden link to another object, its **prototype**. When you ask for a property the object doesn't have, JavaScript automatically looks in the prototype, then in the prototype's prototype, and so on. This lookup path is the **prototype chain**. It ends at `Object.prototype`, whose prototype is `null`.

This is why every array has `.map()` and `.push()` even though you never wrote them. They live on `Array.prototype`, which all arrays link to. Methods are stored once and *shared* by all objects, which saves memory.

Constructor functions plus the `new` keyword were the original way to create many similar objects. `new` creates an empty object, links it to the constructor's `prototype`, runs the constructor with `this` pointing at the new object, and returns it. ES6 classes (next section) do exactly this with cleaner syntax.

Getters and setters look like normal properties but run a function when read or written. They are useful for computed values and validation.

Every object has an internal `[[Prototype]]` link. Property lookup walks the **prototype chain** until `null`.

```
const animal = { eats: true, walk() { return 'walking'; } };
const dog = Object.create(animal);   // dog.[[Prototype]] = animal
dog.barks = true;
dog.walk();                          // 'walking' (found on prototype)
Object.getPrototypeOf(dog) === animal; // true
dog.hasOwnProperty('eats');          // false
Object.hasOwn(dog, 'barks');         // true (modern)

// Constructor function (pre-ES6 pattern)
function Person(name) { this.name = name; }
Person.prototype.hello = function () { return `I am ${this.name}`; };
const p = new Person('A');
p.__proto__ === Person.prototype;    // true
```

What `new` does: (1) creates an empty object, (2) links its prototype to `Constructor.prototype`, (3) calls the constructor with `this` = new object, (4) returns the object (unless the constructor returns another object).

Object utilities

```
const o = { a: 1, b: 2 };
Object.keys(o);      // ['a','b']
Object.values(o);    // [1,2]
```

```
Object.entries(o);   // [['a',1],['b',2]]
Object.fromEntries([['x', 1]]); // { x: 1 }

Object.freeze(o);    // shallow: no add/remove/change
Object.seal(o);      // no add/remove, can change existing

Object.defineProperty(o, 'secret', {
  value: 42, writable: false, enumerable: false, configurable: false,
});

// Getters / setters
const temp = {
  _c: 0,
  get f() { return this._c * 9 / 5 + 32; },
  set f(v) { this._c = (v - 32) * 5 / 9; },
};
```

1.8 Classes (ES6+)

UNDERSTAND THE CONCEPT

A class is a blueprint for creating objects. It bundles data (fields) and behaviour (methods) into one unit. Each object made from a class is an **instance**. For example, the `Car` class is the design, and your car and my car are instances.

Main ideas of **Object-Oriented Programming (OOP)**:

- **Encapsulation:** keep data and the code that manages it together, and hide internal details. In JS, fields that start with `#` are truly private.
- **Inheritance:** a class can `extend` another and reuse its code. `SavingsAccount extends Account` means "a savings account *is an* account, with extras".
- **Polymorphism:** different classes can provide their own version of the same method, and calling code does not need to know which one it has.
- **Abstraction:** expose a simple interface (`deposit()`) and hide the complexity.

Parts of a class:

- **`constructor`** runs automatically when you call `new ClassName(...)` . It sets up the initial data.
- **Methods** are functions available on every instance.
- **`static`** members belong to the class itself, not to instances (e.g. `Account.count`).
- **`super(...)`** calls the parent class's constructor. **`super.method()`** calls the parent's version of a method.
- **getters/setters** (`get balance()`) let you read computed values like a property.

Under the hood, classes still use prototypes. They are "syntactic sugar" that makes prototypes easier to write.

Classes are syntactic sugar over prototypes, with some extras (private fields, static blocks).

```
class Account {
  static #count = 0;          // private static
  #balance = 0;               // private instance field
```

```
  constructor(owner) {
    this.owner = owner;
    Account.#count++;
  }

  deposit(amount) {
    if (amount <= 0) throw new RangeError('Amount must be positive');
    this.#balance += amount;
    return this;               // enables chaining
  }

  get balance() { return this.#balance; }

  static get count() { return Account.#count; }
}

class SavingsAccount extends Account {
  constructor(owner, rate) {
    super(owner);              // must call before using `this`
    this.rate = rate;
  }
  addInterest() {
    return this.deposit(this.balance * this.rate);
  }
}

const s = new SavingsAccount('Anish', 0.05).deposit(1000).addInterest();
s.balance;         // 1050
Account.count;     // 1
// s.#balance      // SyntaxError — truly private
```

1.9 Arrays — Essential Methods

UNDERSTAND THE CONCEPT

An array is an ordered list of values. Each value sits at a numbered position called its **index**, starting at **0**.

```
const fruits = ['apple', 'banana', 'cherry'];
fruits[0];          // 'apple'
fruits.length;      // 3
fruits.push('kiwi');// add to end
fruits.pop();       // remove from end
for (const f of fruits) console.log(f);   // loop over values
```

Modern JavaScript prefers **array methods** to hand-written loops. Each takes a *callback* function that runs for every element:

Method	What it does	Returns
`map`	Transforms every item	New array, same length
`filter`	Keeps items where the callback returns true	New, possibly shorter array
`find`	First item matching a condition	One item or `undefined`
`some` / `every`	Does *any* / *all* match?	`true` / `false`
`reduce`	Combines all items into one value (sum, object, ...)	Anything
`forEach`	Runs code for each item	Nothing (`undefined`)
`sort`	Orders items (**changes the original!**)	Same array

`reduce` takes the most practice. It carries an **accumulator** (a running result) through the array: `[1,2,3].reduce((total, n) => total + n, 0)` goes 0 → 1 → 3 → 6.

Mutating vs non-mutating: methods like `push`, `sort`, `splice` and `reverse` *change* the original array. `map`, `filter`, `slice`, `toSorted` and `toReversed` return *new* arrays and leave the original alone. React and Redux require the non-mutating style.

```
const users = [
  { id: 1, name: 'A', age: 30, role: 'admin' },
  { id: 2, name: 'B', age: 22, role: 'user' },
  { id: 3, name: 'C', age: 35, role: 'user' },
];

users.map(u => u.name);                   // ['A','B','C']
users.filter(u => u.age > 25);            // A, C
users.find(u => u.id === 2);              // B
users.findIndex(u => u.id === 2);         // 1
users.findLast(u => u.role === 'user');   // C (ES2023)
users.some(u => u.role === 'admin');      // true
users.every(u => u.age > 18);             // true
users.reduce((sum, u) => sum + u.age, 0); // 87

// Group by (ES2024)
Object.groupBy(users, u => u.role); // { admin: [A], user: [B, C] }

// Group by with reduce (works everywhere)
const byRole = users.reduce((acc, u) => {
  (acc[u.role] ??= []).push(u);
  return acc;
}, {});

// Non-mutating versions (ES2023)
const sorted = users.toSorted((a, b) => a.age - b.age);
const reversed = users.toReversed();
const replaced = users.with(0, { ...users[0], name: 'Z' });

// Mutating: push, pop, shift, unshift, splice, sort, reverse, fill
[3, 1, 10].sort();          // [1, 10, 3]  — default sort is lexicographic!
[3, 1, 10].sort((a, b) => a - b); // [1, 3, 10]

[[1, [2]], [3]].flat(Infinity);   // [1, 2, 3]
['a b', 'c'].flatMap(s => s.split(' ')); // ['a','b','c']
Array.from({ length: 3 }, (_, i) => i * 2); // [0,2,4]
[...new Set([1, 1, 2])];          // [1, 2] — dedupe
[1, 2, 3].at(-1);                 // 3
```

Implement `map` and `reduce` polyfills

```
Array.prototype.myMap = function (cb, thisArg) {
  const out = [];
  for (let i = 0; i < this.length; i++) {
    if (i in this) out[i] = cb.call(thisArg, this[i], i, this);
  }
  return out;
};

Array.prototype.myReduce = function (cb, init) {
  let i = 0, acc = init;
  if (arguments.length < 2) {
    if (!this.length) throw new TypeError('Reduce of empty array with no initial value');
    acc = this[0]; i = 1;
  }
  for (; i < this.length; i++) acc = cb(acc, this[i], i, this);
```

```
  return acc;
};
```

1.10 Modern Syntax (ES6 → ES2024)

UNDERSTAND THE CONCEPT

The JavaScript language is updated every year. The standard is called **ECMAScript**, so "ES6" means the 2015 edition and "ES2024" the 2024 edition. ES6 was the big modernisation. These features make code shorter and safer:

- **Destructuring:** unpack values from arrays or objects into variables in one line: `const { name, age } = user;` `const [first, second] = list;`
- **Spread `...`:** copy or merge, e.g. `{ ...user, age: 31 }` creates a new object with one field changed.
- **Template literals:** strings in backticks with `${expression}` placeholders, which can span multiple lines.
- **Optional chaining `?.`:** safely read deep properties. `user?.address?.city` returns `undefined` instead of crashing if `address` is missing.
- **Nullish coalescing `??`:** a default value only when the left side is `null` / `undefined`. Compare `||`, which also replaces `0` and `''`. That is why `0 ?? 10` is `0` but `0 || 10` is `10`.
- **Shorthand properties:** `{ name }` instead of `{ name: name }`.

```
// Destructuring with defaults, renaming, rest
const { name: userName = 'guest', ...rest } = { name: 'A', age: 1, role: 'x' };
const [first, , third = 0] = [1, 2];

// Optional chaining & nullish coalescing
const city = user?.address?.city ?? 'Unknown';  // ?? only falls back for null/undefined
const count = 0 || 10;   // 10
const count2 = 0 ?? 10;  // 0

// Logical assignment
opts.timeout ??= 3000;
opts.retries ||= 3;
opts.enabled &&= isAllowed;

// Template literals & tagged templates
const tag = (strings, ...vals) => strings.raw.join('|') + vals.join(',');
tag`a${1}b${2}`;

// Computed keys & shorthand
const key = 'dynamic';
const o2 = { [key]: 1, userName };

// Numeric separators, exponent
const big = 1_000_000;  2 ** 10;

// Symbol — unique keys, well-known symbols
const id = Symbol('id');
const objWithSym = { [id]: 123 };
```

1.11 Map, Set, WeakMap, WeakSet

UNDERSTAND THE CONCEPT

Besides plain objects and arrays, JavaScript has specialised collections:

- **`Map`:** a key-value store like an object, but keys can be **any type** (even objects). It remembers insertion order and has a `.size`. Use it as a dictionary or cache.
- **`Set`:** a collection of **unique** values. Adding the same value twice keeps one copy. Use it to remove duplicates or for fast "have I seen this?" checks.
- **`WeakMap` / `WeakSet`:** like Map and Set, but keys must be objects and are held *weakly*. If nothing else references the key object, it can be garbage-collected (removed from memory) automatically. Use them to attach extra data to objects without causing memory leaks.

Rule of thumb: plain objects for fixed-shape records (`{ id, name }`), and `Map` for dynamic dictionaries where keys are added and removed often.

```
const m = new Map([['a', 1]]);
m.set({ k: 1 }, 'object key OK');  // any key type, keeps insertion order
m.size;

const s = new Set([1, 2, 2]);       // unique values
s.has(2);
// ES2025 Set methods: union, intersection, difference, isSubsetOf...
new Set([1, 2]).union(new Set([2, 3])); // Set {1,2,3}

// WeakMap: keys must be objects, held weakly (garbage-collectable), not iterable
const privateData = new WeakMap();
class Thing {
  constructor() { privateData.set(this, { secret: 1 }); }
}
```

When to use Map over Object: frequent add/remove, non-string keys, need size/order, avoiding prototype key collisions.

1.12 Iterators and Generators

UNDERSTAND THE CONCEPT

Iteration means going through items one by one. Arrays, strings, Maps and Sets are **iterable**: they follow a standard *iterator protocol*, so `for...of` and spread `[...x]` work on them. An **iterator** is an object with a `next()` method that returns `{ value, done }` each time it is called.

Generators (`function*`) are functions that can **pause and resume**. Each `yield` hands out one value and freezes the function until someone asks for the next value. This is useful for:

- **Lazy sequences:** produce values only when needed, even an infinite sequence of IDs.

- **Large data:** process a huge file or paginated API one piece at a time without loading everything into memory.

```
function* threeSteps() {
  yield 'step 1';     // pause here
  yield 'step 2';     // pause here
  return 'done';
}
const g = threeSteps();
g.next();  // { value: 'step 1', done: false }
g.next();  // { value: 'step 2', done: false }
g.next();  // { value: 'done', done: true }
```

Async generators (`async function*` with `for await...of`) do the same for data that arrives over time, like pages from an API.

```
// Custom iterable
const range = {
  from: 1, to: 4,
  [Symbol.iterator]() {
    let cur = this.from, last = this.to;
    return { next: () => (cur <= last ? { value: cur++, done: false } : { value: undefined,
done: true }) };
  },
};
[...range]; // [1,2,3,4]

// Generator — pausable function
function* idGen() {
  let id = 0;
  while (true) {
    const reset = yield ++id;
    if (reset) id = 0;
  }
}
const gen = idGen();
gen.next().value;      // 1
gen.next().value;      // 2
gen.next(true).value;  // 1

// Async generator — paginate an API
async function* fetchPages(url) {
  let next = url;
  while (next) {
    const res = await fetch(next).then(r => r.json());
    yield res.items;
    next = res.nextPage;
  }
}
for await (const items of fetchPages('/api/items?page=1')) console.log(items);
```

1.13 The Event Loop

UNDERSTAND THE CONCEPT

Problem: JavaScript has only one thread, which means it can do only one thing at a time. If it waited (blocked) for every network request, the page would freeze. **Solution:** the **event loop**.

Think of a restaurant with **one chef** (the JavaScript thread):

- The chef cooks one dish at a time. That is the **call stack**, the code currently running.
- Slow jobs like baking (timers, network requests, file reads) are handed to **helpers** (the browser or Node's background system). The chef does not wait for them.
- When a helper finishes, they put a ticket in a **queue**.
- Whenever the chef's hands are free (the call stack is empty), the chef takes the next ticket. That repeating check is the **event loop**.

There are two queues, and one has priority:

1. **Microtask queue (VIP):** promise callbacks (`.then` , code after `await`). It is emptied *completely* after each task.
2. **Macrotask/task queue (regular):** `setTimeout` , `setInterval` , clicks, I/O.

So the order is: run all synchronous code → run all microtasks → run one macrotask → run all microtasks again → and so on. This is why `Promise.then` runs **before** `setTimeout(..., 0)` even though the timeout is "0 ms". Note that `setTimeout(fn, 0)` means "*at least* 0 ms, after the current work", not "immediately".

JavaScript has one call stack. Async work is delegated to the host (Web APIs / libuv); completed callbacks are queued.

- **Call stack** — synchronous code.
- **Microtask queue** — Promise callbacks (`then/catch/finally`), `queueMicrotask` , `MutationObserver` , `await` continuations.
- **Macrotask (task) queue** — `setTimeout` , `setInterval` , I/O, UI events, `MessageChannel` .

Algorithm: run one macrotask → drain *all* microtasks → render (browser) → next macrotask.

```
console.log('1 sync');
setTimeout(() => console.log('2 timeout'), 0);
Promise.resolve().then(() => console.log('3 micro'));
queueMicrotask(() => console.log('4 micro'));
(async () => {
  console.log('5 async start (sync)');
  await null;
  console.log('6 after await (micro)');
})();
console.log('7 sync');

// Output: 1, 5, 7, 3, 4, 6, 2
```

1.14 Promises and Async/Await

UNDERSTAND THE CONCEPT

A Promise is an object representing a value that will be available later, such as the result of a network request. It is like the buzzer you get at a food court: you don't have the food yet, but you have something that will tell you when it is ready, or that something went wrong.

A promise is always in one of three **states**:

1. **Pending:** still waiting.
2. **Fulfilled (resolved):** success, with a value.
3. **Rejected:** failure, with an error (the reason).

Once fulfilled or rejected, it is **settled** and never changes again.

Before promises, async code used nested callbacks, which made "callback hell" (pyramids of indentation that were hard to read and to handle errors in). Promises let you chain steps with `.then()` and handle all errors in one `.catch()`.

`async` / `await` is a cleaner syntax on top of promises:

- Marking a function `async` makes it always return a promise.
- Inside it, `await somePromise` *pauses that function* (not the whole program) until the promise settles, then gives you the value. If the promise rejects, `await` throws an error you can catch with `try/catch`.

```
async function showUser() {
  try {
    const response = await fetch('/api/user/1'); // wait for the network
    const user = await response.json();          // wait for the body to be parsed
    console.log(user.name);
  } catch (error) {
    console.log('Something failed:', error.message);
  }
}
```

Running things in parallel: `await` one after another runs tasks *sequentially*. If tasks are independent, start them all and wait with `Promise.all([...])`. That is much faster.

A Promise is in one of three states: **pending**, **fulfilled**, **rejected**. Once settled it is immutable.

```
const wait = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

function getUser(id) {
  return new Promise((resolve, reject) => {
    if (!id) return reject(new Error('id required'));
    setTimeout(() => resolve({ id, name: 'A' }), 100);
  });
}

getUser(1)
  .then(u => u.name)
  .then(console.log)
  .catch(console.error)
  .finally(() => console.log('done'));

// Combinators
await Promise.all([p1, p2]);        // all fulfill, or reject on first rejection
await Promise.allSettled([p1, p2]); // [{status:'fulfilled',value}, {status:'rejected',reason}]
await Promise.race([p1, timeout]);  // first to settle
await Promise.any([p1, p2]);        // first to fulfill; AggregateError if all reject

// Promise.withResolvers (ES2024)
const { promise, resolve, reject } = Promise.withResolvers();
```

async/await patterns

```
// Sequential (slow) vs parallel (fast)
async function sequential() {
  const a = await fetchA();  // waits
  const b = await fetchB();  // then waits
  return [a, b];
}
async function parallel() {
  const [a, b] = await Promise.all([fetchA(), fetchB()]);
  return [a, b];
}

// Error handling
async function load() {
  try {
    const res = await fetch('/api');
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    return await res.json();
  } catch (err) {
    console.error(err);
    throw err;
  }
}

// Timeout + abort
async function fetchWithTimeout(url, ms = 5000) {
  const controller = new AbortController();
  const id = setTimeout(() => controller.abort(), ms);
  try {
    return await fetch(url, { signal: controller.signal });
  } finally {
    clearTimeout(id);
  }
}
// or: fetch(url, { signal: AbortSignal.timeout(5000) })

// Retry with exponential backoff
async function retry(fn, retries = 3, delay = 500) {
  try {
    return await fn();
  } catch (e) {
    if (retries === 0) throw e;
    await wait(delay);
    return retry(fn, retries - 1, delay * 2);
  }
}

// Concurrency limit (run N at a time)
async function mapLimit(items, limit, fn) {
  const results = [];
  let i = 0;
  const workers = Array.from({ length: limit }, async () => {
    while (i < items.length) {
      const idx = i++;
      results[idx] = await fn(items[idx]);
    }
  });
  await Promise.all(workers);
  return results;
}
```

Implement `Promise.all` (interview favourite)

```
function promiseAll(promises) {
  return new Promise((resolve, reject) => {
    const results = [];
```

```
    let done = 0;
    if (promises.length === 0) return resolve(results);
    promises.forEach((p, i) => {
      Promise.resolve(p).then((val) => {
        results[i] = val;
        if (++done === promises.length) resolve(results);
      }, reject);
    });
  });
}
```

1.15 Error Handling

UNDERSTAND THE CONCEPT

Errors happen when something goes wrong while the program runs: calling a function that doesn't exist, bad input, a failed network request. If nothing handles an error, the program (or that piece of it) stops.

- **`throw`** creates (raises) an error yourself: `throw new Error('Price must be positive')`.
- **`try { } catch (e) { }`** runs risky code in `try`. If an error is thrown, control jumps to `catch` where you deal with it.
- **`finally { }`** always runs, error or not. Use it for clean-up such as hiding a loading spinner or closing a connection.

Built-in error types tell you what went wrong: `ReferenceError` (variable doesn't exist), `TypeError` (wrong kind of value, e.g. calling `undefined()`), `SyntaxError` (invalid code), `RangeError` (number out of range).

You can create **custom error classes** by extending `Error`. Callers can then check `if (e instanceof ValidationError)` and react differently to different problems.

Good practice: catch errors where you can actually *do something* about them (show a message, retry, use a fallback). Otherwise let them travel upward. Never silently swallow errors with an empty `catch {}`.

```
class ValidationError extends Error {
  constructor(field, message) {
    super(message);
    this.name = 'ValidationError';
    this.field = field;
  }
}

try {
  throw new ValidationError('email', 'Invalid email');
} catch (e) {
  if (e instanceof ValidationError) console.log(e.field, e.message);
  else throw e;
} finally {
  // always runs
}

// Error cause (ES2022)
throw new Error('Failed to save user', { cause: originalErr });
```

```
// Global handlers
window.addEventListener('unhandledrejection', (e) => console.error(e.reason));
```

1.16 Modules

UNDERSTAND THE CONCEPT

A module is a separate file whose variables are private by default. You choose what to share with `export` and pull it into another file with `import`. Modules let you split a large program into small, focused files, avoid name clashes, and reuse code.

There are two module systems you will meet:

- **ES Modules (ESM):** the official standard (`import` / `export`), used in browsers, React projects and modern Node.js. Imports are analysed *before* the code runs. That lets bundlers drop unused code, which is called **tree-shaking**.
- **CommonJS (CJS):** Node.js's original system (`require()` / `module.exports`). It is still very common in older Node projects.

Named vs default exports: a file can have many *named* exports (`export const PI`), imported with braces `{ PI }`. It can have one *default* export, imported with any name you like and no braces.

Dynamic import, `await import('./file.js')`, loads a module only when needed. React uses this for *code splitting* (loading a page's code only when the user visits that page).

```
// ES Modules (static, analyzable, tree-shakable, async loading)
export const PI = 3.14;
export default function area(r) { return PI * r * r; }
import area, { PI } from './math.js';
import * as math from './math.js';

// Dynamic import → code splitting
const { default: chart } = await import('./chart.js');

// CommonJS (Node legacy: synchronous, dynamic)
const fs = require('fs');
module.exports = { helper };
```

ESM	CommonJS
`import/export`	`require/module.exports`
Static, hoisted, tree-shakable	Dynamic, runtime
Live bindings	Copies of exported values
Top-level `await` allowed	No top-level await
`this` is `undefined` at top level	`this` is `module.exports`

1.17 Memory Management and Performance

UNDERSTAND THE CONCEPT

Programs need memory to store their values. In JavaScript you never free memory yourself. A **garbage collector (GC)** automatically finds values that can no longer be reached from your code and frees them.

Reachability: a value is kept alive if it can be reached from a "root" (global variables, the currently running functions) through a chain of references. Everything else is garbage. The common algorithm is **mark-and-sweep**: *mark* everything reachable, then *sweep away* everything unmarked.

A **memory leak** happens when you *accidentally* keep references to things you no longer need, so the GC can't free them and memory usage keeps growing. Common causes:

- Event listeners or `setInterval` timers that are never removed.
- Global variables or caches that grow forever.
- Closures that capture large objects.
- References to DOM elements that have already been removed from the page.

The fix is usually "clean up after yourself": `removeEventListener`, `clearInterval`, bounded caches, and in React, effect cleanup functions.

- Garbage collection uses **mark-and-sweep** (generational: young/old space in V8).
- **Common leaks:** forgotten timers/intervals, detached DOM nodes, global variables, closures holding large objects, event listeners never removed, unbounded caches (use `WeakMap`/LRU).
- Use `WeakRef` and `FinalizationRegistry` only for advanced caching scenarios.

```
// Leak: listener never removed
function attach() {
  const big = new Array(1e6).fill('*');
  window.addEventListener('resize', () => console.log(big.length));
}
// Fix: keep a reference and removeEventListener, or use AbortController
const ac = new AbortController();
window.addEventListener('resize', handler, { signal: ac.signal });
ac.abort(); // removes listener
```

1.18 Browser and DOM Essentials

UNDERSTAND THE CONCEPT

In the browser, JavaScript interacts with the page through the **DOM (Document Object Model)**, a tree of objects representing every HTML element. Change the tree and the page updates.

```
const title = document.querySelector('h1');      // find an element (CSS selector)
title.textContent = 'New title';                 // change its text
```

```
title.classList.add('highlight');                // add a CSS class
const li = document.createElement('li');         // create a new element
li.textContent = 'Item';
document.querySelector('ul').appendChild(li);    // insert it into the page
```

Events are things that happen: clicks, key presses, form submissions, scrolling. You react to them with `element.addEventListener('click', handler)`. The handler receives an **event object** with details (which element was clicked, mouse position, which key was pressed).

Event propagation: an event starts at the top of the page and travels *down* to the clicked element (the **capture** phase), then travels back *up* through every parent (the **bubble** phase). Bubbling makes **event delegation** possible: put one listener on a parent instead of hundreds of listeners on children.

Browser storage: `localStorage` keeps small string data permanently in the browser. `sessionStorage` keeps it until the tab is closed. **Cookies** are small values that are automatically sent to the server with each request, often used for login sessions.

Web security basics: XSS (Cross-Site Scripting) is when an attacker gets their script to run on your page, often by injecting HTML. Never insert untrusted text as HTML. **CSRF** tricks a logged-in user's browser into sending an unwanted request. **CORS** is the browser rule that controls which other websites may read responses from your server.

```
// Event delegation — one listener for many children
document.querySelector('#list').addEventListener('click', (e) => {
  const item = e.target.closest('li[data-id]');
  if (item) console.log('clicked', item.dataset.id);
});

// Phases: capture (top→down) → target → bubble (bottom→up)
el.addEventListener('click', fn, { capture: true, once: true, passive: true });
e.stopPropagation();  // stop bubbling
e.preventDefault();   // stop default action (form submit, link nav)

// Intersection Observer — lazy loading / infinite scroll
const io = new IntersectionObserver((entries) => {
  entries.forEach((en) => en.isIntersecting && loadMore());
});
io.observe(document.querySelector('#sentinel'));
```

Storage: `localStorage` (persistent, ~5MB, sync, strings only), `sessionStorage` (per tab), cookies (sent with requests; use `HttpOnly`, `Secure`, `SameSite`), IndexedDB (large structured async storage).

Rendering: DOM + CSSOM → render tree → layout (reflow) → paint → composite. Avoid layout thrashing (reading `offsetHeight` after writes in a loop); animate `transform` / `opacity`.

Security: XSS (never inject untrusted HTML; use `textContent`; CSP), CSRF (SameSite cookies, CSRF tokens), CORS (server-side header controlling cross-origin reads).

1.19 Functional Programming Concepts

UNDERSTAND THE CONCEPT

Functional programming (FP) is a style that builds programs from small, **pure** functions and avoids changing data in place. JavaScript supports FP very well because functions are values.

Core ideas:

- **Pure functions:** output depends only on input, with no side effects (no changing global variables, no network calls). They are easy to test: same input, same output, every time.
- **Immutability:** instead of modifying an object, create a new one with the change (`{ ...user, age: 31 }`). This prevents bugs where one part of the code unexpectedly changes data another part relies on. React depends on this to detect changes.
- **Composition:** combine small functions into bigger ones, like LEGO bricks. `pipe(trim, toLowerCase, replaceSpaces)` runs each function on the result of the previous one.
- **Declarative over imperative:** describe *what* you want (`users.filter(u => u.active)`) rather than *how*, step by step (a manual `for` loop with an index and a push).

- **Pure functions** — same input → same output, no side effects.
- **Immutability** — return new objects instead of mutating.
- **Composition** — build complex functions from small ones.

```
const compose = (...fns) => (x) => fns.reduceRight((v, f) => f(v), x);
const pipe = (...fns) => (x) => fns.reduce((v, f) => f(v), x);

const slugify = pipe(
  (s) => s.trim(),
  (s) => s.toLowerCase(),
  (s) => s.replace(/\s+/g, '-'),
);
slugify('  Hello World '); // 'hello-world'
```

1.20 Design Patterns in JavaScript

UNDERSTAND THE CONCEPT

Design patterns are proven, reusable solutions to problems that come up again and again in software design. They give developers a shared vocabulary ("let's use an observer here").

- **Singleton:** guarantees only **one** instance of something exists, such as a single database connection or a config object.
- **Observer / Pub-Sub:** objects *subscribe* to events and get notified when they happen, like following a channel on YouTube. DOM events, Node's EventEmitter, and Redux store subscriptions all use this.
- **Factory:** a function that decides *which* kind of object to create, so callers don't need to know the details.
- **Module:** hide private details and expose a public API (closures or ES modules).
- **Proxy:** an object that stands in front of another and intercepts operations (validation, logging, reactivity in Vue).

- **Decorator / Higher-order function:** wrap a function to add behaviour (logging, caching) without changing its code.

```
// Singleton (module scope is a natural singleton)
let instance;
export function getDb() { return (instance ??= createConnection()); }

// Observer / Pub-Sub — basis of EventEmitter, Redux subscribe
class EventBus {
  #events = new Map();
  on(evt, fn) {
    if (!this.#events.has(evt)) this.#events.set(evt, new Set());
    this.#events.get(evt).add(fn);
    return () => this.#events.get(evt).delete(fn); // unsubscribe
  }
  emit(evt, ...args) { this.#events.get(evt)?.forEach((fn) => fn(...args)); }
}

// Factory
function createShape(type) {
  const shapes = { circle: () => new Circle(), square: () => new Square() };
  return shapes[type]?.() ?? (() => { throw new Error('Unknown shape'); })();
}

// Proxy — validation, reactivity (Vue 3 uses this)
const validated = new Proxy({}, {
  set(target, prop, value) {
    if (prop === 'age' && !Number.isInteger(value)) throw new TypeError('age must be int');
    target[prop] = value;
    return true;
  },
});
```

1.21 Deep Clone and Deep Equal (Interview Implementations)

UNDERSTAND THE CONCEPT

These are **classic interview exercises**. They test whether you really understand references, recursion and types.

- **Deep clone:** create a completely independent copy of a nested object. The tricky parts are handling nested objects (solved with *recursion*, a function calling itself), special types like `Date` / `Map` / `Set`, and **circular references** (an object that contains itself). Circular references are handled by remembering already-copied objects in a `WeakMap`.
- **Deep equal:** check whether two values have the same content, not just the same reference. `{a:1} === {a:1}` is `false` because they are two different objects in memory.
- **Flatten an object:** turn nested keys into dotted paths. It is useful for forms and logging.

In real projects, use `structuredClone()` for deep copies and a tested library (lodash `isEqual`) for deep equality.

```
function deepClone(value, seen = new WeakMap()) {
  if (value === null || typeof value !== 'object') return value;
  if (seen.has(value)) return seen.get(value);       // handle cycles
  if (value instanceof Date) return new Date(value);
  if (value instanceof Map) return new Map([...value].map(([k, v]) => [k, deepClone(v, seen)]));
  if (value instanceof Set) return new Set([...value].map(v => deepClone(v, seen)));
  const out = Array.isArray(value) ? [] : Object.create(Object.getPrototypeOf(value));
  seen.set(value, out);
  for (const key of Reflect.ownKeys(value)) out[key] = deepClone(value[key], seen);
  return out;
}

function deepEqual(a, b) {
  if (Object.is(a, b)) return true;
  if (typeof a !== 'object' || typeof b !== 'object' || !a || !b) return false;
  const ka = Object.keys(a), kb = Object.keys(b);
  if (ka.length !== kb.length) return false;
  return ka.every((k) => deepEqual(a[k], b[k]));
}

// Flatten an object: { a: { b: 1 } } → { 'a.b': 1 }
function flattenObj(obj, prefix = '', out = {}) {
  for (const [k, v] of Object.entries(obj)) {
    const key = prefix ? `${prefix}.${k}` : k;
    if (v && typeof v === 'object' && !Array.isArray(v)) flattenObj(v, key, out);
    else out[key] = v;
  }
  return out;
}
```

1.22 Quick Revision — Tricky Output Questions

UNDERSTAND THE CONCEPT

These snippets show JavaScript's surprising corners: coercion, sorting, scope and references. For each one, **predict the output before reading the answer**, then reason about *why* using the concepts above. If you can explain every one of them, your fundamentals are solid.

```
// 1
console.log([1, 2, 3] + [4]);       // '1,2,34'
// 2
console.log(typeof typeof 1);       // 'string'
// 3
const x = { a: 1 };
const y = x; y.a = 2;
console.log(x.a);                   // 2
// 4
console.log([10, 1, 3].sort());     // [1, 10, 3]
// 5
let n = 1;
{ let n = 2; }
console.log(n);                     // 1
// 6
console.log(1 < 2 < 3, 3 > 2 > 1);  // true false   (true > 1 → 1 > 1)
// 7
const fn = () => arguments;         // ReferenceError in modules (no own arguments)
// 8
console.log(+'', +' ', +'a');       // 0 0 NaN
// 9
for (const k in [7, 8]) console.log(typeof k); // 'string' 'string' — for...in gives keys
```

Part 2 — TypeScript: Basics to Advanced

UNDERSTAND THE CONCEPT

What is TypeScript? TypeScript (TS) is JavaScript **plus types**. Microsoft created it in 2012. You write `.ts` / `.tsx` files, and the TypeScript **compiler** (`tsc`) checks them for mistakes and then converts them into normal JavaScript, because browsers and Node only understand JS.

What is a "type"? A type describes *what kind of value* something is and what you can do with it: a `number` can be added, a `string` can be upper-cased, a `User` has a `name` and an `email` .

Why bother? In JavaScript, many mistakes show up only when the code runs, sometimes in production in front of users:

```
function total(price, qty) { return price * qty; }
total('10', 2);          // JS happily returns 20... or worse, 'abc' * 2 = NaN
user.nmae;               // typo → undefined, no warning
```

With TypeScript, the editor underlines these errors **as you type**:

```
function total(price: number, qty: number): number { return price * qty; }
total('10', 2);          // ❌ Error: Argument of type 'string' is not assignable to
'number'
```

Benefits: fewer bugs, safe refactoring (rename something and TS shows every place that broke), autocomplete in the editor, and types that work as living documentation. **Important:** types exist only at compile time. They are removed from the final JavaScript and do **not** check data at run time.

2.1 Why TypeScript

UNDERSTAND THE CONCEPT

TypeScript is a **superset** of JavaScript: every valid JavaScript program is also valid TypeScript. You can adopt it gradually, one file at a time.

Type annotations are the `: type` you write after a variable or parameter: `let age: number = 30` . Often you don't need them, because TypeScript **infers** (works out) the type from the value: `let age = 30` is already known to be a `number` .

Structural typing is a key idea. TypeScript compares types by their **shape** (which properties they have), not by their names. If an object has all the properties an interface requires, it is accepted, even if it was never declared as that interface. This matches how JavaScript itself works ("if it walks like a duck and quacks like a duck, it's a duck").

The compile step:

```
app.ts ──(tsc / Vite / esbuild / ts-node)──►  type-check  ──►  app.js (types removed)  ──►
run in browser/Node
```

TypeScript is a **statically typed superset** of JavaScript that compiles (transpiles) to plain JS. Types are erased at runtime — they exist only for the compiler and editor. Benefits: early error detection, refactoring safety, self-documenting APIs, excellent IntelliSense.

Structural typing ("duck typing"): compatibility is based on shape, not on declared names.

```ts
interface Point { x: number; y: number }
const p = { x: 1, y: 2, z: 3 };
const q: Point = p;            // OK — p has at least x and y
// const r: Point = { x: 1, y: 2, z: 3 }; // Error: excess property check on literals
```

2.2 Key `tsconfig.json` Options

UNDERSTAND THE CONCEPT

`tsconfig.json` is the configuration file that tells the TypeScript compiler how strict to be and what JavaScript to produce. Every TS project has one at its root.

The most important settings for a beginner:

- **`strict: true`** turns on all the safety checks. Always use it in new projects.
- **`strictNullChecks`** (part of strict) means `null` and `undefined` are not silently allowed everywhere, which prevents the famous "cannot read property of undefined" crash.
- **`noImplicitAny`** makes TS report an error when it can't work out a type, instead of silently giving up and using `any`.
- **`target`** sets which JavaScript version to output (e.g. ES2022).
- **`module`** sets which module system to output (ESM or CommonJS).
- **`outDir` / `rootDir`** set where compiled files go and where source files live.
- **`jsx`** controls how React's JSX is compiled.
- **`paths`** adds import shortcuts such as `@/components/Button`.

```jsonc
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "NodeNext",
    "moduleResolution": "NodeNext",
    "strict": true,                  // enables all strict checks below
    "noImplicitAny": true,
    "strictNullChecks": true,
    "noUncheckedIndexedAccess": true, // arr[i] is T | undefined
    "exactOptionalPropertyTypes": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "outDir": "dist",
    "rootDir": "src",
    "declaration": true,
    "sourceMap": true,
    "jsx": "react-jsx",
```

```
    "paths": { "@/*": ["./src/*"] }
  }
}
```

2.3 Basic Types

UNDERSTAND THE CONCEPT

These are the building blocks. You attach them with a colon after a name.

- **`number`, `string`, `boolean`:** the everyday primitives.
- **Arrays:** `number[]` or `Array<number>`, a list where every item is a number.
- **Tuples:** `[string, number]`, a fixed-length array where each position has a specific type (e.g. `['age', 30]`).
- **`any`:** "turn off type checking for this value". It is an escape hatch. Avoid it, because it spreads and hides bugs.
- **`unknown`:** "could be anything, but you must check before using it". It is the *safe* version of `any`, and the right type for data whose shape you don't know yet (like JSON from an API).
- **`void`:** the function returns nothing useful.
- **`never`:** a value that can never exist, for example the return type of a function that always throws, or the leftover case after you've handled every possibility.
- **`null` / `undefined`:** the empty values. With strict mode you must include them explicitly, e.g. `string | null`.
- **Literal types:** an exact value as a type, e.g. `'up' | 'down'` allows only those two strings.

Union types (`A | B`) mean "either A or B": `let id: string | number`.

```ts
let id: number = 1;
let name: string = 'Anish';
let active: boolean = true;
let big: bigint = 100n;
let nothing: null = null;
let notSet: undefined = undefined;

let list: number[] = [1, 2];
let list2: Array<string> = ['a'];
let tuple: [string, number] = ['age', 30];
let namedTuple: [name: string, age?: number] = ['A'];
let readonlyArr: readonly number[] = [1, 2];

let anything: any = 4;       // opts out of type checking — avoid
let safe: unknown = 4;       // must narrow before use — prefer over any
if (typeof safe === 'number') safe.toFixed(2);

function fail(msg: string): never { throw new Error(msg); } // never returns
function log(msg: string): void { console.log(msg); }

// Literal types
let dir: 'up' | 'down' = 'up';
const PORT = 3000;           // type is literal 3000
```

2.4 Interfaces vs Type Aliases

UNDERSTAND THE CONCEPT

Both **interfaces** and **type aliases** let you give a name to the shape of an object, so you can reuse it:

```
interface User { id: number; name: string; email?: string }  // ? = optional property
type Product = { id: number; title: string; price: number };

const u: User = { id: 1, name: 'Anish' };      // OK — email is optional
```

- `interface` is designed for object shapes. It can be *extended* (`interface Admin extends User`) and *merged* (declaring the same interface twice combines them, which is useful for adding fields to library types).
- `type` can name *anything*: object shapes, unions (`'admin' | 'user'`), tuples, functions and computed types.

Modifiers: `?` makes a property optional. `readonly` prevents it from being changed after creation.

Intersection (`A & B`) means "has everything from A **and** B". Use it to combine types.

Practical rule: use `interface` for objects and public APIs, and `type` for unions and anything more complex. Consistency within a project matters more than the choice itself.

```
interface User {
  readonly id: number;
  name: string;
  email?: string;                  // optional
  [key: string]: unknown;          // index signature
}

interface Admin extends User { permissions: string[] }

// Declaration merging — only interfaces
interface Window { myGlobal: string }

type ID = string | number;         // unions — only type aliases
type Point = { x: number; y: number };
type Point3D = Point & { z: number }; // intersection
type Handler = (e: Event) => void;
```

	`interface`	`type`
Object shapes	Yes	Yes
Unions / primitives / tuples	No	Yes
Declaration merging	Yes	No
`extends`	Yes	Via `&`
Mapped/conditional types	No	Yes

Rule of thumb: `interface` for public object contracts and class shapes; `type` for unions, utility compositions and everything else.

2.5 Functions

UNDERSTAND THE CONCEPT

In TypeScript you annotate a function's **parameters** and, optionally, its **return type**: `function add(a: number, b: number): number`. TypeScript then checks that callers pass the right arguments and that the function really returns what it promises.

- **Optional parameters** (`name?: string`) may be left out. Their value is then `undefined`.
- **Default parameters** (`b = 0`) are automatically optional.
- **Function types** describe the shape of a function so you can type callbacks: `type Callback = (error: Error | null, data?: string) => void`.
- **Overloads** let one function have several call signatures, for example returning a `number` when given a `string` and a `string` when given a `number`.

Annotating return types on exported functions is good practice. If you accidentally return the wrong thing, the error appears inside the function, not somewhere far away where it is used.

```
function add(a: number, b: number = 0): number { return a + b; }
const greet = (name: string, greeting?: string): string => `${greeting ?? 'Hi'} ${name}`;
function sum(...nums: number[]): number { return nums.reduce((a, b) => a + b, 0); }

// Function type
type BinaryOp = (a: number, b: number) => number;

// Overloads
function parse(input: string): number;
function parse(input: number): string;
function parse(input: string | number) {
  return typeof input === 'string' ? Number(input) : String(input);
}

// `this` parameter
function onClick(this: HTMLButtonElement, e: MouseEvent) { this.disabled = true; }
```

2.6 Narrowing and Type Guards

UNDERSTAND THE CONCEPT

When a value can be several types (`string | number`), TypeScript only lets you use what's common to *all* of them. **Narrowing** is how you prove to TypeScript which specific type you have at a given point, using ordinary JavaScript checks:

- `typeof x === 'string'` checks primitives.
- `x instanceof Date` checks class instances.
- `'swim' in animal` checks whether a property exists.
- `x == null` checks for null/undefined.
- **Type guards** are your own functions returning `x is Type`.

Inside the `if` block, TypeScript *knows* the narrower type, so autocomplete and checks become precise.

Discriminated unions are one of TypeScript's most useful patterns. Every member of a union has a common property with a unique literal value, the *discriminant* (often `kind`, `type` or `status`). Switching on it narrows to exactly one member. Combined with a `never` check in the `default` branch, TypeScript will **warn you if you forget to handle a case**. This is perfect for modelling states like `loading | success | error`.

```ts
function format(v: string | number | Date | null) {
  if (v == null) return '';                 // null/undefined narrowing
  if (typeof v === 'string') return v.toUpperCase();
  if (typeof v === 'number') return v.toFixed(2);
  if (v instanceof Date) return v.toISOString();
}

// `in` narrowing
type Fish = { swim: () => void };
type Bird = { fly: () => void };
function move(a: Fish | Bird) { 'swim' in a ? a.swim() : a.fly(); }

// User-defined type guard
function isString(x: unknown): x is string { return typeof x === 'string'; }

// Assertion function
function assertDefined<T>(v: T, msg = 'missing'): asserts v is NonNullable<T> {
  if (v == null) throw new Error(msg);
}
```

Discriminated Unions + Exhaustiveness

```ts
type Shape =
  | { kind: 'circle'; radius: number }
  | { kind: 'rect'; w: number; h: number }
  | { kind: 'triangle'; base: number; height: number };

function area(s: Shape): number {
  switch (s.kind) {
    case 'circle':   return Math.PI * s.radius ** 2;
    case 'rect':     return s.w * s.h;
    case 'triangle': return 0.5 * s.base * s.height;
    default: {
      const _exhaustive: never = s; // compile error if a case is missing
      return _exhaustive;
    }
  }
}

// Typical API state modelling
type AsyncState<T> =
  | { status: 'idle' }
  | { status: 'loading' }
  | { status: 'success'; data: T }
  | { status: 'error'; error: string };
```

2.7 Generics

UNDERSTAND THE CONCEPT

Generics let you write code that works with many types while keeping type safety. They are "type parameters", placeholders for a type that's filled in when the code is used. They are written in angle brackets: `<T>`.

An analogy: a shipping box labelled "Box of ___". Once you fill in "Box of Books", everyone knows what comes out. Without generics you'd either write a separate box for every item (`BookBox`, `ToyBox`) or use `any` and lose all safety.

```
function firstItem<T>(items: T[]): T | undefined {
  return items[0];
}
const n = firstItem([1, 2, 3]);        // T = number → n is number | undefined
const s = firstItem(['a', 'b']);       // T = string → s is string | undefined
```

- **Constraints** (`<T extends { length: number }>`) say "T can be any type, *as long as* it has a `length`".
- **`keyof`** with generics lets you safely access properties: `getProp(obj, key)` only accepts real keys of `obj`.
- Generics appear everywhere: `Array<T>`, `Promise<T>`, `useState<T>()`, `Map<K, V>`, and API response wrappers like `ApiResponse<User>`.

```
function identity<T>(value: T): T { return value; }
identity<string>('a');
identity(42);                      // inferred T = number

// Constraints
function getLength<T extends { length: number }>(x: T) { return x.length; }

// keyof constraint — type-safe property access
function getProp<T, K extends keyof T>(obj: T, key: K): T[K] { return obj[key]; }
getProp({ a: 1, b: 'x' }, 'b');    // string
// getProp({ a: 1 }, 'z');         // Error

// Generic interfaces & defaults
interface ApiResponse<T = unknown> {
  data: T;
  error?: string;
  status: number;
}

// Generic class
class Stack<T> {
  private items: T[] = [];
  push(item: T) { this.items.push(item); }
  pop(): T | undefined { return this.items.pop(); }
  peek(): T | undefined { return this.items.at(-1); }
}

// Generic fetch helper
async function http<T>(url: string, init?: RequestInit): Promise<T> {
  const res = await fetch(url, init);
  if (!res.ok) throw new Error(res.statusText);
  return res.json() as Promise<T>;
}
const users = await http<User[]>('/api/users');
```

2.8 Built-in Utility Types

UNDERSTAND THE CONCEPT

TypeScript ships with ready-made **utility types**. These are generic "type functions" that transform an existing type into a new one, so you don't have to repeat yourself.

Imagine a `User` type with `id`, `name`, `email` and `password`:

- **`Partial<User>`** makes all fields optional, which suits an "update" form where any field may change.
- **`Required<User>`** makes all fields mandatory.
- **`Readonly<User>`** means no field can be changed.
- **`Pick<User, 'id' | 'name'>`** keeps only some fields, for a public profile.
- **`Omit<User, 'password'>`** removes some fields, for an API response without secrets.
- **`Record<'admin' | 'user', number>`** builds an object type with given keys and value type.
- **`ReturnType<typeof fn>` / `Parameters<typeof fn>`** extract a function's return type or parameter types.
- **`Awaited<Promise<X>>`** gives you `X`, the value inside a promise.

They keep related types **in sync**: when `User` changes, every derived type updates automatically.

```ts
interface Todo { id: number; title: string; done: boolean; tags?: string[] }

type A = Partial<Todo>;                  // all optional
type B = Required<Todo>;                 // all required
type C = Readonly<Todo>;                 // all readonly
type D = Pick<Todo, 'id' | 'title'>;     // subset
type E = Omit<Todo, 'id'>;               // remove keys
type F = Record<'admin' | 'user', number>; // { admin: number; user: number }
type G = Exclude<'a' | 'b' | 'c', 'a'>;  // 'b' | 'c'
type H = Extract<'a' | 'b', 'a' | 'z'>;  // 'a'
type I = NonNullable<string | null>;     // string
type J = ReturnType<typeof setTimeout>;
type K = Parameters<(a: number, b: string) => void>; // [number, string]
type L = Awaited<Promise<Promise<number>>>; // number
type M = InstanceType<typeof Date>;
type N = Uppercase<'hi'>;                 // 'HI' (also Lowercase, Capitalize, Uncapitalize)

// Common patterns
type CreateTodoDto = Omit<Todo, 'id'>;
type UpdateTodoDto = Partial<Omit<Todo, 'id'>> & Pick<Todo, 'id'>;
```

2.9 Advanced Types

UNDERSTAND THE CONCEPT

This section is about **type-level programming**: using TypeScript's type system to *compute* new types from existing ones. Most application code doesn't need it, but library code and well-typed APIs do. Understanding it also helps you read complex error messages.

- `typeof value` (in a type position) gets the type of an existing variable. `keyof Type` gets a union of its property names. `Type['prop']` reads the type of one property (an *indexed access*).
- `as const` freezes a value into its most specific literal type (e.g. `'admin'` instead of `string`) and makes it read-only.
- **Mapped types** (`{ [K in keyof T]: ... }`) loop over every key of a type and build a new one. This is how `Partial` and `Readonly` are implemented.
- **Conditional types** (`T extends U ? X : Y`) are an "if/else" for types.
- `infer` captures a part of a type inside a conditional, e.g. "the type that this Promise wraps".
- **Template literal types** build string types from other string types (`` `on${Capitalize<Event>}` `` → `'onClick'`).
- `satisfies` checks that a value matches a type *without* widening it, so you keep the precise inferred type.
- **Branded types** prevent mixing up values that share a basic type, such as a `UserId` string and an `OrderId` string.

`keyof`, `typeof`, indexed access

```ts
const config = { host: 'localhost', port: 5432 } as const;
type Config = typeof config;             // { readonly host: 'localhost'; readonly port: 5432 }
type ConfigKey = keyof Config;           // 'host' | 'port'
type Port = Config['port'];              // 5432

const ROLES = ['admin', 'editor', 'viewer'] as const;
type Role = (typeof ROLES)[number];      // 'admin' | 'editor' | 'viewer'
```

Mapped Types

```ts
type MyReadonly<T> = { readonly [K in keyof T]: T[K] };
type MyPartial<T> = { [K in keyof T]?: T[K] };
type Mutable<T> = { -readonly [K in keyof T]: T[K] };   // remove modifier
type Nullable<T> = { [K in keyof T]: T[K] | null };

// Key remapping with `as`
type Getters<T> = {
  [K in keyof T as `get${Capitalize<string & K>}`]: () => T[K];
};
type UserGetters = Getters<{ name: string; age: number }>;
// { getName: () => string; getAge: () => number }

// Filter keys by value type
type PickByType<T, V> = { [K in keyof T as T[K] extends V ? K : never]: T[K] };
```

Conditional Types and `infer`

```ts
type IsString<T> = T extends string ? true : false;

// Distributive over unions
type ToArray<T> = T extends any ? T[] : never;
type X = ToArray<string | number>;        // string[] | number[]
type NoDistribute<T> = [T] extends [any] ? T[] : never; // (string | number)[]

// infer — extract inner types
type MyReturnType<T> = T extends (...args: any[]) => infer R ? R : never;
```

```ts
type ElementType<T> = T extends (infer U)[] ? U : T;
type UnwrapPromise<T> = T extends Promise<infer U> ? UnwrapPromise<U> : T;

// Deep partial (recursive)
type DeepPartial<T> = {
  [K in keyof T]?: T[K] extends object ? DeepPartial<T[K]> : T[K];
};
```

Template Literal Types

```ts
type HttpMethod = 'GET' | 'POST' | 'PUT' | 'DELETE';
type Route = `/api/${'users' | 'orders'}`;
type Endpoint = `${HttpMethod} ${Route}`;     // 'GET /api/users' | ...

type EventName<T extends string> = `on${Capitalize<T>}`;
type ClickHandler = EventName<'click'>;       // 'onClick'

// Extract route params: '/users/:id/posts/:postId' → 'id' | 'postId'
type Params<S extends string> =
  S extends `${string}:${infer P}/${infer Rest}` ? P | Params<Rest>
  : S extends `${string}:${infer P}` ? P
  : never;
type R = Params<'/users/:id/posts/:postId'>;  // 'id' | 'postId'
```

`satisfies` operator (TS 4.9+)

```ts
type Colors = Record<'primary' | 'secondary', string | [number, number, number]>;

const palette = {
  primary: '#f00',
  secondary: [0, 128, 255],
} satisfies Colors;      // validates shape BUT keeps the narrow inferred type

palette.primary.toUpperCase();   // OK — known to be string
palette.secondary.map(n => n);   // OK — known to be tuple
```

Branded / Nominal Types

```ts
type Brand<T, B extends string> = T & { readonly __brand: B };
type UserId = Brand<string, 'UserId'>;
type OrderId = Brand<string, 'OrderId'>;

const toUserId = (s: string) => s as UserId;
function getUser(id: UserId) { /* ... */ }
getUser(toUserId('u_1'));
// getUser('o_1' as OrderId); // Error — prevents mixing IDs
```

2.10 Classes in TypeScript

UNDERSTAND THE CONCEPT

TypeScript adds safety features on top of JavaScript classes:

- **Access modifiers:** `public` (anyone can use it, the default), `private` (only inside this class), `protected` (this class and subclasses). These are checked only by the compiler. JavaScript's `#field` syntax is private at run time too.
- **`readonly`:** a property that can be set only in the constructor.

- **Parameter properties:** `constructor(private name: string)` declares and assigns a field in one go.
- **`abstract` classes:** cannot be instantiated directly. They define methods that subclasses *must* implement. They work as a template for a family of classes.
- **`implements`:** a class promises to satisfy an interface's shape, and the compiler checks the promise.
- **`override`:** marks a method that replaces a parent's method, so TS errors if the parent method doesn't exist (catching typos and renamed methods).
- **Generic classes:** `class Stack<T>` works for stacks of any type.

```ts
abstract class Repository<T extends { id: number }> {
  protected items = new Map<number, T>();
  constructor(protected readonly tableName: string) {}   // parameter property

  abstract validate(item: T): boolean;

  save(item: T): T {
    if (!this.validate(item)) throw new Error('Invalid');
    this.items.set(item.id, item);
    return item;
  }
  findById(id: number): T | undefined { return this.items.get(id); }
}

interface Product { id: number; name: string; price: number }

class ProductRepo extends Repository<Product> implements Iterable<Product> {
  constructor() { super('products'); }
  validate(p: Product) { return p.price >= 0; }
  override findById(id: number) { return super.findById(id); }
  *[Symbol.iterator]() { yield* this.items.values(); }
}
```

Access modifiers: `public` (default), `protected` (class + subclasses), `private` (TS compile-time only), `#private` (JS runtime-private). `readonly` prevents re-assignment.

2.11 Enums vs Union Literals

UNDERSTAND THE CONCEPT

An **enum** ("enumeration") is a named set of constants, like days of the week or order statuses. It makes code more readable than magic strings or numbers scattered around.

```ts
enum OrderStatus { Pending = 'PENDING', Shipped = 'SHIPPED', Delivered = 'DELIVERED' }
let s: OrderStatus = OrderStatus.Shipped;
```

Unlike most TypeScript features, enums **generate real JavaScript objects** at run time. Many teams prefer the simpler alternative: a **union of string literals** (`type Status = 'PENDING' | 'SHIPPED'`), optionally backed by an `as const` object. It produces no extra code, works naturally with JSON from APIs, and gives the same autocomplete.

Avoid **numeric enums** unless you have a reason. TypeScript lets *any* number be assigned to them, which defeats the purpose.

```ts
enum Status { Active = 'ACTIVE', Inactive = 'INACTIVE' }   // emits runtime object
const enum Dir { Up, Down }                                // inlined, no runtime object

// Often preferred: union + as const object (no TS-only runtime emit)
const STATUS = { Active: 'ACTIVE', Inactive: 'INACTIVE' } as const;
type StatusT = (typeof STATUS)[keyof typeof STATUS];       // 'ACTIVE' | 'INACTIVE'
```

Numeric enums are reverse-mapped and accept any number — a common source of bugs. Prefer string enums or union types.

2.12 Decorators (TS 5 Standard Decorators)

UNDERSTAND THE CONCEPT

A **decorator** is a special function, written with `@name` above a class, method or property, that wraps or modifies the thing it decorates. It is a declarative way to add behaviour like logging, validation, dependency injection or routing without cluttering the main code.

```ts
@Controller('/users')        // NestJS: this class handles /users routes
class UsersController {
  @Get(':id')                // this method handles GET /users/:id
  findOne() { ... }
}
```

Frameworks such as **NestJS**, **Angular** and **TypeORM** are built around decorators. TypeScript 5 supports the official ECMAScript decorator standard. Older frameworks use the earlier "experimental" version, enabled with `experimentalDecorators` in tsconfig.

```ts
function log<T extends (...args: any[]) => any>(target: T, ctx: ClassMethodDecoratorContext) {
  return function (this: unknown, ...args: Parameters<T>): ReturnType<T> {
    console.log(`→ ${String(ctx.name)}`, args);
    return target.apply(this, args);
  } as T;
}

class Calculator {
  @log
  add(a: number, b: number) { return a + b; }
}
```

Frameworks like NestJS, TypeORM and Angular use (legacy) `experimentalDecorators` with `emitDecoratorMetadata`.

2.13 Declaration Files and Module Augmentation

UNDERSTAND THE CONCEPT

Declaration files (`.d.ts`) contain *only types*, with no runnable code. They describe the shape of JavaScript code so TypeScript can type-check how you use it. When you install a plain-JS library, its types often come from a separate `@types/library-name` package (e.g. `npm i -D @types/express`).

You write your own `.d.ts` when:

- A library has no types: `declare module 'old-lib' { ... }`.
- You want to type global things such as `process.env` variables or `window` additions.
- You want to **augment** (extend) an existing type, for example adding a `user` property to Express's `Request` after your auth middleware sets it.

`declare` tells TypeScript "this exists at run time, trust me, here is its type". It produces no JavaScript.

```
// types/env.d.ts
declare namespace NodeJS {
  interface ProcessEnv {
    DATABASE_URL: string;
    NODE_ENV: 'development' | 'production' | 'test';
  }
}

// Untyped JS library
declare module 'legacy-lib' {
  export function doThing(x: string): number;
}

// Augment Express Request
declare global {
  namespace Express {
    interface Request { user?: { id: string; role: string } }
  }
}
export {};
```

2.14 TypeScript with React

UNDERSTAND THE CONCEPT

Using TypeScript with React means typing:

- **Props:** describe what a component accepts with an interface or type. TS then errors if a parent forgets a required prop or passes the wrong type.
- **State:** `useState<User | null>(null)` when the initial value doesn't reveal the full type.
- **Refs:** `useRef<HTMLInputElement>(null)` so `ref.current.focus()` is known to be valid.
- **Events:** `React.ChangeEvent<HTMLInputElement>`, `React.FormEvent<HTMLFormElement>` and so on, so `e.target.value` is typed.
- **Children:** `ReactNode` is anything React can render.
- **Context:** create it with a type and a helper hook that throws if used outside the provider.

Use `ComponentProps<'button'>` to accept every native button attribute, so wrapper components behave like real HTML elements. **Generic components** (`List<T>`) let one component render lists of any item type with full type safety.

```
import { useState, useRef, useReducer, createContext, useContext, type ReactNode, type
ComponentProps } from 'react';
```

```tsx
type ButtonProps = ComponentProps<'button'> & {
  variant?: 'primary' | 'ghost';
  loading?: boolean;
};

export function Button({ variant = 'primary', loading, children, ...rest }: ButtonProps) {
  return (
    <button className={variant} disabled={loading || rest.disabled} {...rest}>
      {loading ? 'Loading…' : children}
    </button>
  );
}

// State & refs
const [user, setUser] = useState<User | null>(null);
const inputRef = useRef<HTMLInputElement>(null);

// Events
const onChange = (e: React.ChangeEvent<HTMLInputElement>) => setValue(e.target.value);
const onSubmit = (e: React.FormEvent<HTMLFormElement>) => e.preventDefault();

// Generic component
type ListProps<T> = { items: T[]; render: (item: T) => ReactNode; keyOf: (item: T) => string };
function List<T>({ items, render, keyOf }: ListProps<T>) {
  return <ul>{items.map(i => <li key={keyOf(i)}>{render(i)}</li>)}</ul>;
}

// Typed reducer
type Action = { type: 'inc' } | { type: 'set'; payload: number };
function reducer(state: number, action: Action): number {
  switch (action.type) {
    case 'inc': return state + 1;
    case 'set': return action.payload;
  }
}

// Typed context with safe hook
type AuthCtx = { user: User | null; login: (u: User) => void };
const AuthContext = createContext<AuthCtx | null>(null);
export function useAuth() {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error('useAuth must be used inside <AuthProvider>');
  return ctx;
}
```

2.15 TypeScript with Node/Express

UNDERSTAND THE CONCEPT

On the backend, TypeScript types the request and response objects of your server framework. The most important lesson is that **TypeScript cannot check data coming from outside your program at run time**: HTTP request bodies, query strings, environment variables, database results, third-party APIs. The types are gone when the code runs, so a client can send anything.

The solution is a **runtime validation library** such as **Zod**. You define a *schema* once. Zod validates real data against it at run time, and you can **derive the TypeScript type from the schema** (`z.infer<typeof Schema>`), so the type and the validation can never disagree.

```ts
import express, { type Request, type Response, type NextFunction } from 'express';
import { z } from 'zod';

const CreateUser = z.object({
  name: z.string().min(2),
  email: z.string().email(),
  age: z.number().int().positive().optional(),
});
type CreateUserDto = z.infer<typeof CreateUser>;   // derive TS type from runtime schema

const app = express();
app.use(express.json());

app.post('/users', (req: Request<{}, {}, CreateUserDto>, res: Response, next: NextFunction) => {
  const parsed = CreateUser.safeParse(req.body);
  if (!parsed.success) return res.status(400).json(parsed.error.flatten());
  res.status(201).json({ id: 1, ...parsed.data });
});
```

Key insight: TS types vanish at runtime, so validate external input (HTTP bodies, env vars, JSON files) with a runtime schema (Zod, Valibot, io-ts) and *infer* the static type from it.

2.16 Type Challenges (Practice)

UNDERSTAND THE CONCEPT

These are **type challenges**: small puzzles where you rebuild built-in utility types or create new ones. They are a good way to practise mapped types, conditional types and `infer`. Try to write each one yourself before reading the solution. Search "type-challenges" on GitHub for hundreds more, graded from easy to extreme.

```ts
// 1. MyPick
type MyPick<T, K extends keyof T> = { [P in K]: T[P] };

// 2. TupleToUnion
type TupleToUnion<T extends readonly unknown[]> = T[number];

// 3. Last element of tuple
type Last<T extends unknown[]> = T extends [...infer _, infer L] ? L : never;

// 4. DeepReadonly
type DeepReadonly<T> = {
  readonly [K in keyof T]: T[K] extends (...a: any[]) => any ? T[K]
    : T[K] extends object ? DeepReadonly<T[K]> : T[K];
};

// 5. Union to intersection
type UnionToIntersection<U> =
  (U extends any ? (x: U) => void : never) extends (x: infer I) => void ? I : never;

// 6. Required keys of T
type RequiredKeys<T> = { [K in keyof T]-?: {} extends Pick<T, K> ? never : K }[keyof T];
```

2.17 Variance, `any` vs `unknown` vs `never` — Summary

UNDERSTAND THE CONCEPT

A summary of the special types, which interviewers often ask about:

- **`any`:** "I give up on types". Anything goes in and anything comes out. It is unsafe.
- **`unknown`:** "I don't know yet". Anything goes in, but you must narrow it before using it. It is safe.
- **`never`:** "this cannot happen". Nothing goes in. It is used for exhaustiveness checks and functions that never return.
- **`void`:** "returns nothing meaningful".

Variance describes when one type can be used where another is expected. For example, a function that accepts any `Animal` can safely be used where a function accepting a `Dog` is required, but not the other way round. You rarely need the theory, but it explains some confusing callback errors.

- `any`: disables checking in both directions (assignable to/from everything).
- `unknown`: top type — anything assignable *to* it; must narrow before use.
- `never`: bottom type — assignable to everything, nothing assignable to it (except `never`). Represents impossible states.
- Function parameters are checked **contravariantly** under `strictFunctionTypes` (for function-type properties, not methods).
- `object` = any non-primitive; `{}` = any non-nullish value; `Object` — avoid.

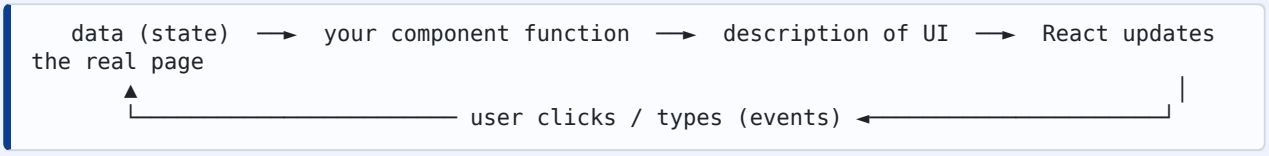
Part 3 — React: Basics to Advanced

UNDERSTAND THE CONCEPT

What is React? React is a JavaScript **library for building user interfaces**, created by Facebook (Meta) in 2013. A user interface (UI) is everything the user sees and interacts with: buttons, forms, lists, menus.

The problem React solves. With plain JavaScript you change the page by hand: find this element, change its text, add that class, remove that row. In a large app with lots of data changing, it becomes very hard to keep the page in sync with the data, and bugs creep in.

React's idea: you describe **what the UI should look like for the current data**, and React works out how to update the page efficiently whenever the data changes. You never touch the DOM directly.



Components are the building blocks: small, reusable, independent pieces of UI (`<Navbar />` , `<ProductCard />` , `<Button />`). You compose them like LEGO bricks to build whole pages.

A first component:

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);         // a piece of data that can change
  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>Click me</button>
    </div>
  );
}
```

Clicking the button calls `setCount` . React then runs `Counter` again with the new `count` , compares the result with the old one, and updates only the `<p>` text on screen.

3.1 Core Ideas

UNDERSTAND THE CONCEPT

Declarative vs imperative. *Imperative* code gives step-by-step instructions ("find the list, create an li, set its text, append it"). *Declarative* code describes the end result ("the list shows these items"). React is declarative, which makes UIs easier to predict and debug.

JSX is the HTML-like syntax used inside React components. It is *not* HTML and not a string. A build tool converts it into plain JavaScript function calls that create lightweight objects describing the UI (called **React elements**). Inside JSX, curly braces `{ }` let you insert any JavaScript expression: variables, calculations, function calls. A few differences from HTML: `className` instead of `class`, `htmlFor` instead of `for`, event names in camelCase (`onClick`), and every tag must be closed.

Virtual DOM. Updating the real browser DOM is relatively slow. React keeps a lightweight copy of the UI as JavaScript objects (the "virtual DOM"). When data changes:

1. React calls your components again to produce a **new** virtual tree (this is called **rendering**).
2. It **compares** (diffs) the new tree with the previous one. This is called **reconciliation**.
3. It applies only the **differences** to the real DOM (the **commit**).

Fiber is the name of React's internal engine (since React 16). It splits rendering work into small units, so React can pause, prioritise urgent updates like typing over less urgent ones like a big list, and resume later. This is called **concurrent rendering**.

Keys. When rendering a list, give each item a stable, unique `key` (usually its database id). Keys let React tell which item is which between renders. Without good keys, React may reuse the wrong component for the wrong item, mixing up state such as checkbox values or input text.

- **Declarative UI:** describe *what* the UI should look like for a given state; React figures out *how* to update the DOM.
- **Components:** functions that take props and return JSX.
- **Unidirectional data flow:** data goes down via props; events go up via callbacks.
- **UI = f(state):** re-rendering is just calling the component function again.

JSX

JSX compiles to `jsx()` calls (React 17+ automatic runtime), producing plain objects (React elements).

```
const el = <h1 className="title">Hello {name}</h1>;
// compiles to → jsx('h1', { className: 'title', children: ['Hello ', name] })

// Rules: one root (or <>…</> Fragment), className/htmlFor, camelCase events,
// {} for expressions, self-close empty tags, style={{ color: 'red' }}
```

Virtual DOM, Reconciliation and Fiber

1. State changes → React re-renders the component → new element tree.
2. **Reconciliation / diffing:** compares new tree vs previous. Heuristics: different element type ⇒ destroy and rebuild subtree; same type ⇒ update props; lists are matched by `key`.
3. **Commit phase:** applies the minimal DOM mutations, runs layout effects, then passive effects.

Fiber (React 16+) is the reconciliation engine: work is split into units so rendering can be paused, prioritized and resumed — enabling **concurrent rendering** (React 18+).

Render phase (pure, may run multiple times, interruptible) vs **commit phase** (side effects, synchronous).

Keys

```
// Bad — index keys break state when items are reordered/inserted
{todos.map((t, i) => <TodoItem key={i} todo={t} />)}
// Good — stable unique id
{todos.map(t => <TodoItem key={t.id} todo={t} />)}

// Trick: changing `key` resets a component's state entirely
<ProfileForm key={userId} userId={userId} />
```

3.2 Components and Props

UNDERSTAND THE CONCEPT

A **component** is a JavaScript function that returns JSX. Its name must start with a capital letter (`UserCard` , not `userCard`) so React can tell it apart from HTML tags.

Props ("properties") are the inputs to a component, passed like HTML attributes: `<Avatar src="/me.png" size={40} />` . Inside the component they arrive as one object. Props are **read-only**: a component must never change its own props. Data flows **one way**, from parent to child. If a child needs to tell its parent something, the parent passes down a **callback function** as a prop, and the child calls it (e.g. `onSelect(id)`).

`children` is a special prop containing whatever you put *between* a component's opening and closing tags. It lets you build wrapper components like `<Card>...anything...</Card>` or `<Modal>...</Modal>` . This is called **composition**, and it is React's main way of reusing UI (instead of inheritance).

Conditional rendering means showing different UI based on data, using plain JavaScript: the ternary `cond ? <A/> : <B/>` , `cond && <A/>` , or an early `return` .

Forms: controlled vs uncontrolled. In a **controlled** input, React state holds the value. Each keystroke updates state, and state feeds the input's `value` . This gives full control for validation and formatting. In an **uncontrolled** input, the browser keeps the value and you read it when needed (via a ref or on submit). That is simpler, with less re-rendering.

```
function Avatar({ src, size = 40, alt = '' }) {
  return <img src={src} width={size} height={size} alt={alt} />;
}

// children & composition
function Card({ title, children, footer }) {
  return (
    <section className="card">
      <h2>{title}</h2>
      <div>{children}</div>
      {footer && <footer>{footer}</footer>}
```

```
    </section>
  );
}

<Card title="Profile" footer={<button>Save</button>}>
  <Avatar src="/me.png" />
</Card>
```

Conditional rendering:

```
{isLoggedIn ? <Dashboard /> : <Login />}
{items.length > 0 && <List items={items} />}   // careful: {0 && <X/>} renders "0"
{items.length ? <List /> : null}
```

Controlled vs uncontrolled inputs:

```
// Controlled — React state is the source of truth
const [email, setEmail] = useState('');
<input value={email} onChange={e => setEmail(e.target.value)} />

// Uncontrolled — DOM holds the value; read via ref / FormData
const ref = useRef(null);
<input defaultValue="x" ref={ref} />
```

3.3 Hooks — Complete Reference

UNDERSTAND THE CONCEPT

Hooks are special functions, whose names start with `use`, that let function components "hook into" React features: remembering data (state), running side effects, accessing context and so on. Before hooks (2019), these features needed class components.

Two rules you must follow (React relies on the *order* hooks are called in to know which state belongs to which hook):

1. Call hooks only at the **top level** of a component, never inside `if` statements, loops or nested functions.
2. Call hooks only from **React components or custom hooks**, not from regular functions.

State (`useState`) is data that belongs to a component and can change over time: the text in an input, whether a menu is open, a list of todos. When you call the **setter** (`setCount`), React **re-renders** the component with the new value. Regular variables are reset on every render and changing them does nothing visible. State is *remembered* between renders.

- State updates are **asynchronous and batched**. Calling `setCount` does not change `count` immediately on the next line. The new value appears on the next render.
- When the new state depends on the old one, use the **function form** `setCount(c => c + 1)`, so it always uses the latest value.
- **Never mutate state** (`user.name = 'x'` or `todos.push(...)`). Always create a new object or array. React compares by reference, so a mutated object looks "unchanged" and the screen won't update.

Side effects (`useEffect`). Rendering should be *pure*: it only calculates UI. Anything that reaches outside the component is a **side effect**: fetching data, timers, subscriptions, manually touching the DOM, logging. `useEffect(fn, deps)` runs `fn` **after** React has updated the screen.

- The **dependency array** controls *when* it re-runs. No array: after every render. `[]` : only once after the first render ("mount"). `[userId]` : whenever `userId` changes.
- The function can return a **cleanup function**. React runs it before the effect runs again and when the component is removed ("unmounts"). Use it to cancel timers, unsubscribe and abort requests.

Refs (`useRef`) give you a box (`ref.current`) that survives re-renders, and changing it does **not** trigger a re-render. Two uses: (1) holding a reference to a DOM element (to focus an input, measure size), and (2) storing mutable values such as timer IDs.

Memoisation (`useMemo` , `useCallback`). Every render re-runs all the code in your component. `useMemo` caches the *result of a calculation* until its dependencies change. `useCallback` caches a *function* so it keeps the same identity between renders. They are performance tools. Use them when a calculation is expensive or when a stable reference matters, not everywhere.

`useReducer` is an alternative to `useState` for complex state. All the update logic lives in one **reducer** function, `(currentState, action) => newState` , and components **dispatch** actions such as `{ type: 'added', item }` . It is the same pattern Redux uses.

`useContext` reads a value from **Context**, React's way to share data (current user, theme, language) with many components deep in the tree without passing props through every level. Passing props through many levels is called **prop drilling**.

React 19 adds `use()` (read a promise or context), **Actions** (async functions passed to forms), `useActionState` , `useFormStatus` and `useOptimistic` (show an expected result immediately while the server request is still running).

Rules of Hooks: only call at the top level (not in loops/conditions/nested functions) and only from React functions or custom hooks. React relies on call order to associate state with hooks.

useState

```
const [count, setCount] = useState(0);
const [data, setData] = useState(() => expensiveInit()); // lazy initializer — runs once

setCount(count + 1);             // uses the value from this render
setCount(c => c + 1);            // functional update — use when new state depends on old

// State updates are batched (automatic batching in React 18 for all updates)
function handle() {
  setCount(c => c + 1);
  setCount(c => c + 1);          // final: +2 ; with setCount(count + 1) twice → +1
}

// Objects/arrays: always create new references (immutability)
setUser(u => ({ ...u, name: 'New' }));
setTodos(ts => ts.map(t => (t.id === id ? { ...t, done: !t.done } : t)));
setTodos(ts => ts.filter(t => t.id !== id));
```

useEffect

Synchronizes a component with an **external system** (network, subscriptions, timers, DOM APIs). Runs *after* paint.

```
useEffect(() => {
  // runs after every render if no deps array
});

useEffect(() => {
  // runs once after mount
}, []);

useEffect(() => {
  const controller = new AbortController();
  let ignore = false;

  async function load() {
    try {
      const res = await fetch(`/api/users/${userId}`, { signal: controller.signal });
      const json = await res.json();
      if (!ignore) setUser(json);
    } catch (e) {
      if (e.name !== 'AbortError') setError(e);
    }
  }
  load();

  return () => {           // cleanup: runs before next effect & on unmount
    ignore = true;
    controller.abort();
  };
}, [userId]);              // re-run when userId changes (avoids race conditions)
```

In development with `<StrictMode>`, effects run **mount → unmount → mount** to surface missing cleanups.

You might not need an effect:

```
// Bad — derived state via effect causes an extra render
const [fullName, setFullName] = useState('');
useEffect(() => setFullName(`${first} ${last}`), [first, last]);

// Good — compute during render
const fullName = `${first} ${last}`;

// Bad — resetting state on prop change via effect. Good — use `key`.
// Bad — handling user events in effects. Good — do it in the event handler.
```

useLayoutEffect

Same signature as `useEffect`, but fires synchronously **after DOM mutations, before paint**. Use for measuring layout (tooltip positioning) to avoid flicker. Blocks painting — use sparingly.

useRef

```
const inputRef = useRef(null);
useEffect(() => inputRef.current.focus(), []);
<input ref={inputRef} />

// Mutable value that persists across renders WITHOUT causing re-render
```

```
const renderCount = useRef(0);
renderCount.current++;

// Keep the latest value for use in callbacks / intervals
const savedCallback = useRef(callback);
useEffect(() => { savedCallback.current = callback; }, [callback]);
```

useMemo and useCallback

```
// Memoize an expensive computation
const visibleTodos = useMemo(() => filterTodos(todos, tab), [todos, tab]);

// Memoize a function reference (useful when passed to memoized children / effect deps)
const handleSelect = useCallback((id) => setSelected(id), []);

// useCallback(fn, deps) === useMemo(() => fn, deps)
```

Do not memoize everything — it has a cost. Use it when (1) the computation is expensive, (2) referential stability matters for `React.memo` children or effect dependencies. The **React Compiler** (React 19 ecosystem) can auto-memoize.

useReducer

```
const initialState = { items: [], loading: false, error: null };

function reducer(state, action) {
  switch (action.type) {
    case 'FETCH_START':   return { ...state, loading: true, error: null };
    case 'FETCH_SUCCESS': return { ...state, loading: false, items: action.payload };
    case 'FETCH_ERROR':   return { ...state, loading: false, error: action.error };
    case 'REMOVE':        return { ...state, items: state.items.filter(i => i.id !== action.id)
};
    default: throw new Error(`Unknown action ${action.type}`);
  }
}

const [state, dispatch] = useReducer(reducer, initialState);
dispatch({ type: 'FETCH_START' });
```

Prefer `useReducer` when state has multiple sub-values that change together or next state depends on complex logic.

useContext

```
const ThemeContext = createContext('light');

function App() {
  const [theme, setTheme] = useState('dark');
  const value = useMemo(() => ({ theme, setTheme }), [theme]); // avoid re-render storms
  return (
    <ThemeContext.Provider value={value}>
      <Toolbar />
    </ThemeContext.Provider>
  );
}

function Toolbar() {
  const { theme, setTheme } = useContext(ThemeContext);
  return <button onClick={() => setTheme(t => t === 'dark' ? 'light' : 'dark')}>{theme}
</button>;
}
```

All consumers re-render when the context value changes. Split contexts (state vs dispatch) or use a store library for frequently changing global state.

Other Hooks

```
// useId — stable unique ids (SSR-safe) for accessibility
const id = useId();
<label htmlFor={id}>Email</label><input id={id} />

// useTransition — mark updates as non-urgent (keeps input responsive)
const [isPending, startTransition] = useTransition();
const onChange = (e) => {
  setQuery(e.target.value);                          // urgent
  startTransition(() => setResults(search(e.target.value))); // can be interrupted
};

// useDeferredValue — defer re-rendering of an expensive part
const deferredQuery = useDeferredValue(query);
<SlowList query={deferredQuery} />

// useImperativeHandle — expose a custom API to a parent ref
function FancyInput({ ref }) {       // React 19: ref is a normal prop
  const inner = useRef();
  useImperativeHandle(ref, () => ({ focus: () => inner.current.focus() }));
  return <input ref={inner} />;
}

// useSyncExternalStore — subscribe to external stores safely in concurrent mode
const isOnline = useSyncExternalStore(
  (cb) => { window.addEventListener('online', cb); window.addEventListener('offline', cb);
            return () => { window.removeEventListener('online', cb);
window.removeEventListener('offline', cb); }; },
  () => navigator.onLine,
);
```

React 19 Hooks and APIs

```
// use() — read a promise or context (can be called conditionally)
function Comments({ commentsPromise }) {
  const comments = use(commentsPromise);   // suspends until resolved
  return comments.map(c => <p key={c.id}>{c.text}</p>);
}

// Actions + useActionState — form handling with pending & errors
async function updateName(prevState, formData) {
  const error = await saveName(formData.get('name'));
  return error ? { error } : { error: null };
}
function NameForm() {
  const [state, formAction, isPending] = useActionState(updateName, { error: null });
  return (
    <form action={formAction}>
      <input name="name" />
      <SubmitButton />
      {state.error && <p>{state.error}</p>}
    </form>
  );
}

// useFormStatus — status of the parent <form>
function SubmitButton() {
  const { pending } = useFormStatus();
  return <button disabled={pending}>{pending ? 'Saving…' : 'Save'}</button>;
}

// useOptimistic — show the result instantly while the request is in flight
```

```
const [optimisticTodos, addOptimistic] = useOptimistic(todos, (state, newTodo) => [...state,
newTodo]);
```

Other React 19 changes: `ref` as a prop (no more `forwardRef` for new code), `<Context>` usable directly as provider, document metadata (`<title>`, `<meta>`) hoisting, ref cleanup functions.

3.4 Custom Hooks

UNDERSTAND THE CONCEPT

A **custom hook** is your own function, whose name starts with `use`, that calls other hooks. It lets you **extract and reuse stateful logic** between components. For example, "fetch data with loading and error states" or "sync a value with localStorage" can be written once and used anywhere.

Important: custom hooks share **logic**, not **state**. Each component that calls `useFetch()` gets its own independent copy of the state inside it.

Think of custom hooks as the React version of utility functions. They keep components short and focused on what to display.

Custom hooks extract reusable **stateful logic** (not state itself — every call gets its own state).

```
function useFetch(url) {
  const [state, setState] = useState({ data: null, loading: true, error: null });

  useEffect(() => {
    const controller = new AbortController();
    setState(s => ({ ...s, loading: true }));
    fetch(url, { signal: controller.signal })
      .then(r => { if (!r.ok) throw new Error(r.statusText); return r.json(); })
      .then(data => setState({ data, loading: false, error: null }))
      .catch(error => error.name !== 'AbortError' && setState({ data: null, loading: false,
error }));
    return () => controller.abort();
  }, [url]);

  return state;
}

function useDebounce(value, delay = 300) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const t = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(t);
  }, [value, delay]);
  return debounced;
}

function useLocalStorage(key, initial) {
  const [value, setValue] = useState(() => {
    try { const v = localStorage.getItem(key); return v ? JSON.parse(v) : initial; }
    catch { return initial; }
  });
  useEffect(() => { localStorage.setItem(key, JSON.stringify(value)); }, [key, value]);
  return [value, setValue];
}

function usePrevious(value) {
```

```
  const ref = useRef();
  useEffect(() => { ref.current = value; }, [value]);
  return ref.current;
}

function useInterval(callback, delay) {
  const saved = useRef(callback);
  useEffect(() => { saved.current = callback; }, [callback]);
  useEffect(() => {
    if (delay == null) return;
    const id = setInterval(() => saved.current(), delay);
    return () => clearInterval(id);
  }, [delay]);
}

function useOnClickOutside(ref, handler) {
  useEffect(() => {
    const listener = (e) => { if (!ref.current || ref.current.contains(e.target)) return;
handler(e); };
    document.addEventListener('mousedown', listener);
    return () => document.removeEventListener('mousedown', listener);
  }, [ref, handler]);
}
```

3.5 Stale Closures — The Most Common Hook Bug

UNDERSTAND THE CONCEPT

This is the bug that confuses the most React developers, so it gets its own section.

Every render of a component is like a **snapshot**. The functions created during that render, such as event handlers and effect callbacks, **close over** (remember) the props and state **of that particular render** (see Closures, 1.6). If such a function lives on, like a `setInterval` created once in an effect with `[]` deps, it keeps seeing the *old* values forever. That is a **stale closure**.

Fixes: use the functional state update (`setCount(c => c + 1)`), list every value you use in the dependency array (the ESLint rule `react-hooks/exhaustive-deps` reminds you), or keep the latest value in a `ref` .

```
function Timer() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    const id = setInterval(() => {
      setCount(count + 1);   // BUG: `count` is always 0 (captured from first render)
    }, 1000);
    return () => clearInterval(id);
  }, []);                    // missing dependency

  // FIX 1: functional update → setCount(c => c + 1)
  // FIX 2: add count to deps (interval re-created every tick)
  // FIX 3: keep latest value in a ref
}
```

Always enable `eslint-plugin-react-hooks` (`exhaustive-deps`).

3.6 Performance Optimization

UNDERSTAND THE CONCEPT

When does a component re-render? When (1) its own state changes, (2) its **parent** re-renders, which by default makes all children re-render too, or (3) a context value it reads changes.

Re-rendering is normally cheap and fine. It becomes a problem only with very large trees, expensive calculations or long lists. **Measure first** with React DevTools' *Profiler* before optimising.

Tools:

- **`React.memo(Component)`:** skip re-rendering a child when its props are the same as last time (compared shallowly). It only works if the props you pass are stable, which is why it pairs with `useCallback` / `useMemo`.
- **Keep state close to where it's used:** state at the top of the app makes the whole app re-render on every change.
- **Code splitting (`React.lazy` + `Suspense`):** don't download code for pages the user hasn't opened.
- **Virtualisation:** for lists of thousands of rows, render only the rows visible on screen.

```
// React.memo — skip re-render if props are shallow-equal
const Row = React.memo(function Row({ item, onSelect }) {
  return <li onClick={() => onSelect(item.id)}>{item.name}</li>;
});

function List({ items }) {
  const [selected, setSelected] = useState(null);
  const onSelect = useCallback((id) => setSelected(id), []); // stable ref → memo works
  return <ul>{items.map(i => <Row key={i.id} item={i} onSelect={onSelect} />)}</ul>;
}
```

Why does a component re-render? (1) its state changed, (2) its parent re-rendered, (3) a context it uses changed. Props changing is *not* itself a trigger — parent re-render is.

Techniques:

- Move state down (colocate) so fewer components re-render.
- Lift content up — pass expensive subtrees as `children` (they don't re-render when the wrapper's state changes).
- Code splitting with `React.lazy` + `Suspense`.
- Virtualize long lists (`react-window`, `@tanstack/react-virtual`).
- Debounce input; use `useTransition` / `useDeferredValue`.
- Avoid creating new objects/arrays in props unnecessarily.
- Profile with React DevTools Profiler; check "why did this render".

```
const Settings = lazy(() => import('./Settings'));

<Suspense fallback={<Spinner />}>
```

```
  <Settings />
</Suspense>
```

3.7 Advanced Component Patterns

UNDERSTAND THE CONCEPT

As apps grow, certain **component patterns** help structure reusable UI:

- **Compound components:** a group of components that work together and share hidden state through context, like HTML's `<select>` and `<option>`. For example `<Tabs>`, `<Tabs.Tab>`, `<Tabs.Panel>`. The user of the component gets a flexible, readable API.
- **Higher-Order Component (HOC):** a function that takes a component and returns a new, enhanced one (`withAuth(Dashboard)`). It was common before hooks.
- **Render props:** a component receives a *function* as a prop and calls it to decide what to render, sharing its internal data with the caller.
- **Portals:** render a component into a different place in the DOM (such as directly under `<body>`) while keeping it in the same React tree. They are used for modals, tooltips and toasts, so they are not clipped by parent CSS.

Today, **custom hooks** are the preferred way to share logic. HOCs and render props are still found in older code and some libraries.

Compound Components

```
const TabsContext = createContext(null);

function Tabs({ defaultIndex = 0, children }) {
  const [active, setActive] = useState(defaultIndex);
  return <TabsContext.Provider value={{ active, setActive }}>{children}</TabsContext.Provider>;
}
Tabs.List = function TabList({ children }) {
  return <div role="tablist">{children}</div>;
};
Tabs.Tab = function Tab({ index, children }) {
  const { active, setActive } = useContext(TabsContext);
  return <button role="tab" aria-selected={active === index} onClick={() => setActive(index)}>
{children}</button>;
};
Tabs.Panel = function Panel({ index, children }) {
  const { active } = useContext(TabsContext);
  return active === index ? <div role="tabpanel">{children}</div> : null;
};

<Tabs>
  <Tabs.List><Tabs.Tab index={0}>A</Tabs.Tab><Tabs.Tab index={1}>B</Tabs.Tab></Tabs.List>
  <Tabs.Panel index={0}>Panel A</Tabs.Panel>
  <Tabs.Panel index={1}>Panel B</Tabs.Panel>
</Tabs>
```

Higher-Order Components (HOC)

```
function withAuth(Component) {
  return function Wrapped(props) {
```

```
    const { user } = useAuth();
    if (!user) return <Navigate to="/login" />;
    return <Component {...props} user={user} />;
  };
}
export default withAuth(Dashboard);
```

Render Props

```
function MouseTracker({ render }) {
  const [pos, setPos] = useState({ x: 0, y: 0 });
  return <div onMouseMove={e => setPos({ x: e.clientX, y: e.clientY })}>{render(pos)}</div>;
}
<MouseTracker render={({ x, y }) => <p>{x}, {y}</p>} />
```

Custom hooks have largely replaced HOCs and render props for logic reuse.

Controlled/Uncontrolled hybrid & Portals

```
// Portal — render outside parent DOM (modals, tooltips); events still bubble through React tree
function Modal({ open, onClose, children }) {
  if (!open) return null;
  return createPortal(
    <div className="backdrop" onClick={onClose}>
      <div className="modal" onClick={e => e.stopPropagation()}>{children}</div>
    </div>,
    document.body,
  );
}
```

3.8 Error Boundaries

UNDERSTAND THE CONCEPT

Normally, if any component throws an error while rendering, **the entire React app unmounts**, and the user sees a blank page. An **error boundary** is a component that catches errors from the components *inside* it and shows a fallback UI ("Something went wrong") instead, so the rest of the app keeps working. It is like a circuit breaker in a house: one faulty appliance doesn't black out the whole building.

Error boundaries catch errors during **rendering** and lifecycle methods. They do **not** catch errors in event handlers or async code (use `try/catch` there). They must be written as class components, or you can use the popular `react-error-boundary` package.

Catch rendering errors in the subtree (not in event handlers, async code, or SSR). Must be class components (or use `react-error-boundary`).

```
class ErrorBoundary extends React.Component {
  state = { hasError: false, error: null };
  static getDerivedStateFromError(error) { return { hasError: true, error }; }
  componentDidCatch(error, info) { logToService(error, info.componentStack); }
  render() {
    if (this.state.hasError) return this.props.fallback ?? <h2>Something went wrong.</h2>;
    return this.props.children;
```

```
  }
}
```

3.9 Class Component Lifecycle (for legacy code)

UNDERSTAND THE CONCEPT

Before hooks, components with state had to be **classes**. You will still see them in older codebases.

Every component goes through a **lifecycle**:

1. **Mounting:** it is created and inserted into the page for the first time.
2. **Updating:** it re-renders because props or state changed.
3. **Unmounting:** it is removed from the page.

Class components have special methods that React calls at each stage (`componentDidMount` , `componentDidUpdate` , `componentWillUnmount`). With hooks, `useEffect` covers all three: the effect body ≈ mount/update, and the cleanup ≈ unmount.

Phase	Methods
Mount	`constructor` → `getDerivedStateFromProps` → `render` → `componentDidMount`
Update	`getDerivedStateFromProps` → `shouldComponentUpdate` → `render` → `getSnapshotBeforeUpdate` → `componentDidUpdate`
Unmount	`componentWillUnmount`
Error	`getDerivedStateFromError` , `componentDidCatch`

Hook equivalents: `componentDidMount` ≈ `useEffect(fn, [])` ; `componentDidUpdate` ≈ `useEffect(fn, [deps])` ; `componentWillUnmount` ≈ cleanup function; `shouldComponentUpdate` ≈ `React.memo` .

3.10 State Management

UNDERSTAND THE CONCEPT

State management means deciding *where* each piece of data lives and *how* components read and update it. A useful way to think about it is to sort state into categories:

- **Local UI state:** belongs to one component (is this dropdown open?). Use `useState` .
- **Shared state:** needed by a few nearby components. **Lift it up** to their closest common parent and pass it down as props.
- **Global client state:** needed all over the app (logged-in user, cart, theme). Use **Context** for rarely-changing values, or a store library: **Redux Toolkit** (structured, predictable, great dev tools), **Zustand** (minimal and simple).
- **Server state:** data that really lives on the backend (products, orders). It needs caching, background refreshing and loading/error handling. Use **TanStack Query** (React Query) or RTK Query instead of hand-written `useEffect` fetching.

Redux core ideas: the whole app's global state lives in one **store**. You never change it directly. You **dispatch** an **action** (a plain object describing *what happened*, e.g. `{ type: 'cart/itemAdded', payload: item }`). A **reducer** (a pure function) calculates the new state from the old state plus the action. Components **select** the slices of state they need. This one-way flow makes changes predictable and easy to debug.

Kind of state	Tool
Local UI state	`useState` / `useReducer`
Shared across a subtree	Lift state up / Context
Global client state	Redux Toolkit, Zustand, Jotai
Server state (cache, refetch)	TanStack Query, RTK Query, SWR
URL state	Router search params
Form state	React Hook Form + Zod

Redux Toolkit

```
import { configureStore, createSlice, createAsyncThunk } from '@reduxjs/toolkit';

export const fetchTodos = createAsyncThunk('todos/fetch', async () => {
  const res = await fetch('/api/todos');
  return res.json();
});

const todosSlice = createSlice({
  name: 'todos',
  initialState: { items: [], status: 'idle' },
  reducers: {
    added(state, action) { state.items.push(action.payload); }, // Immer makes "mutation" safe
    toggled(state, action) {
      const t = state.items.find(t => t.id === action.payload);
      if (t) t.done = !t.done;
    },
  },
  extraReducers: (builder) => {
    builder
      .addCase(fetchTodos.pending, (s) => { s.status = 'loading'; })
      .addCase(fetchTodos.fulfilled, (s, a) => { s.status = 'idle'; s.items = a.payload; })
      .addCase(fetchTodos.rejected, (s) => { s.status = 'failed'; });
  },
});

export const { added, toggled } = todosSlice.actions;
export const store = configureStore({ reducer: { todos: todosSlice.reducer } });

// In components
const todos = useSelector((s) => s.todos.items);
const dispatch = useDispatch();
dispatch(added({ id: 1, title: 'x', done: false }));
```

Redux principles: single source of truth, state is read-only (change via actions), changes via pure reducers. Middleware (thunk, saga, listener) handle side effects.

Zustand

```
import { create } from 'zustand';
const useCart = create((set, get) => ({
  items: [],
  add: (item) => set((s) => ({ items: [...s.items, item] })),
  total: () => get().items.reduce((sum, i) => sum + i.price, 0),
}));
const items = useCart((s) => s.items);  // selector → re-render only when items change
```

TanStack Query (server state)

```
const { data, isLoading, error } = useQuery({
  queryKey: ['users', page],
  queryFn: () => fetch(`/api/users?page=${page}`).then(r => r.json()),
  staleTime: 60_000,
});

const queryClient = useQueryClient();
const mutation = useMutation({
  mutationFn: (user) => fetch('/api/users', { method: 'POST', body: JSON.stringify(user) }),
  onSuccess: () => queryClient.invalidateQueries({ queryKey: ['users'] }),
});
```

3.11 Routing (React Router v6/v7)

UNDERSTAND THE CONCEPT

A **single-page application (SPA)** loads one HTML page. When the user navigates, JavaScript swaps the content instead of asking the server for a whole new page, which is fast and smooth. **Routing** is mapping URLs (`/products/42`) to components, so the address bar, back button and bookmarks still work.

React Router concepts:

- **Routes:** the URL pattern → component mapping. **Dynamic segments** (`:id`) capture parts of the URL, which you read with `useParams()` .
- **Nested routes + `<Outlet />` :** shared layouts (navbar, sidebar) with changing inner content.
- **`<Link>` / `<NavLink>` :** navigate without a full page reload. Never use a plain `<a href>` for internal links.
- **`useNavigate()` :** navigate from code, e.g. after a form submit.
- **Search params (`?page=2`):** store filter and page state in the URL so it can be shared.
- **Protected routes:** redirect to `/login` if the user isn't authenticated.

```
import { createBrowserRouter, RouterProvider, Outlet, Link, NavLink, useParams, useNavigate,
useSearchParams, Navigate } from 'react-router-dom';

const router = createBrowserRouter([
  {
    path: '/',
    element: <Layout />,
    errorElement: <ErrorPage />,
    children: [
```

```
      { index: true, element: <Home /> },
      { path: 'users/:id', element: <UserPage />, loader: ({ params }) => fetchUser(params.id)
},
      { path: 'admin', element: <RequireAuth><Admin /></RequireAuth> },
      { path: '*', element: <NotFound /> },
    ],
  },
]);

function Layout() { return <><nav><NavLink to="/">Home</NavLink></nav><Outlet /></>; }

function UserPage() {
  const { id } = useParams();
  const [params, setParams] = useSearchParams();
  const navigate = useNavigate();
  return <button onClick={() => navigate('/')}>Back</button>;
}

function RequireAuth({ children }) {
  const { user } = useAuth();
  return user ? children : <Navigate to="/login" replace />;
}
```

3.12 Forms — React Hook Form + Zod

UNDERSTAND THE CONCEPT

Forms are one of the most common and fiddly parts of front-end work: tracking every field's value, validating input, showing error messages, handling submission and disabling the button while saving.

React Hook Form handles field state efficiently: it registers inputs, avoids re-rendering on every keystroke, and tracks errors and submission status. **Zod** defines validation rules as a schema (e.g. "email must be a valid email, password at least 8 characters"). Connected with a *resolver*, the schema validates the form, and the same schema gives you the TypeScript type of the form data.

Always validate on the **server** as well. Client-side validation is only for a better user experience, because users can bypass it.

```
import { useForm } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { z } from 'zod';

const schema = z.object({
  email: z.string().email(),
  password: z.string().min(8),
});
type FormValues = z.infer<typeof schema>;

export function LoginForm() {
  const { register, handleSubmit, formState: { errors, isSubmitting } } =
    useForm<FormValues>({ resolver: zodResolver(schema) });

  const onSubmit = async (values: FormValues) => { await login(values); };

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      <input {...register('email')} />
      {errors.email && <span>{errors.email.message}</span>}
```

```
      <input type="password" {...register('password')} />
      <button disabled={isSubmitting}>Login</button>
    </form>
  );
}
```

3.13 Rendering Strategies and Server Components

UNDERSTAND THE CONCEPT

Where is the HTML produced? This choice affects speed, SEO (search engines) and server cost.

- **CSR (Client-Side Rendering):** the server sends an almost empty HTML file plus JavaScript. The browser runs the JS to build the page. It is simple, but the first load is slower and less SEO-friendly.
- **SSR (Server-Side Rendering):** the server runs React on each request and sends ready-made HTML, so the user sees content immediately. Then JavaScript loads and **hydrates** the page, attaching event handlers so it becomes interactive.
- **SSG (Static Site Generation):** HTML is generated once at build time and served from a CDN. It is very fast, and good for content that rarely changes.
- **ISR:** static pages that are regenerated in the background every so often.

Frameworks like **Next.js** provide all of these plus routing and data fetching on top of React.

React Server Components (RSC) are components that run **only on the server**. They can read the database directly, and their code is never sent to the browser, so less JavaScript is downloaded. **Client Components** (marked `'use client'`) run in the browser and are needed for interactivity (state, effects, event handlers). You mix both: server components for data and layout, and client components for the interactive parts. **Server Actions** (`'use server'`) are functions that run on the server and that you can call directly from forms.

Strategy	When HTML is generated	Use case
CSR	In browser	Dashboards behind auth
SSR	Per request on server	Personalized, SEO pages
SSG	At build time	Docs, blogs, marketing
ISR	Build + revalidate periodically	E-commerce catalogs
Streaming SSR	Server streams chunks with Suspense	Faster TTFB/FCP

Hydration: attaching event listeners to server-rendered HTML. Mismatches (e.g., `Date.now()` or `window` used during render) cause hydration errors.

React Server Components (Next.js App Router):

```
// app/users/page.tsx — Server Component (default): runs only on server, can access DB, zero JS shipped
```

```tsx
import { db } from '@/lib/db';
import LikeButton from './LikeButton';

export default async function UsersPage() {
  const users = await db.user.findMany();
  return users.map(u => <div key={u.id}>{u.name} <LikeButton id={u.id} /></div>);
}

// app/users/LikeButton.tsx — Client Component: interactivity, hooks, browser APIs
'use client';
import { useState } from 'react';
export default function LikeButton({ id }: { id: number }) {
  const [liked, setLiked] = useState(false);
  return <button onClick={() => setLiked(!liked)}>{liked ? '♥' : '♡'}</button>;
}

// Server Action
'use server';
export async function createUser(formData: FormData) {
  await db.user.create({ data: { name: String(formData.get('name')) } });
  revalidatePath('/users');
}
```

3.14 Testing React

UNDERSTAND THE CONCEPT

Automated tests are code that checks your code, so you can change things confidently without manually clicking through the app.

Kinds of tests:

- **Unit tests:** a single function or hook.
- **Component/integration tests:** render a component, interact with it, check the result.
- **End-to-end (E2E) tests:** a real browser drives the whole app (Playwright, Cypress).

React Testing Library encourages testing **the way a user uses the app**: find elements by their visible role or label ("the button named Save"), click and type, then check what appears on screen. Don't test internal details like state variable names. Tests written this way survive refactoring. **Jest** or **Vitest** run the tests and provide `expect` assertions and mocks (fake versions of functions or network calls).

```tsx
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';

test('increments counter', async () => {
  render(<Counter />);
  await userEvent.click(screen.getByRole('button', { name: /increment/i }));
  expect(screen.getByText('Count: 1')).toBeInTheDocument();
});

test('loads users', async () => {
  global.fetch = vi.fn().mockResolvedValue({ ok: true, json: async () => [{ id: 1, name: 'A' }]
});
  render(<Users />);
  expect(await screen.findByText('A')).toBeInTheDocument(); // findBy waits
});
```

```
// Testing a custom hook
import { renderHook, act } from '@testing-library/react';
const { result } = renderHook(() => useCounter());
act(() => result.current.inc());
expect(result.current.count).toBe(1);
```

Query priority: `getByRole` > `getByLabelText` > `getByPlaceholderText` > `getByText` > `getByTestId`. Mock network with MSW for realistic tests.

3.15 Interview Questions Checklist

UNDERSTAND THE CONCEPT

This is a checklist of questions you should be able to answer **in your own words**. For each one, try explaining it out loud as if to a colleague, then check the relevant section above.

- Difference between state and props; why state is immutable.
- What triggers a re-render; how `React.memo`, `useMemo`, `useCallback` relate.
- Why keys matter; why index keys are bad.
- `useEffect` vs `useLayoutEffect`; effect cleanup; StrictMode double invocation.
- Controlled vs uncontrolled components.
- Context performance pitfalls.
- How batching works in React 18.
- What is Fiber / concurrent rendering / Suspense.
- SSR vs CSR vs SSG; hydration; Server Components vs Client Components.
- How to prevent race conditions in data fetching.
- Lifting state up vs global stores; server state vs client state.

Part 4 — Node.js: Basics to Advanced

UNDERSTAND THE CONCEPT

What is Node.js? Node.js is a program that lets you run **JavaScript outside the browser**, mainly on servers. It was created in 2009 by taking Chrome's V8 JavaScript engine and adding the ability to work with files, networks and the operating system. So you can use the same language for the frontend (React) and the backend (APIs, databases).

What is a backend / server? The backend is the part of an application users don't see. It receives HTTP requests from browsers or apps, applies business rules ("is this user allowed?", "is the item in stock?"), reads and writes the database, and sends back responses, usually JSON.

What makes Node special: non-blocking I/O. "I/O" (Input/Output) means talking to the outside world: reading files, querying databases, calling other APIs. These operations are *slow* compared to the CPU. Traditional servers often dedicate one thread per request, and that thread sits idle while waiting. Node uses **one main thread** and never waits. It starts the I/O, moves on to serve other requests, and handles the result when it arrives (the event loop). That makes Node very efficient for apps with many simultaneous connections: APIs, chat, real-time dashboards.

A minimal server:

```
// save as server.js, run with: node server.js, then open http://localhost:3000
const http = require('http');
http.createServer((req, res) => {
  res.end('Hello from Node!');
}).listen(3000);
```

Weakness: heavy CPU calculations (image processing, big loops) block the single thread and freeze *every* request. Such work must be moved to worker threads or other services.

4.1 Architecture

UNDERSTAND THE CONCEPT

Node.js is made of three main pieces:

1. **V8:** the JavaScript engine (the same as in Chrome). It runs your JS code.
2. **libuv:** a C library that provides the **event loop** and handles asynchronous operations. For network operations it uses the operating system's efficient notification systems. For operations that have no async OS version (file system, DNS lookups,

some crypto, compression), it uses a small **thread pool** (4 threads by default) running in the background.
3. **Core modules:** built-in libraries such as `fs` (files), `http` (web servers), `path`, `crypto`, `events` and `stream`.

Your JavaScript runs on a single thread, but the I/O work happens in parallel in the background. Picture a waiter (your JS thread) who takes orders and delivers food while the kitchen (libuv and the OS) cooks many dishes at the same time.

I/O-bound vs CPU-bound: an I/O-bound task spends most of its time *waiting* (database, network), and Node excels at these. A CPU-bound task spends its time *computing* (encryption of big files, video processing), and it blocks Node's thread.

Node.js = **V8** (JS engine) + **libuv** (C library: event loop, thread pool, async I/O) + core bindings (fs, net, crypto, zlib, http).

- **Single-threaded event loop** for JavaScript execution.
- **Non-blocking I/O:** network I/O uses OS async mechanisms (epoll/kqueue/IOCP); file system, DNS lookup, crypto and zlib use the **libuv thread pool** (default 4 threads, `UV_THREADPOOL_SIZE`).
- Great for I/O-bound workloads (APIs, real-time); CPU-heavy work blocks the loop → use worker threads or offload.

4.2 The Node.js Event Loop Phases

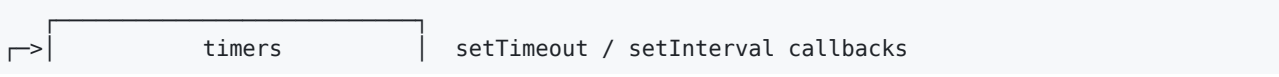
UNDERSTAND THE CONCEPT

You learned the browser event loop in 1.13. Node's event loop follows the same idea but has more structure. It goes round in a cycle of **phases**, and each phase has its own queue of callbacks:

1. **Timers:** callbacks from `setTimeout` and `setInterval` whose time has come.
2. **Pending callbacks:** some system-level callbacks deferred from the previous cycle.
3. **Poll:** the heart of the loop. It waits for new I/O events (incoming requests, finished file reads) and runs their callbacks.
4. **Check:** callbacks scheduled with `setImmediate`.
5. **Close callbacks:** e.g. when a socket closes.

Between every callback, Node first empties two high-priority queues: the **`process.nextTick` queue**, then the **promise microtask queue**.

Practical meaning: `process.nextTick` callbacks run first, then promises, then timers/immediates. Inside an I/O callback, `setImmediate` always runs before `setTimeout(..., 0)`. Most importantly, if your code runs a long synchronous computation, **none** of these phases can run. Every user waits. This is called "**blocking the event loop**".



```
┌─────────────────────────┐
│     pending callbacks   │  deferred I/O callbacks (e.g. TCP errors)
└─────────────────────────┘
┌─────────────────────────┐
│       idle, prepare     │  internal
└─────────────────────────┘
┌─────────────────────────┐
│           poll          │  retrieve new I/O events; run I/O callbacks
└─────────────────────────┘
┌─────────────────────────┐
│           check         │  setImmediate callbacks
└─────────────────────────┘
┌─────────────────────────┐
│      close callbacks    │  socket.on('close')
└─────────────────────────┘
Between every callback: process.nextTick queue, then Promise microtask queue
```

```js
const fs = require('fs');

setTimeout(() => console.log('timeout'), 0);
setImmediate(() => console.log('immediate'));
process.nextTick(() => console.log('nextTick'));
Promise.resolve().then(() => console.log('promise'));
console.log('sync');
// sync, nextTick, promise, then timeout/immediate (order not guaranteed in main module)

fs.readFile(__filename, () => {
  setTimeout(() => console.log('timeout in I/O'), 0);
  setImmediate(() => console.log('immediate in I/O')); // ALWAYS first inside an I/O callback
});
```

`process.nextTick` runs before promise microtasks. Recursive `nextTick` can starve the event loop.

4.3 Modules

UNDERSTAND THE CONCEPT

A **module** is a file whose code is private unless exported. Node supports two systems (also see 1.16):

- **CommonJS** (`require` / `module.exports`): Node's original system. Modules load **synchronously**, and each is **cached** after its first `require` , so every file that requires it gets the *same* instance. That makes modules natural singletons, which is handy for a shared database connection.
- **ES Modules** (`import` / `export`): the modern standard. Enable it with `"type": "module"` in `package.json` or by using `.mjs` files.

`package.json` is your project's ID card. It lists the name and version, **dependencies** (libraries needed to run), **devDependencies** (only needed for development: test tools, TypeScript), and **scripts** (shortcuts like `npm run dev`). **`package-lock.json`** records the *exact* versions installed so everyone gets identical dependencies.

npm (Node Package Manager) downloads libraries from the npm registry into the `node_modules` folder. **Semantic versioning (semver)** is `MAJOR.MINOR.PATCH` . A major bump means breaking changes, minor means new features, patch means bug fixes.

```
// CommonJS
const path = require('node:path');
module.exports = { sum };          // or exports.sum = sum (never reassign `exports`)
// Module wrapper: (function (exports, require, module, __filename, __dirname) { ... })
// Modules are cached after first require → effectively singletons

// ESM — "type": "module" in package.json or .mjs
import { readFile } from 'node:fs/promises';
import path from 'node:path';
import { fileURLToPath } from 'node:url';
const __dirname = path.dirname(fileURLToPath(import.meta.url)); // or import.meta.dirname (Node 20.11+)
```

package.json essentials: `"type"`, `"main"`, `"exports"` (entry points & conditional exports), `"scripts"`, `"engines"`, `dependencies` vs `devDependencies` vs `peerDependencies`. Semver: `^1.2.3` (minor+patch), `~1.2.3` (patch only). Commit lock files; use `npm ci` in CI.

4.4 Core Modules

UNDERSTAND THE CONCEPT

Node comes with **core modules**, built-in libraries that need no installation:

- **`fs`** (file system): read, write, delete and list files and folders. Prefer the promise-based `fs/promises` with `await`. Avoid the `...Sync` functions in servers, because they block the event loop.
- **`path`:** build file paths safely across operating systems (Windows uses `\`, Linux/Mac use `/`).
- **`os`:** information about the machine (CPU count, memory, platform).
- **`url`:** parse and build URLs.
- **`crypto`:** hashing (turning data into a fixed "fingerprint", e.g. SHA-256), random IDs and tokens, encryption, and secure password hashing (`scrypt`).

Hashing vs encryption: a hash is one-way. You cannot get the original back, so use it for passwords. Encryption is two-way. With the key you can decrypt, so use it for data you need to read again.

File System

```
import { readFile, writeFile, mkdir, readdir, stat, rm } from 'node:fs/promises';
import { existsSync, createReadStream } from 'node:fs';

await mkdir('./out', { recursive: true });
await writeFile('./out/a.json', JSON.stringify({ a: 1 }, null, 2));
const data = JSON.parse(await readFile('./out/a.json', 'utf8'));
for (const entry of await readdir('.', { withFileTypes: true })) {
  console.log(entry.name, entry.isDirectory());
}
// Avoid *Sync methods in request handlers — they block the event loop
```

path, os, url, crypto

```
import path from 'node:path';
path.join('a', 'b', '../c');           // 'a/c'
path.resolve('src', 'index.js');       // absolute path
path.extname('file.tar.gz');           // '.gz'

import os from 'node:os';
os.cpus().length; os.freemem(); os.platform();

import crypto from 'node:crypto';
crypto.randomUUID();
crypto.createHash('sha256').update('data').digest('hex');
crypto.randomBytes(32).toString('hex');
// Password hashing: scrypt (built-in) or bcrypt/argon2
const salt = crypto.randomBytes(16);
const hash = crypto.scryptSync('password', salt, 64);
crypto.timingSafeEqual(hashA, hashB); // constant-time comparison
```

4.5 EventEmitter

UNDERSTAND THE CONCEPT

Much of Node is **event-driven**: objects announce that "something happened", and other code that is listening reacts. The `EventEmitter` class provides this:

- `emitter.on('eventName', listener)` subscribes a function.
- `emitter.emit('eventName', data)` announces the event, and every listener is called with the data.
- `emitter.once(...)` listens only for the first occurrence.

Many core objects are EventEmitters: HTTP servers emit `'request'`, streams emit `'data'` and `'end'`, processes emit `'exit'`. It is the **Observer pattern** (1.20).

Note that listeners run **synchronously**, one after another, when `emit` is called. Always listen for `'error'` events. An `'error'` event without a listener crashes the process.

```
import { EventEmitter, once } from 'node:events';

class OrderService extends EventEmitter {
  place(order) {
    // ... save
    this.emit('order:placed', order);
  }
}

const svc = new OrderService();
svc.on('order:placed', (o) => console.log('send email', o.id));
svc.once('order:placed', () => console.log('first order only'));
svc.on('error', (err) => console.error(err)); // unhandled 'error' event crashes the process!
svc.place({ id: 1 });

const [value] = await once(svc, 'order:placed'); // promise-based wait
```

Listeners run **synchronously** in registration order. Default max listeners warning at 10 (`setMaxListeners`).

4.6 Buffers and Streams

UNDERSTAND THE CONCEPT

Buffers. JavaScript strings are text, but servers deal with raw **binary data**: images, files, network packets. A `Buffer` is a fixed-size chunk of raw bytes. You convert between buffers and strings using an *encoding* (utf8, base64, hex).

Streams. Imagine copying a 5 GB video file. Reading it all into memory first could crash the server. A **stream** processes data **piece by piece (in chunks)** as it flows, like drinking through a straw instead of swallowing a whole bottle at once. Memory use stays small and constant, and processing starts immediately.

Four types:

- **Readable:** a source you read from (file read stream, incoming HTTP request).
- **Writable:** a destination you write to (file write stream, HTTP response).
- **Duplex:** both readable and writable (a network socket).
- **Transform:** a duplex that modifies data passing through (compression, encryption).

You connect streams with **`pipeline()`** (or `.pipe()`): `readFile → gzip → writeFile`.

Backpressure: if the destination is slower than the source (slow network, slow disk), data would pile up in memory. Streams handle this automatically. The writable signals "I'm full", and the readable pauses until it drains.

Buffer: fixed-size raw binary data outside the V8 heap.

```
const buf = Buffer.from('hello', 'utf8');
buf.toString('base64');     // 'aGVsbG8='
Buffer.alloc(10);           // zero-filled
Buffer.concat([b1, b2]);
```

Streams process data in chunks → constant memory for huge files. Four types: **Readable**, **Writable**, **Duplex** (socket), **Transform** (zlib, crypto).

```
import { createReadStream, createWriteStream } from 'node:fs';
import { pipeline } from 'node:stream/promises';
import { Transform } from 'node:stream';
import zlib from 'node:zlib';

// Compress a 10GB file using ~constant memory; pipeline handles errors + cleanup
await pipeline(
  createReadStream('big.log'),
  zlib.createGzip(),
  createWriteStream('big.log.gz'),
);

// Custom transform: uppercase
const upper = new Transform({
  transform(chunk, enc, cb) { cb(null, chunk.toString().toUpperCase()); },
});

// Async iteration over a readable
import readline from 'node:readline';
const rl = readline.createInterface({ input: createReadStream('data.csv') });
```

```
for await (const line of rl) { /* process each line */ }

// Streaming an HTTP response
app.get('/download', (req, res) => {
  res.setHeader('Content-Type', 'text/csv');
  createReadStream('report.csv').pipe(res);
});
```

Backpressure: when a writable can't keep up, `write()` returns `false`; wait for `'drain'`. `pipe()` / `pipeline()` handle this automatically.

4.7 HTTP Server from Scratch

UNDERSTAND THE CONCEPT

Node's built-in `http` module can create a web server with no frameworks. You give it one function that is called for **every request**. It receives:

- `req` **(request):** method (`GET` , `POST` ...), URL, headers, and the body, which arrives as a *stream* of chunks.
- `res` **(response):** you set the status code and headers, write the body, and call `end()` .

Doing everything by hand (parsing URLs, routing, reading JSON bodies, handling errors) gets repetitive quickly. That is why frameworks like **Express** exist. Seeing the raw version once helps you understand what frameworks do for you.

```
import http from 'node:http';

const server = http.createServer(async (req, res) => {
  const url = new URL(req.url, `http://${req.headers.host}`);

  if (req.method === 'GET' && url.pathname === '/health') {
    res.writeHead(200, { 'Content-Type': 'application/json' });
    return res.end(JSON.stringify({ ok: true }));
  }

  if (req.method === 'POST' && url.pathname === '/echo') {
    let body = '';
    for await (const chunk of req) body += chunk;
    res.writeHead(200, { 'Content-Type': 'application/json' });
    return res.end(body);
  }

  res.writeHead(404).end('Not found');
});

server.listen(3000, () => console.log('listening on 3000'));
```

4.8 Express.js In Depth

UNDERSTAND THE CONCEPT

Express is the most popular minimal web framework for Node. It adds:

- **Routing:** map an HTTP method + URL pattern to a handler function: `app.get('/users/:id', handler)`. `:id` is a **route parameter**, available as `req.params.id`. Query strings (`?page=2`) appear in `req.query`, and parsed JSON bodies in `req.body`.
- **Middleware:** the central concept. A middleware is a function `(req, res, next)` that runs **in the middle**, between the request arriving and the response being sent. Middleware functions form a **pipeline** in the order you register them. Each one can read or modify `req` / `res`, end the request by sending a response, or call `next()` to pass control to the next one.

```
request → [logger] → [parse JSON] → [check auth] → [route handler] → response
                                          │
                                          └─ not logged in? respond 401 and stop here
```

Typical middleware: logging, parsing bodies, CORS, security headers, authentication, validation, rate limiting.

- **Error-handling middleware** has **four** parameters `(err, req, res, next)` and is registered **last**. When any handler calls `next(error)` or throws, Express skips straight to it. That gives you one central place to format error responses.
- **Routers** (`express.Router()`) group related routes (all `/api/users` routes) into separate files.

Layered architecture keeps code organised as the app grows. **Routes** map URLs. **Controllers** handle HTTP details. **Services** hold the business logic. **Repositories** talk to the database. Each layer has one job, which makes it easy to test and change.

```
import express from 'express';
import helmet from 'helmet';
import cors from 'cors';
import rateLimit from 'express-rate-limit';

const app = express();

// Global middleware (order matters!)
app.use(helmet());                                          // security headers
app.use(cors({ origin: ['https://app.example.com'], credentials: true }));
app.use(express.json({ limit: '1mb' }));
app.use(express.urlencoded({ extended: true }));
app.use(rateLimit({ windowMs: 15 * 60_000, limit: 100 }));

// Custom middleware: request logging
app.use((req, res, next) => {
  const start = Date.now();
  res.on('finish', () => console.log(`${req.method} ${req.originalUrl} ${res.statusCode}
${Date.now() - start}ms`));
  next();
});

// Router
const users = express.Router();
users.get('/', async (req, res) => {
  const { page = 1, limit = 20 } = req.query;
  res.json(await UserService.list({ page: +page, limit: +limit }));
});
users.get('/:id', async (req, res, next) => {
  try {
    const user = await UserService.get(req.params.id);
```

```
    if (!user) throw new NotFoundError('User not found');
    res.json(user);
  } catch (e) { next(e); }   // Express 5 forwards rejected promises automatically
});
app.use('/api/users', users);

// 404 handler
app.use((req, res) => res.status(404).json({ message: 'Route not found' }));

// Error-handling middleware: 4 arguments, registered LAST
class AppError extends Error {
  constructor(message, status = 500) { super(message); this.status = status; }
}
class NotFoundError extends AppError { constructor(m) { super(m, 404); } }

app.use((err, req, res, next) => {
  const status = err.status ?? 500;
  if (status >= 500) console.error(err);
  res.status(status).json({
    message: status >= 500 ? 'Internal Server Error' : err.message,
    ...(process.env.NODE_ENV !== 'production' && { stack: err.stack }),
  });
});

app.listen(process.env.PORT ?? 3000);
```

Async wrapper (Express 4)

```
const asyncHandler = (fn) => (req, res, next) => Promise.resolve(fn(req, res,
next)).catch(next);
router.get('/', asyncHandler(async (req, res) => res.json(await svc.list())));
```

Layered Architecture

```
src/
  routes/        → URL → controller mapping
  controllers/   → parse req, call service, shape res (no business logic)
  services/      → business logic, transactions
  repositories/  → DB access (SQL/ORM)
  models/        → schemas / entities
  middlewares/   → auth, validation, error handling
  config/        → env parsing & validation
  utils/
```

4.9 Authentication and Authorization

UNDERSTAND THE CONCEPT

Authentication (AuthN) answers "**who are you?**" (logging in). **Authorization (AuthZ)** answers "**what are you allowed to do?**" (permissions, roles).

Never store passwords as plain text. Store a **hash** produced by a slow, salted algorithm (bcrypt, argon2). At login, hash the entered password the same way and compare. A **salt** is random data added before hashing, so identical passwords produce different hashes and pre-computed attack tables don't work.

Two common ways to remember a logged-in user:

1. **Sessions (stateful):** after login, the server stores a session record (in memory, a DB or Redis) and gives the browser a cookie containing just a random session ID. Each request sends the cookie, and the server looks up the session. It is easy to log users out instantly.
2. **JWT (JSON Web Token, stateless):** after login, the server creates a **signed** token containing the user's ID and role. The client sends it with each request (usually in the `Authorization: Bearer <token>` header). The server verifies the **signature** with its secret key, so it can trust the contents without a database lookup. Anyone can *read* a JWT, because it is only encoded, not encrypted. They just can't *modify* it without breaking the signature. JWTs are hard to revoke before they expire, so access tokens are kept **short-lived** (minutes) and paired with a longer-lived **refresh token**.

Status codes: `401 Unauthorized` means you are not logged in or the token is invalid. `403 Forbidden` means you are logged in but not allowed to do this.

JWT

```
import jwt from 'jsonwebtoken';
import bcrypt from 'bcrypt';

async function login(req, res) {
  const { email, password } = req.body;
  const user = await User.findOne({ email });
  if (!user || !(await bcrypt.compare(password, user.passwordHash))) {
    return res.status(401).json({ message: 'Invalid credentials' });
  }
  const accessToken = jwt.sign({ sub: user.id, role: user.role }, process.env.JWT_SECRET, {
expiresIn: '15m' });
  const refreshToken = jwt.sign({ sub: user.id }, process.env.REFRESH_SECRET, { expiresIn: '7d'
});

  res.cookie('refreshToken', refreshToken, {
    httpOnly: true, secure: true, sameSite: 'strict', maxAge: 7 * 24 * 3600 * 1000,
  });
  res.json({ accessToken });
}

function authenticate(req, res, next) {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ message: 'No token' });
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch {
    res.status(401).json({ message: 'Invalid or expired token' });
  }
}

const authorize = (...roles) => (req, res, next) =>
  roles.includes(req.user.role) ? next() : res.status(403).json({ message: 'Forbidden' });

router.delete('/:id', authenticate, authorize('admin'), deleteUser);
```

JWT structure: `header.payload.signature` (base64url). Payload is *encoded, not encrypted* — never store secrets in it. Short-lived access token + rotating refresh token; store refresh token in `HttpOnly` cookie. **Sessions** are stateful (server store like Redis), easy to revoke; JWTs are stateless, harder to revoke (use denylist or short expiry).

401 vs 403: 401 = not authenticated; 403 = authenticated but not allowed.

4.10 Databases from Node

UNDERSTAND THE CONCEPT

Applications store data permanently in a **database**. From Node you connect using a **driver** (a library that speaks the database's protocol) or a higher-level tool:

- **Raw driver** (e.g. `pg` for PostgreSQL): you write SQL yourself. It gives full control and the best performance.
- **Query builder** (Knex): build SQL with JavaScript methods.
- **ORM** (Object-Relational Mapper: Prisma, TypeORM, Sequelize): work with database rows as JavaScript objects. The ORM writes the SQL for you.
- **ODM** (Mongoose) does the same for MongoDB, a *document* (NoSQL) database that stores JSON-like documents instead of table rows.

Key concepts:

- **Connection pool:** opening a database connection is slow. A pool keeps several connections open and lends them to requests as needed.
- **Parameterized queries:** always pass user values separately (`$1` , `?`) instead of building SQL strings. This prevents **SQL injection** attacks (see 7.19).
- **Transactions:** group several queries so they all succeed or all fail together. A money transfer must never debit one account without crediting the other.

```
// PostgreSQL with pg — ALWAYS parameterize (prevents SQL injection)
import pg from 'pg';
const pool = new pg.Pool({ connectionString: process.env.DATABASE_URL, max: 10 });

const { rows } = await pool.query('SELECT * FROM users WHERE email = $1', [email]);

// Transaction
const client = await pool.connect();
try {
  await client.query('BEGIN');
  await client.query('UPDATE accounts SET balance = balance - $1 WHERE id = $2', [amt, from]);
  await client.query('UPDATE accounts SET balance = balance + $1 WHERE id = $2', [amt, to]);
  await client.query('COMMIT');
} catch (e) {
  await client.query('ROLLBACK');
  throw e;
} finally {
  client.release();
}

// Mongoose (MongoDB)
const userSchema = new mongoose.Schema({
  email: { type: String, required: true, unique: true, lowercase: true },
  name: String,
  posts: [{ type: mongoose.Schema.Types.ObjectId, ref: 'Post' }],
}, { timestamps: true });
userSchema.index({ email: 1 });
userSchema.pre('save', async function () {
  if (this.isModified('password')) this.password = await bcrypt.hash(this.password, 12);
});
const User = mongoose.model('User', userSchema);
await User.find({ name: /an/i }).populate('posts').lean().limit(10);

// Prisma
```

```
const user = await prisma.user.create({ data: { email, posts: { create: [{ title: 'Hi' }] } }
});
const users = await prisma.user.findMany({ where: { email: { contains: '@x.com' } }, include: {
posts: true } });
await prisma.$transaction([prisma.a.update(...), prisma.b.update(...)]);
```

4.11 Caching with Redis

UNDERSTAND THE CONCEPT

A **cache** stores the result of slow work (database queries, external API calls) in fast storage, so the next request gets it instantly. **Redis** is an extremely fast **in-memory key-value store**, the most common cache for web apps.

Cache-aside pattern (the most common):

1. Request arrives → check the cache.
2. **Hit** (found): return the cached value. That is fast.
3. **Miss** (not found): query the database, save the result in the cache with an expiry time (**TTL**, time-to-live), and return it.
4. When the data changes, delete (**invalidate**) the cached copy.

The hardest part of caching is **invalidation**: making sure users never see outdated data for too long. Redis is also used for sessions, rate limiting, queues and pub/sub messaging.

```
import { createClient } from 'redis';
const redis = createClient({ url: process.env.REDIS_URL });
await redis.connect();

// Cache-aside pattern
async function getProduct(id) {
  const key = `product:${id}`;
  const cached = await redis.get(key);
  if (cached) return JSON.parse(cached);

  const product = await db.product.findUnique({ where: { id } });
  await redis.set(key, JSON.stringify(product), { EX: 300 }); // TTL 5 min
  return product;
}
// Invalidate on write: await redis.del(`product:${id}`)
```

Use cases: caching, sessions, rate limiting (`INCR` + `EXPIRE`), pub/sub, queues (BullMQ), distributed locks.

4.12 Scaling Node.js

UNDERSTAND THE CONCEPT

A single Node process uses **one CPU core** for your JavaScript. Servers have many cores, so how do you use them all, and what about heavy computations?

- **Cluster module / PM2:** start **several copies (processes)** of your app, one per CPU core, all sharing the same port. Incoming connections are distributed among them. If one crashes, a new one is started. In cloud setups you often run multiple containers behind a **load balancer** instead.
- **Worker threads:** run CPU-heavy JavaScript (image resizing, report generation, complex calculations) on a **separate thread**, so the main event loop stays free to answer requests.
- **Child processes:** run *other programs* (a Python script, `ffmpeg`, shell commands) from Node.

Horizontal scaling (more machines or processes) generally requires your app to be **stateless**: don't keep sessions or uploaded files in the memory of one process. Store them in Redis, a database or object storage, so any instance can handle any request.

Cluster (multi-process on one machine)

```
import cluster from 'node:cluster';
import os from 'node:os';

if (cluster.isPrimary) {
  for (let i = 0; i < os.availableParallelism(); i++) cluster.fork();
  cluster.on('exit', (w) => { console.log(`worker ${w.process.pid} died`); cluster.fork(); });
} else {
  startServer();  // workers share the same port
}
// In production often PM2 (`pm2 start app.js -i max`) or Kubernetes replicas instead.
```

Worker Threads (CPU-bound work)

```
// main.js
import { Worker } from 'node:worker_threads';
function runHeavy(n) {
  return new Promise((resolve, reject) => {
    const w = new Worker(new URL('./fib-worker.js', import.meta.url), { workerData: n });
    w.once('message', resolve);
    w.once('error', reject);
  });
}

// fib-worker.js
import { parentPort, workerData } from 'node:worker_threads';
const fib = (n) => (n < 2 ? n : fib(n - 1) + fib(n - 2));
parentPort.postMessage(fib(workerData));
```

	`cluster`	`worker_threads`	`child_process`
Unit	Process	Thread	Process
Memory	Separate	Shared possible (`SharedArrayBuffer`)	Separate
Use	Scale HTTP across cores	CPU-intensive JS	Run external commands/scripts

child_process

```
import { exec, spawn, execFile } from 'node:child_process';
spawn('ls', ['-la'], { stdio: 'inherit' });   // streams, for long-running/large output
execFile('git', ['status'], (err, stdout) => console.log(stdout)); // safer than exec (no shell)
```

4.13 Background Jobs and Queues

UNDERSTAND THE CONCEPT

Some work shouldn't happen during the HTTP request: sending emails, generating PDFs, processing videos, calling slow third-party APIs. The user shouldn't wait for it, and if it fails it should be **retried** automatically.

A **job queue** solves this. The web server quickly **adds a job** to a queue (stored in Redis) and responds immediately. Separate **worker** processes take jobs from the queue and process them in the background, with retries, delays, priorities and scheduling. **BullMQ** is the popular Redis-based queue for Node (similar to Celery in Python, see 6.14).

```
import { Queue, Worker } from 'bullmq';
const emailQueue = new Queue('emails', { connection });

await emailQueue.add('welcome', { userId: 1 }, { attempts: 3, backoff: { type: 'exponential',
delay: 1000 } });

new Worker('emails', async (job) => {
  await sendWelcomeEmail(job.data.userId);
}, { connection, concurrency: 5 });
```

4.14 Real-time: WebSockets / Socket.IO

UNDERSTAND THE CONCEPT

Normal HTTP is **request → response**: the client always asks first, and the server can't push data on its own. For **real-time** features (chat, live notifications, multiplayer games, live dashboards) you need the server to send data whenever something happens.

- **WebSockets** open a **permanent two-way connection** between browser and server. Either side can send messages at any time.
- **Socket.IO** is a library on top of WebSockets that adds automatic reconnection, **rooms** (broadcast to a group, such as one chat room), events and fallbacks.
- **Server-Sent Events (SSE)** are a simpler one-way stream from server to client over normal HTTP, good for notifications and progress updates.

When running several server instances, use a shared message channel (e.g. the Redis adapter) so a message sent from one instance reaches clients connected to the others.

```
import { Server } from 'socket.io';
const io = new Server(httpServer, { cors: { origin: '*' } });

io.use((socket, next) => {               // auth middleware
  try { socket.user = jwt.verify(socket.handshake.auth.token, SECRET); next(); }
  catch { next(new Error('unauthorized')); }
});

io.on('connection', (socket) => {
  socket.join(`room:${socket.user.sub}`);
  socket.on('chat:message', (msg) => io.to(msg.roomId).emit('chat:message', msg));
  socket.on('disconnect', () => console.log('bye'));
});
// Scale horizontally with the Redis adapter.
```

Alternatives: Server-Sent Events (one-way server → client over HTTP), long polling.

4.15 Error Handling and Process Management

UNDERSTAND THE CONCEPT

There are two kinds of errors in a server:

- **Operational errors:** expected problems in normal operation, such as invalid user input, a missing record, a timeout from another service. **Handle them**: return a proper status code and message.
- **Programmer errors:** bugs, such as reading a property of `undefined` or calling a function wrongly. The process may be in a broken state. The safest response is to **log it, crash, and let a process manager restart** the app.

`unhandledRejection` / `uncaughtException` are last-resort handlers for errors nobody caught. Log them and exit.

Graceful shutdown: when the server is told to stop (during a deploy), it should stop accepting new requests, **finish the requests in progress**, close database connections, and then exit. Users then don't get errors in the middle of a deploy.

```
// Operational errors (expected: invalid input, timeouts) → handle & respond
// Programmer errors (bugs) → crash and let the process manager restart

process.on('unhandledRejection', (reason) => { logger.error(reason); throw reason; });
process.on('uncaughtException', (err) => { logger.fatal(err); process.exit(1); });

// Graceful shutdown
const server = app.listen(3000);
async function shutdown(signal) {
  console.log(`${signal} received`);
  server.close(async () => {          // stop accepting new connections
    await pool.end();
    await redis.quit();
    process.exit(0);
  });
  setTimeout(() => process.exit(1), 10_000).unref(); // force exit
}
process.on('SIGTERM', shutdown);
process.on('SIGINT', shutdown);
```

4.16 Configuration and Logging

UNDERSTAND THE CONCEPT

Configuration (database URLs, API keys, port numbers) must not be hard-coded. It differs between your laptop, testing and production, and secrets must never be committed to Git. Use **environment variables**, often loaded from a `.env` file during development. **Validate them at startup** so the app fails immediately with a clear message if something is missing, instead of failing mysteriously later.

Logging records what your application is doing so you can debug problems in production, where you can't attach a debugger. Use **log levels** (`debug` , `info` , `warn` , `error` , `fatal`) and **structured logs** (JSON with fields like `userId` , `requestId`) so they can be searched in log tools. `pino` and `winston` are popular loggers. Never log passwords, tokens or personal data.

```
// Validate env at startup — fail fast
import { z } from 'zod';
const Env = z.object({
  NODE_ENV: z.enum(['development', 'production', 'test']).default('development'),
  PORT: z.coerce.number().default(3000),
  DATABASE_URL: z.string().url(),
  JWT_SECRET: z.string().min(32),
});
export const env = Env.parse(process.env);
// Node 20.6+: node --env-file=.env app.js

// Structured logging with pino
import pino from 'pino';
const logger = pino({ level: env.NODE_ENV === 'production' ? 'info' : 'debug' });
logger.info({ userId: 1, orderId: 9 }, 'order placed');
```

4.17 Security Checklist

UNDERSTAND THE CONCEPT

Security is about assuming that **anything coming from outside is potentially malicious**. The most common risks for Node APIs (many come from the **OWASP Top 10** list):

- **Injection** (SQL/NoSQL): attacker input changes your query. Prevent it with parameterized queries and validation.
- **Broken authentication / access control:** users accessing other users' data. Always check ownership on the server.
- **Sensitive data exposure:** use HTTPS, hash passwords, don't leak stack traces in production.
- **Misconfiguration:** missing security headers (use `helmet`), CORS that is too open.
- **Vulnerable dependencies:** run `npm audit` and keep packages updated.
- **Denial of service:** limit request rates and body sizes, and avoid regexes that can take exponential time.

- Validate and sanitize all input (Zod/Joi); parameterized queries (SQL injection); avoid `eval`.
- `helmet` for headers; strict CORS; rate limiting; request size limits.
- Hash passwords with bcrypt/argon2; never log secrets/tokens.
- NoSQL injection: reject objects where strings are expected (`{ "$gt": "" }`).
- Prototype pollution: don't deep-merge untrusted objects.
- `npm audit`, Dependabot; pin versions; lockfiles.
- HTTPS, secure cookies (`HttpOnly`, `Secure`, `SameSite`), CSRF protection for cookie auth.
- Avoid ReDoS (catastrophic regex backtracking); avoid blocking sync calls.

4.18 Performance

UNDERSTAND THE CONCEPT

For a Node server, performance mostly comes down to **keeping the event loop free** and **avoiding unnecessary work**:

- Never run long synchronous work in a request handler.
- Use streams for big data, and pagination for big lists.
- Reuse database connections (pools) and HTTP connections (keep-alive).
- Cache what is expensive and frequently requested.
- **Measure before optimising:** use profilers (`node --inspect` with Chrome DevTools, `clinic.js`) to find the real bottleneck instead of guessing.

- Never block the event loop: avoid sync fs/crypto, giant `JSON.parse`, heavy loops in handlers.
- Use streams for large data; pagination for large queries.
- Connection pooling for DBs; HTTP keep-alive agents.
- Cache (in-memory LRU, Redis, HTTP cache headers / ETags), compression.
- Profile: `node --inspect`, `--prof`, `clinic.js`, `0x` flame graphs; measure event loop lag (`perf_hooks.monitorEventLoopDelay`).
- Memory leaks: heap snapshots in Chrome DevTools; watch for growing global caches, listeners, closures.

4.19 Testing Node

UNDERSTAND THE CONCEPT

For Node APIs you usually write:

- **Unit tests:** test services and helper functions in isolation, replacing the database or external APIs with **mocks** (fake implementations).
- **Integration tests:** send real HTTP requests to your Express app (using **Supertest**, without starting a real network server) and check the status code and JSON. Often run against a test database.

Test runners: **Jest**, **Vitest**, or Node's built-in `node:test`. A good test is independent (doesn't rely on other tests), repeatable and fast.

```
// Jest/Vitest + Supertest
import request from 'supertest';
import { app } from '../src/app';

describe('GET /api/users/:id', () => {
  it('returns 404 for unknown user', async () => {
    const res = await request(app).get('/api/users/999');
    expect(res.status).toBe(404);
  });

  it('returns user', async () => {
    vi.spyOn(UserService, 'get').mockResolvedValue({ id: 1, name: 'A' });
    const res = await request(app).get('/api/users/1').set('Authorization', `Bearer ${token}`);
    expect(res.body).toMatchObject({ id: 1 });
  });
});

// Built-in test runner (Node 18+)
import { test } from 'node:test';
import assert from 'node:assert/strict';
test('adds', () => assert.equal(1 + 1, 2));
```

4.20 REST API Design Quick Reference

UNDERSTAND THE CONCEPT

REST (REpresentational State Transfer) is a set of conventions for designing HTTP APIs:

- Everything is a **resource** identified by a URL using nouns, not verbs: `/users`, `/users/42`, `/users/42/orders`.
- The **HTTP method** says what to do: GET reads, POST creates, PUT replaces, PATCH partially updates, DELETE removes.
- **Status codes** say how it went: `2xx` success, `3xx` redirect, `4xx` client made a mistake, `5xx` server failed.
- **Stateless:** each request carries everything needed (e.g. its auth token). The server doesn't remember previous requests.

Idempotent means doing the same request several times has the same effect as doing it once. GET, PUT and DELETE are idempotent. POST is not: two POSTs create two records. That's why payment APIs use **idempotency keys**.

Method	Purpose	Idempotent	Typical success
GET	Read	Yes	200
POST	Create	No	201 + Location
PUT	Replace	Yes	200 / 204
PATCH	Partial update	Not necessarily	200
DELETE	Remove	Yes	204

Status codes: 200 OK, 201 Created, 204 No Content, 301/302 Redirect, 304 Not Modified, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 422 Unprocessable Entity, 429 Too Many Requests, 500 Internal Error, 502 Bad Gateway, 503 Service Unavailable.

Good practices: plural nouns (`/users/42/orders`), versioning (`/v1`), pagination (`?page=&limit=` or cursor-based), filtering/sorting via query params, consistent error shape, idempotency keys for payments, OpenAPI documentation.

Part 5 — Python: Basics to Advanced

UNDERSTAND THE CONCEPT

What is Python? Python is a general-purpose programming language created by Guido van Rossum in 1991. It is famous for **readability**: its code looks almost like English, and it uses **indentation** (spaces at the start of lines) instead of curly braces to group code. It is used for web backends (Django, FastAPI, Flask), data analysis, machine learning and AI, automation scripts, and much more.

Your first program (save as `hello.py` and run `python hello.py`):

```
name = "World"                          # a variable — no `let`/`const` needed
print(f"Hello, {name}!")                # f-string: insert values inside {}

def square(n):                          # define a function with `def`
    return n * n                        # the indented block is the function body

if square(5) > 20:                      # a condition — note the colon and indentation
    print("big number")

for i in range(1, 4):                   # loop: i = 1, 2, 3
    print("Count", i)

fruits = ["apple", "banana"]            # a list
person = {"name": "Anish", "age": 30}  # a dictionary (key → value)
print(person["name"])
```

Python vs JavaScript, quick map: `def` ↔ `function`; `None` ↔ `null`; `True/False` ↔ `true/false`; `and/or/not` ↔ `&&/||/!`; list `[]` ↔ array `[]`; dict `{}` ↔ object `{}`; indentation ↔ `{ }`; `#` comment ↔ `//`.

The Zen of Python (type `import this`) sums up the philosophy: "Readability counts", "Simple is better than complex", "There should be one obvious way to do it".

5.1 How Python Runs

UNDERSTAND THE CONCEPT

When you run `python app.py`, the standard interpreter (called **CPython**, because it's written in C) first **compiles** your code into **bytecode**, simple instructions stored in `__pycache__` folders, and then a **virtual machine** executes that bytecode line by line.

Everything in Python is an object: numbers, strings, functions, classes, even modules. Every object has:

- an **identity**, its address in memory (`id(x)`);
- a **type** (`type(x)`), which decides what operations it supports;
- a **value**.

Variables are labels (names), not boxes. `a = [1, 2]` creates a list object and sticks the label `a` on it. `b = a` sticks a *second label* on the **same** object. It doesn't copy. That's why changing the list through `b` is visible through `a`.

`is` vs `==`: `==` asks "do they have equal *values*?" `is` asks "are they the *same object*?" Use `is` only for `None` (`if x is None`).

Mutable vs immutable. *Mutable* objects can be changed in place (lists, dicts, sets). *Immutable* objects cannot (ints, strings, tuples). "Changing" an immutable value actually creates a new object. Only immutable values can be dictionary keys or set members, because they must be *hashable* (have a stable hash value).

Dynamic but strong typing: a variable can refer to any type (dynamic), but Python won't silently mix incompatible types. `"1" + 1` is an error, unlike JavaScript.

CPython compiles source to **bytecode** (`.pyc` in `__pycache__`) and executes it on a stack-based virtual machine. Everything is an **object** with an identity (`id()`), a type and a value. Python is **dynamically** and **strongly** typed (`'1' + 1` raises `TypeError`).

Names are references. Assignment binds a name to an object; it never copies.

```
a = [1, 2]
b = a           # same object
b.append(3)
print(a)        # [1, 2, 3]
a is b          # True  (identity)
a == [1, 2, 3]  # True  (equality)

x = 256; y = 256; x is y   # True — small ints (-5..256) are cached; never rely on `is` for values
```

Mutable: `list`, `dict`, `set`, `bytearray`, user objects. **Immutable:** `int`, `float`, `bool`, `str`, `tuple`, `frozenset`, `bytes`, `None`.

5.2 Core Data Types

UNDERSTAND THE CONCEPT

Python's built-in data types cover most needs:

- **`int`, `float`:** whole numbers (unlimited size) and decimal numbers. Use `decimal.Decimal` for money, because floats have rounding errors.
- **`str`:** text. It is immutable, and has many useful methods (`split`, `join`, `strip`, `replace`, `upper`). **f-strings** (`f"Hi {name}"`) are the modern way to format text.
- **`bool`:** `True` / `False`. **`None`** means "no value".
- **`list`:** an ordered, changeable collection: `[1, "a", 3.5]`. It is like a JS array.
- **`tuple`:** an ordered, **unchangeable** collection: `(1, 2)`. Use it for fixed groups of values, such as coordinates or returning multiple values from a function.
- **`set`:** an unordered collection of **unique** items, with very fast membership checks (`x in s`) and math-like operations (union, intersection).

- `dict` : a **key → value** mapping, `{"name": "A"}` . It is the most important Python data structure, with very fast lookup by key. It keeps insertion order.

Indexing and slicing: positions start at 0. Negative indexes count from the end (`-1` is the last). Slicing `s[start:stop:step]` extracts a part. `stop` is *not* included.

Choosing a structure: need order and duplicates? Use a list. Need uniqueness or fast "is it there?" checks? Use a set. Need to look things up by a name or ID? Use a dict. Need a fixed record? Use a tuple or dataclass.

Strings

```
s = "Hello, World"
s.lower(), s.upper(), s.title()
s.split(", ")              # ['Hello', 'World']
", ".join(["a", "b"])      # 'a, b'
s.strip(), s.replace("l", "L"), s.startswith("He"), s.find("o")
s[::-1]                    # reverse
s[0:5]                     # slicing [start:stop:step]

name, score = "Anish", 92.456
f"{name:>10} | {score:.2f} | {score:08.3f} | {1_000_000:,}"
f"{name=}"                 # "name='Anish'"  (debug)
```

Lists, Tuples, Sets, Dicts

```
nums = [3, 1, 2]
nums.append(4); nums.extend([5, 6]); nums.insert(0, 0)
nums.pop(); nums.remove(3)
sorted(nums, reverse=True)       # new list
nums.sort(key=lambda x: -x)      # in-place

t = (1, 2, 3)
a, *rest = t                     # unpacking: a=1, rest=[2,3]
single = (1,)                    # comma makes the tuple

s = {1, 2, 3}
s | {4}, s & {1, 9}, s - {1}, s ^ {1, 9}   # union, intersection, difference, sym. diff

d = {"a": 1, "b": 2}
d.get("z", 0)                    # default if missing
d.setdefault("c", []).append(1)
d.items(), d.keys(), d.values()
merged = d | {"b": 99}           # 3.9+ merge
{**d, "x": 1}                    # unpack merge
del d["a"]; d.pop("b", None)
```

Time complexity

Operation	list	dict / set
Index / lookup	O(1) index, O(n) `in`	O(1) average
Append	O(1) amortized	O(1)
Insert/delete at start	O(n)	—
Sort	O(n log n)	—

Use `collections.deque` for O(1) appends/pops at both ends.

5.3 Control Flow and Comprehensions

UNDERSTAND THE CONCEPT

Control flow decides which code runs and how often:

- **`if / elif / else :`** choose between branches.
- **`for item in collection :`** loop over any iterable (list, string, dict, file lines). Use `range(n)` for counting, `enumerate()` to get index and value together, and `zip()` to loop over several lists in parallel.
- **`while condition :`** repeat as long as the condition is true.
- **`break`** exits a loop early. **`continue`** skips to the next iteration.

Comprehensions are a concise, very "Pythonic" way to build a new list, dict or set from another collection, combining a loop and an optional filter in one line:

```
# the long way
squares = []
for x in range(10):
    if x % 2 == 0:
        squares.append(x * x)

# the comprehension way — same result
squares = [x * x for x in range(10) if x % 2 == 0]
```

Read it as: "give me `x * x` **for** each `x` in `range(10)` **if** `x` is even". A **generator expression** uses `( )` instead of `[ ]` and produces values lazily, one at a time, without building the whole list in memory.

Pattern matching (`match` / `case` , Python 3.10+) compares a value against *shapes* (dict structure, list length, class type) and extracts parts of it. It is far more powerful than a simple switch statement.

```
# Comprehensions
squares = [x * x for x in range(10) if x % 2 == 0]
matrix_t = [[row[i] for row in matrix] for i in range(len(matrix[0]))]
word_len = {w: len(w) for w in ["hi", "there"]}
unique = {x % 3 for x in range(10)}
gen = (x * x for x in range(10**9))   # generator expression — lazy, O(1) memory

# for/else — else runs if loop wasn't broken
for n in nums:
    if n < 0:
        break
else:
    print("no negatives")

# Walrus operator (3.8+)
if (n := len(data)) > 10:
    print(f"too long: {n}")

# enumerate / zip
for i, (name, age) in enumerate(zip(names, ages), start=1):
    print(i, name, age)
```

```python
# Structural pattern matching (3.10+)
def handle(event):
    match event:
        case {"type": "click", "x": x, "y": y}:
            return f"click at {x},{y}"
        case {"type": "key", "key": "Enter" | "Return"}:
            return "submit"
        case [first, *others]:
            return f"list starting with {first}"
        case Point(x=0, y=0):
            return "origin"
        case _:
            return "unknown"
```

5.4 Functions

UNDERSTAND THE CONCEPT

A **function**, defined with `def`, is a named, reusable block of code. Python has very flexible parameter options:

- **Positional arguments:** matched by position, e.g. `greet("A", "Hi")`.
- **Keyword arguments:** matched by name, e.g. `greet(name="A", greeting="Hi")`. These are clearer and the order doesn't matter.
- **Default values:** `def greet(name, greeting="Hello")`.
- `*args` collects extra positional arguments into a tuple. `**kwargs` collects extra keyword arguments into a dict. They are used to write flexible functions and wrappers.

The mutable default argument trap: default values are created **once**, when the function is defined, not each time it is called. So a default list or dict is *shared between calls* and keeps growing. Use `None` as the default and create the list inside the function.

`lambda` creates a tiny anonymous one-expression function, e.g. `lambda x: x * 2`. It is used as a short argument to `sorted(key=...)`, `map` and `filter`.

Scope (LEGB rule): when Python looks up a name, it searches the **L**ocal function, then any **E**nclosing functions, then the **G**lobal (module) level, then **B**uilt-ins (`len`, `print`). To *assign* to an outer variable you must declare it with `global` or `nonlocal`. Without that, assigning creates a new local variable.

Closures work just like in JavaScript: an inner function remembers variables from the outer function that created it.

```python
def greet(name, greeting="Hello", *args, sep=", ", **kwargs):
    return f"{greeting}{sep}{name}"

greet("A", "Hi", sep=" ")

# Positional-only (/) and keyword-only (*) parameters
def f(a, b, /, c, *, d):
    ...
f(1, 2, 3, d=4)       # a,b positional-only; d keyword-only
```

```python
# Mutable default argument pitfall
def add_item(item, bucket=[]):     # BAD — same list shared across calls
    bucket.append(item)
    return bucket

def add_item(item, bucket=None):   # GOOD
    if bucket is None:
        bucket = []
    bucket.append(item)
    return bucket

# Lambdas, map/filter, functools
from functools import reduce, partial
square = lambda x: x * x
list(map(square, [1, 2]))
list(filter(lambda x: x > 1, [1, 2, 3]))
reduce(lambda acc, x: acc + x, [1, 2, 3], 0)
int_from_bin = partial(int, base=2)
int_from_bin("101")   # 5
```

Scope — LEGB

Lookup order: **L**ocal → **E**nclosing → **G**lobal → **B**uilt-in.

```python
count = 0
def outer():
    total = 0
    def inner():
        nonlocal total       # modify enclosing variable
        global count         # modify global
        total += 1
        count += 1
        return total
    return inner

counter = outer()
counter(); counter()         # 2  — closure keeps `total` alive
```

5.5 Decorators

UNDERSTAND THE CONCEPT

A **decorator** is a function that **wraps another function to add behaviour before and/or after it**, without changing that function's code. You apply it with `@decorator_name` on the line above a function definition.

Think of gift wrapping: the gift (your function) stays the same, but the wrapping adds something around it. Common uses are timing, logging, caching results, retrying on failure, and checking permissions. Django's `@login_required` is one example.

How it works, step by step:

1. Functions are objects, so they can be passed to other functions.
2. A decorator receives the original function and defines an inner **wrapper** function that does extra work and calls the original.
3. The decorator returns the wrapper.
4. `@timer` above `def f` is just shorthand for `f = timer(f)`. The name `f` now points to the wrapper.

```python
def shout(func):
    def wrapper(*args, **kwargs):
        result = func(*args, **kwargs)     # call the original
        return result.upper()              # add behaviour after
    return wrapper

@shout
def greet(name):
    return f"hello {name}"

greet("anish")   # 'HELLO ANISH'
```

Always use `@functools.wraps(func)` on the wrapper so the decorated function keeps its original name and docstring. A **decorator with arguments** (`@retry(times=3)`) needs one extra outer layer: a function that receives the arguments and returns the actual decorator.

A decorator is a callable that takes a function and returns a new function.

```python
import time
import functools

def timer(func):
    @functools.wraps(func)                # preserves __name__, __doc__
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        print(f"{func.__name__} took {time.perf_counter() - start:.4f}s")
        return result
    return wrapper

# Decorator with arguments → three levels
def retry(times=3, exceptions=(Exception,), delay=0.5):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(1, times + 1):
                try:
                    return func(*args, **kwargs)
                except exceptions:
                    if attempt == times:
                        raise
                    time.sleep(delay * 2 ** (attempt - 1))
        return wrapper
    return decorator

@timer
@retry(times=5, exceptions=(ConnectionError,))
def call_api():
    ...

# Built-in caching
@functools.lru_cache(maxsize=128)
def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)

# Class-based decorator
class CountCalls:
    def __init__(self, func):
        functools.update_wrapper(self, func)
        self.func, self.calls = func, 0
    def __call__(self, *args, **kwargs):
        self.calls += 1
        return self.func(*args, **kwargs)
```

Decorators are applied bottom-up: `@a @b def f` ⇒ `f = a(b(f))`.

5.6 Iterators and Generators

UNDERSTAND THE CONCEPT

Iteration is going through items one by one. Anything you can loop over with `for` is an **iterable** (lists, strings, dicts, files). Behind the scenes, `for` asks the iterable for an **iterator**, an object with a `__next__()` method that returns the next item each time and raises `StopIteration` when there are no more.

A **generator** is the easiest way to create an iterator. It's a function that uses `yield` instead of `return`. Each `yield` produces one value and **pauses** the function, keeping all its local variables. The next request resumes right after the `yield`.

Why this matters: **laziness**. A generator produces values *on demand*, so it can process a 50 GB log file or an endless stream using almost no memory, because only one item exists at a time. You can chain generators into processing **pipelines**: read lines → filter errors → parse → take the first 10.

`itertools` and `collections` are standard-library toolboxes. `itertools` has tools for combining, slicing and grouping iterators. `collections` adds `Counter` (count things), `defaultdict` (a dict with automatic default values), `deque` (a fast double-ended queue) and `namedtuple` (a tuple with named fields).

```
# Iterator protocol: __iter__ returns self, __next__ raises StopIteration
class Countdown:
    def __init__(self, start): self.current = start
    def __iter__(self): return self
    def __next__(self):
        if self.current <= 0:
            raise StopIteration
        self.current -= 1
        return self.current + 1

# Generator — much simpler
def countdown(n):
    while n > 0:
        yield n
        n -= 1

def read_large_file(path):
    with open(path) as f:
        for line in f:          # file objects are lazy iterators
            yield line.rstrip("\n")

# Pipeline of generators — processes GBs with constant memory
lines = read_large_file("app.log")
errors = (l for l in lines if "ERROR" in l)
first_10 = list(itertools.islice(errors, 10))

# yield from — delegate to sub-generator
def flatten(items):
    for x in items:
        if isinstance(x, list):
            yield from flatten(x)
        else:
            yield x
```

```python
# send() — coroutine-style generator
def averager():
    total = count = 0
    avg = None
    while True:
        value = yield avg
        total += value; count += 1
        avg = total / count
g = averager(); next(g); g.send(10); g.send(20)  # 15.0
```

itertools / collections

```python
import itertools as it
from collections import Counter, defaultdict, deque, namedtuple, OrderedDict, ChainMap

it.chain([1], [2]); it.product("ab", repeat=2); it.permutations([1, 2, 3], 2)
it.combinations([1, 2, 3], 2); it.groupby(sorted(data, key=k), key=k)
it.accumulate([1, 2, 3]); it.count(10); it.cycle("ab"); it.batched(range(10), 3)  # 3.12

Counter("mississippi").most_common(2)     # [('i', 4), ('s', 4)]
graph = defaultdict(list); graph["a"].append("b")
q = deque(maxlen=3); q.appendleft(1); q.rotate(1)
Point = namedtuple("Point", "x y")
```

5.7 Object-Oriented Python

UNDERSTAND THE CONCEPT

Object-Oriented Programming (OOP) organises code around **objects** that bundle data (**attributes**) and behaviour (**methods**). A **class** is the blueprint, and an **object/instance** is one thing built from it. (The concepts are the same as in JavaScript, see 1.8.)

Python specifics:

- `__init__` is the initialiser, called automatically when you create an object: `Rectangle(3, 4)`.
- `self` is the first parameter of every instance method and refers to the object the method was called on. Python passes it automatically. It is like JS's `this`, but explicit.
- **Instance attributes** (`self.name`) are separate for each object. **Class attributes** (defined directly in the class body) are shared by all instances.
- `@classmethod` receives the *class* (`cls`) instead of an instance. It is often used as an alternative constructor (`Rectangle.square(3)`). `@staticmethod` receives neither. It's just a utility function kept inside the class.
- `@property` makes a method behave like an attribute. You use it for computed values or to validate values when they are set.
- **Privacy is by convention:** `_name` means "internal, please don't touch". `__name` triggers *name mangling* (renamed internally) to avoid clashes in subclasses. Nothing is truly private.
- **Inheritance:** `class Rectangle(Shape)` reuses and extends `Shape`. `super()` calls the parent's version of a method.
- **Abstract base classes (`ABC`)** define methods subclasses *must* implement.

Dunder ("double underscore") methods such as `__str__`, `__eq__`, `__add__` and `__len__` let your objects work with Python's built-in syntax: `print(obj)`, `a == b`, `a + b`, `len(obj)`, `obj[i]`, `for x in obj`. Python calls them automatically. This is how Python's "protocols" work.

MRO (Method Resolution Order): with multiple inheritance, Python follows a fixed, predictable order to find which parent's method to use (shown by `Class.__mro__`). `super()` follows this order. Django's class-based views rely heavily on this through **mixins** (small classes that add one feature each).

Dataclasses (`@dataclass`) automatically generate `__init__`, `__repr__` and `__eq__` for classes that mainly hold data, which removes a lot of boilerplate.

```python
from abc import ABC, abstractmethod

class Shape(ABC):
    count = 0                           # class attribute (shared)

    def __init__(self, name: str):
        self.name = name                # instance attribute
        Shape.count += 1

    @abstractmethod
    def area(self) -> float: ...

    @classmethod
    def total(cls) -> int:              # receives the class — alternative constructors
        return cls.count

    @staticmethod
    def unit() -> str:                  # no self/cls — utility namespaced in class
        return "cm"

    def __str__(self):  return f"{self.name}: {self.area():.2f}{self.unit()}²"   # user-facing
    def __repr__(self): return f"{type(self).__name__}(name={self.name!r})"      # dev-facing


class Rectangle(Shape):
    def __init__(self, w: float, h: float):
        super().__init__("rect")
        self._w = w                     # "protected" by convention
        self.__secret = 1               # name-mangled → _Rectangle__secret
        self.h = h

    @property
    def w(self):                        # getter
        return self._w

    @w.setter
    def w(self, value):
        if value <= 0:
            raise ValueError("width must be positive")
        self._w = value

    def area(self):
        return self._w * self.h

    @classmethod
    def square(cls, side):
        return cls(side, side)


r = Rectangle.square(3)
```

```
r.w = 4
print(r, repr(r), Shape.total())
```

Dunder (Magic) Methods

```
class Money:
    __slots__ = ("amount", "currency")         # no __dict__ → less memory, faster attrs

    def __init__(self, amount, currency="INR"):
        self.amount, self.currency = amount, currency
    def __add__(self, other):                  # +
        self._check(other)
        return Money(self.amount + other.amount, self.currency)
    def __eq__(self, other):                   # ==
        return (self.amount, self.currency) == (other.amount, other.currency)
    def __lt__(self, other): return self.amount < other.amount
    def __hash__(self): return hash((self.amount, self.currency))  # needed if __eq__ defined
    def __bool__(self): return self.amount != 0
    def __repr__(self): return f"Money({self.amount}, {self.currency!r})"
    def _check(self, o):
        if o.currency != self.currency:
            raise ValueError("currency mismatch")

# Container protocol
class Playlist:
    def __init__(self, songs): self._songs = list(songs)
    def __len__(self): return len(self._songs)
    def __getitem__(self, i): return self._songs[i]        # enables indexing, slicing,
iteration
    def __contains__(self, s): return s in self._songs
    def __iter__(self): return iter(self._songs)
    def __call__(self): return self._songs[0]              # instance is callable
```

Inheritance and MRO

```
class A:
    def hello(self): return "A"
class B(A):
    def hello(self): return "B" + super().hello()
class C(A):
    def hello(self): return "C" + super().hello()
class D(B, C):
    pass

D().hello()   # 'BCA'
D.__mro__     # (D, B, C, A, object) — C3 linearization; super() follows the MRO, not the parent
```

Mixins add reusable behaviour (heavily used in Django CBVs: `LoginRequiredMixin`, etc.).

Dataclasses

```
from dataclasses import dataclass, field, asdict

@dataclass(frozen=True, slots=True, order=True)
class Employee:
    name: str
    salary: float = 0.0
    skills: list[str] = field(default_factory=list)   # never use a mutable default directly

    def __post_init__(self):
        if self.salary < 0:
            raise ValueError("salary < 0")
```

```
e = Employee("A", 100, ["python"])
asdict(e)
```

Also know: `typing.NamedTuple`, `enum.Enum`, `Pydantic` models (validation; used by FastAPI, Django Ninja).

5.8 Context Managers

UNDERSTAND THE CONCEPT

Some resources must always be **released** after use, even if an error occurs: files must be closed, database transactions committed or rolled back, locks released. Forgetting this leads to leaks and corrupted data.

A **context manager**, used with the `with` statement, guarantees the setup and cleanup:

```
with open("data.txt") as f:     # setup: open the file
    content = f.read()          # use it
# cleanup: the file is closed automatically here — even if read() raised an error
```

Under the hood, an object is a context manager if it has `__enter__` (runs at the start, and its return value goes into `as f`) and `__exit__` (always runs at the end and receives any exception). The `@contextmanager` decorator lets you write one as a simple generator: code before `yield` is the setup and code after it is the cleanup. Django's `transaction.atomic()` is a context manager.

```
with open("a.txt") as f, open("b.txt", "w") as g:
    g.write(f.read())

# Class-based
class Timer:
    def __enter__(self):
        self.start = time.perf_counter()
        return self
    def __exit__(self, exc_type, exc, tb):
        self.elapsed = time.perf_counter() - self.start
        return False       # True would suppress the exception

# Generator-based
from contextlib import contextmanager, suppress

@contextmanager
def db_transaction(conn):
    cur = conn.cursor()
    try:
        yield cur
        conn.commit()
    except Exception:
        conn.rollback()
        raise
    finally:
        cur.close()

with suppress(FileNotFoundError):
    os.remove("tmp.txt")
```

5.9 Exceptions

UNDERSTAND THE CONCEPT

An **exception** is Python's way of signalling that something went wrong at run time (a file not found, a bad conversion, a missing key). If nothing handles it, the program stops and prints a **traceback** (the chain of calls that led to the error). Read tracebacks from the **bottom**: the last line says what went wrong.

- `try:` contains code that might fail.
- `except SomeError as e:` handles that specific error type. Catch *specific* exceptions, never a bare `except:`, which would also hide bugs.
- `else:` runs only if no exception happened.
- `finally:` always runs, for cleanup.
- `raise` throws an exception yourself. `raise NewError(...) from e` links a new error to its original cause.

Common built-ins: `ValueError` (right type, bad value), `TypeError` (wrong type), `KeyError` (missing dict key), `IndexError` (list position out of range), `AttributeError`, `FileNotFoundError`, `ZeroDivisionError`.

Create **custom exceptions** by subclassing `Exception`, which gives your app meaningful error types (`NotFound`, `PaymentFailed`).

EAFP vs LBYL: Python style prefers "**E**asier to **A**sk **F**orgiveness than **P**ermission" (try it and catch the error) over "**L**ook **B**efore **Y**ou **L**eap" (check every condition first).

```python
class AppError(Exception):
    """Base for app errors."""

class NotFound(AppError):
    def __init__(self, resource, id_):
        super().__init__(f"{resource} {id_} not found")
        self.resource, self.id = resource, id_

try:
    value = int(user_input)
except (ValueError, TypeError) as e:
    logger.warning("bad input: %s", e)
    raise AppError("invalid number") from e      # exception chaining
else:
    print("runs only if no exception")
finally:
    print("always runs")

# Exception groups (3.11+)
try:
    raise ExceptionGroup("many", [ValueError(1), TypeError(2)])
except* ValueError as eg:
    print("value errors", eg.exceptions)
```

EAFP ("easier to ask forgiveness than permission") is idiomatic: try the operation and catch the exception rather than checking first.

5.10 Type Hints

UNDERSTAND THE CONCEPT

Python is dynamically typed, but since Python 3.5 you can add **type hints**: optional annotations describing what types variables, parameters and return values should have:

```
def total(prices: list[float], discount: float = 0.0) -> float:
    ...
```

Key point: **Python itself ignores type hints at run time.** They are checked by separate tools (**mypy**, **pyright**, your editor) before running, much like TypeScript checks JavaScript. Benefits are the same as TypeScript: catching bugs early, better autocomplete, and self-documenting code.

Useful hint tools: `X | None` (may be None), `Literal["a", "b"]` (exact values), `TypedDict` (a dict with fixed keys), `Protocol` (structural typing: "any object that has these methods"), and generics with `TypeVar` or the new `def f[T](...)` syntax. Libraries like **Pydantic** *do* use type hints at run time to validate data. FastAPI and Django Ninja are built on that.

```
from typing import TypedDict, Protocol, Literal, Callable, TypeVar, Generic, Optional, Any,
Final, overload
from collections.abc import Iterable, Iterator, Sequence, Mapping

def total(prices: list[float], discount: float | None = None) -> float: ...

Status = Literal["active", "inactive"]

class UserDict(TypedDict, total=False):
    id: int
    name: str

class SupportsClose(Protocol):          # structural typing (duck typing for type checkers)
    def close(self) -> None: ...

T = TypeVar("T")
def first(items: Sequence[T]) -> T | None:
    return items[0] if items else None

# 3.12 generic syntax
def last[T](items: list[T]) -> T: return items[-1]
class Box[T]:
    def __init__(self, item: T): self.item = item
type Pair[T] = tuple[T, T]

Handler = Callable[[str, int], bool]
MAX_RETRIES: Final = 3
```

Check with `mypy` or `pyright`. Type hints are not enforced at runtime (use Pydantic for that).

5.11 Concurrency: Threads, Processes, Asyncio

UNDERSTAND THE CONCEPT

Concurrency means making progress on several tasks during overlapping time periods. **Parallelism** means literally running them at the same instant on multiple CPU cores.

The GIL (Global Interpreter Lock): in standard CPython, only **one thread can execute Python code at a time**. This has big consequences:

- For **I/O-bound** work (waiting for network, disk, database), threads still help a lot. While one thread waits, the GIL is released and another runs. Use `threading` / `ThreadPoolExecutor`, or `asyncio`.
- For **CPU-bound** work (heavy calculations), threads do *not* speed things up. Use **multiple processes** (`multiprocessing` / `ProcessPoolExecutor`). Each process has its own interpreter and its own GIL, so they truly run in parallel.

`asyncio` is Python's version of JavaScript's event loop and `async/await`. One thread runs many **coroutines** (functions defined with `async def`). Whenever a coroutine hits `await` on something slow, it pauses and lets another coroutine run. It is ideal for handling thousands of network connections efficiently (FastAPI, async Django). Rule: never call **blocking** functions (`time.sleep`, `requests.get`) inside async code, because they freeze the whole loop. Use async libraries (`httpx`, `aiohttp`) or `asyncio.to_thread`.

Race condition: when threads share data and the result depends on unlucky timing. Protect shared data with a **Lock**.

GIL (Global Interpreter Lock): in standard CPython only one thread executes Python bytecode at a time. Consequences:

- **I/O-bound** → threads or `asyncio` work well (GIL released during I/O).
- **CPU-bound** → use `multiprocessing` / `ProcessPoolExecutor` (separate interpreters), or C extensions (NumPy). Python 3.13+ offers an experimental **free-threaded** (no-GIL) build.

```python
from concurrent.futures import ThreadPoolExecutor, ProcessPoolExecutor, as_completed
import requests

urls = [...]
with ThreadPoolExecutor(max_workers=10) as pool:              # I/O-bound
    futures = {pool.submit(requests.get, u, timeout=5): u for u in urls}
    for fut in as_completed(futures):
        print(futures[fut], fut.result().status_code)

def crunch(n): return sum(i * i for i in range(n))
if __name__ == "__main__":
    with ProcessPoolExecutor() as pool:                        # CPU-bound
        print(list(pool.map(crunch, [10**6] * 8)))

# Thread safety
import threading
lock = threading.Lock()
counter = 0
def incr():
    global counter
    with lock:
        counter += 1
```

asyncio

```
import asyncio
import aiohttp

async def fetch(session, url):
    async with session.get(url) as resp:
        return await resp.json()

async def main():
    async with aiohttp.ClientSession() as session:
        # run concurrently
        results = await asyncio.gather(*(fetch(session, u) for u in urls),
return_exceptions=True)

        # limit concurrency
        sem = asyncio.Semaphore(5)
        async def bounded(u):
            async with sem:
                return await fetch(session, u)

        # structured concurrency (3.11+)
        async with asyncio.TaskGroup() as tg:
            t1 = tg.create_task(bounded(urls[0]))
            t2 = tg.create_task(bounded(urls[1]))

        # timeout
        async with asyncio.timeout(3):
            await fetch(session, urls[2])

        # run blocking code without blocking the loop
        data = await asyncio.to_thread(blocking_io_function)

asyncio.run(main())
```

Never call blocking functions (`time.sleep` , `requests.get`) inside `async def` — use `await asyncio.sleep` , `httpx.AsyncClient` , etc.

5.12 Modules, Packages and Environments

UNDERSTAND THE CONCEPT

- A **module** is simply a `.py` file. You use its contents in another file with `import` .
- A **package** is a folder of modules, traditionally marked by an `__init__.py` file. Packages let you organise large projects (`myapp/services/payment.py`).
- `if __name__ == "__main__":` runs code only when the file is executed directly, not when it's imported by another file.

Virtual environments (`venv`) give each project its **own isolated set of installed packages**, so project A can use Django 4 while project B uses Django 5 on the same computer. Always create one per project.

pip installs packages from PyPI (the Python Package Index). Dependencies are listed in `requirements.txt` or, in modern projects, `pyproject.toml` . Tools like **Poetry** and **uv** manage environments and lock exact versions.

```python
# package/
#   __init__.py
#   utils.py
from package.utils import helper       # absolute import
from .utils import helper              # relative import (inside package)

if __name__ == "__main__":             # runs only when executed directly
    main()
```

- Virtual envs: `python -m venv .venv && source .venv/bin/activate`; tools: `pip`, `pip-tools`, `poetry`, `uv`.
- `pyproject.toml` is the modern project config (dependencies, build, ruff/black/mypy config).
- Formatting & linting: `ruff`, `black`, `isort`; testing: `pytest`.

5.13 Memory Model and Internals

UNDERSTAND THE CONCEPT

How Python manages memory, in simple terms:

- **Reference counting:** every object keeps a count of how many names or containers refer to it. When the count drops to zero, the memory is freed immediately.
- **Garbage collector:** reference counting can't free **cycles** (A refers to B and B refers to A, but nothing else refers to either). Python's cyclic garbage collector periodically finds and frees these.

Shallow vs deep copy (the same idea as JavaScript, see 1.3): `copy.copy()` creates a new outer container but shares the inner objects. `copy.deepcopy()` copies everything recursively.

Descriptors are an advanced mechanism: objects stored on a class that control what happens when an attribute is read or set on its instances. `@property`, methods and **Django model fields** all work through descriptors.

- **Reference counting** frees objects immediately when the count hits zero; a **cyclic garbage collector** (`gc` module) handles reference cycles.
- `sys.getrefcount(obj)`, `sys.getsizeof(obj)`, `tracemalloc` for leak hunting.
- **Shallow vs deep copy:**

```python
import copy
a = [[1, 2], [3]]
b = copy.copy(a)        # new outer list, same inner lists
c = copy.deepcopy(a)    # fully independent
b[0].append(99)         # affects a, not c
```

- **Interning:** short strings/identifiers may be interned; `sys.intern()`.
- **Descriptors:** objects with `__get__`/`__set__`/`__delete__`; power `property`, methods, `classmethod`, and Django model fields.

```python
class Positive:
    def __set_name__(self, owner, name): self.name = "_" + name
    def __get__(self, obj, objtype=None): return getattr(obj, self.name)
    def __set__(self, obj, value):
```

```
        if value <= 0: raise ValueError(f"{self.name[1:]} must be > 0")
        setattr(obj, self.name, value)

class Order:
    qty = Positive()
    def __init__(self, qty): self.qty = qty
```

5.14 Metaclasses (Advanced)

UNDERSTAND THE CONCEPT

In Python, **classes are objects too**, and the thing that creates a class is called a **metaclass**. The default metaclass is `type`. A custom metaclass can automatically modify or register classes **at the moment they are defined**.

You will rarely write one yourself, but it explains "magic" in frameworks. When you write a Django model with `name = models.CharField()`, Django's metaclass reads those field declarations and builds the database mapping, the `objects` manager and more. For most needs, the simpler **`__init_subclass__`** hook or a class decorator is enough.

A metaclass is the "class of a class" — `type` by default. Used by Django ORM (`ModelBase`) to turn field declarations into a model.

```
class RegistryMeta(type):
    registry = {}
    def __new__(mcls, name, bases, ns):
        cls = super().__new__(mcls, name, bases, ns)
        if bases:
            mcls.registry[name.lower()] = cls
        return cls

class Plugin(metaclass=RegistryMeta): ...
class CsvExporter(Plugin): ...
RegistryMeta.registry   # {'csvexporter': <class CsvExporter>}

# Often simpler: __init_subclass__
class Base:
    plugins = []
    def __init_subclass__(cls, **kw):
        super().__init_subclass__(**kw)
        Base.plugins.append(cls)
```

5.15 Testing with pytest

UNDERSTAND THE CONCEPT

pytest is the most popular Python testing framework. A test is simply a function whose name starts with `test_`, using plain `assert` statements. Pytest finds and runs them automatically and shows clear failure messages.

Key concepts:

- **Fixtures** (`@pytest.fixture`) are reusable setup code (a test user, a database, an API client), injected into a test just by naming them as parameters.
- **Parametrize** runs the same test with many different inputs.
- **`pytest.raises`** checks that code raises the expected exception.
- **Mocking** (`unittest.mock` , `pytest-mock`) replaces real dependencies (a network call, sending an email) with fakes you control, so tests are fast and reliable and you can check how they were called.

```python
# test_cart.py
import pytest
from cart import Cart

@pytest.fixture
def cart():
    return Cart()

def test_add_item(cart):
    cart.add("apple", 2)
    assert cart.total_items == 2

@pytest.mark.parametrize("qty, expected", [(1, 10), (3, 30), (0, 0)])
def test_price(cart, qty, expected):
    cart.add("x", qty, price=10)
    assert cart.total == expected

def test_negative_qty_raises(cart):
    with pytest.raises(ValueError, match="positive"):
        cart.add("x", -1)

def test_external_api(mocker):                    # pytest-mock
    mock_get = mocker.patch("cart.requests.get")
    mock_get.return_value.json.return_value = {"rate": 1.1}
    assert Cart().convert(10) == 11

# unittest.mock directly
from unittest.mock import patch, MagicMock
with patch("module.send_email") as fake:
    do_signup()
    fake.assert_called_once_with("a@b.com")
```

5.16 Common Interview Snippets

UNDERSTAND THE CONCEPT

These are common interview snippets and "predict the output" puzzles. For each puzzle, explain *why* it behaves that way using the concepts above: references, mutability, late binding of closures, and floating-point precision.

```python
# Reverse words
" ".join(reversed("hello big world".split()))

# Anagram check
from collections import Counter
Counter("listen") == Counter("silent")

# Two sum — O(n)
def two_sum(nums, target):
```

```
    seen = {}
    for i, n in enumerate(nums):
        if target - n in seen:
            return seen[target - n], i
        seen[n] = i

# Flatten nested dict
def flatten(d, parent=""):
    out = {}
    for k, v in d.items():
        key = f"{parent}.{k}" if parent else k
        out.update(flatten(v, key) if isinstance(v, dict) else {key: v})
    return out

# LRU cache implementation
from collections import OrderedDict
class LRU:
    def __init__(self, cap): self.cap, self.d = cap, OrderedDict()
    def get(self, k):
        if k not in self.d: return -1
        self.d.move_to_end(k); return self.d[k]
    def put(self, k, v):
        self.d[k] = v; self.d.move_to_end(k)
        if len(self.d) > self.cap: self.d.popitem(last=False)

# Singleton
class Singleton:
    _instance = None
    def __new__(cls, *a, **kw):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance
```

Tricky outputs:

```
print([[0] * 2] * 2)        # [[0,0],[0,0]] but rows share the same list!
x = [[0] * 2] * 2; x[0][0] = 1; print(x)   # [[1,0],[1,0]] — use [[0]*2 for _ in range(2)]
print(0.1 + 0.2 == 0.3)     # False → math.isclose
print(bool([]), bool([0]))  # False True
print(round(2.5), round(3.5))  # 2 4 — banker's rounding
fs = [lambda: i for i in range(3)]; print([f() for f in fs])  # [2,2,2] — late binding
fs = [lambda i=i: i for i in range(3)]                        # [0,1,2]
```

Part 6 — Django & Django REST Framework

UNDERSTAND THE CONCEPT

What is Django? Django is a **web framework** for Python, first released in 2005. A framework gives you a ready-made structure and tools for the common jobs every web application needs, so you focus on *your* features instead of reinventing the basics. Django's motto is "*the web framework for perfectionists with deadlines*", and it follows a "**batteries included**" philosophy. Out of the box it provides:

- an **ORM** to work with databases using Python classes instead of SQL;
- **migrations** to evolve the database structure safely;
- **URL routing**, **views** and **templates**;
- **authentication** (users, passwords, permissions, sessions);
- an automatic **admin panel** to manage your data;
- **forms** and validation;
- **security protections** (CSRF, XSS, SQL injection, clickjacking) enabled by default.

Django REST Framework (DRF) is a library on top of Django for building **REST APIs** that return JSON. It's what you use when the frontend is React or a mobile app.

Project vs app: a Django **project** is the whole website (settings, root URLs). It is made of several **apps**, each a self-contained feature module (`users` , `orders` , `payments`) with its own models, views and URLs. Apps keep large codebases organised and can be reused.

6.1 Architecture: MTV and the Request Lifecycle

UNDERSTAND THE CONCEPT

MTV (Model–Template–View) is how Django divides responsibilities:

- **Model:** your **data** and its rules. It is a Python class describing a database table.
- **Template:** the **presentation**, i.e. HTML with placeholders that get filled with data.
- **View:** the **logic** that receives a request, gets or changes data through models, and returns a response (a rendered template or JSON).

(This is the same idea as MVC: Django's "view" ≈ MVC's "controller", and Django's "template" ≈ MVC's "view".)

What happens when a request arrives:

1. A web server (Gunicorn for **WSGI**, the traditional synchronous interface, or Uvicorn for **ASGI**, the newer interface that supports async) passes the request to Django.

2. Django wraps it in an `HttpRequest` object and passes it through each **middleware** in order (security, sessions, authentication...).
3. The **URL dispatcher** compares the path with your `urls.py` patterns and picks the matching view.
4. The **view** runs your code and returns an `HttpResponse`.
5. The response passes back through the middleware in **reverse** order and is sent to the browser.

`manage.py` is the command-line tool for your project: start the dev server, create migrations, open a shell, create admin users, run tests.

Django follows **MTV** — **M**odel (data), **T**emplate (presentation), **V**iew (business logic; the "controller" role is played by the framework + URLconf).

Request lifecycle:

1. WSGI/ASGI server (Gunicorn / Uvicorn) receives the request → `WSGIHandler` / `ASGIHandler`.
2. **Middleware** (request phase, top → bottom of `MIDDLEWARE`).
3. **URL resolver** matches `urls.py` patterns → view.
4. **View** runs (talks to models/ORM, services, cache).
5. Returns `HttpResponse` (template render or JSON).
6. **Middleware** (response phase, bottom → top).
7. Response sent to client.

```
myproject/
  manage.py
  config/            # settings.py, urls.py, wsgi.py, asgi.py
  apps/
    orders/
      models.py  views.py  serializers.py  urls.py  admin.py
      services.py  selectors.py  signals.py  tasks.py  tests/
      migrations/
```

```
django-admin startproject config .
python manage.py startapp orders
python manage.py makemigrations && python manage.py migrate
python manage.py createsuperuser
python manage.py runserver
python manage.py shell_plus --print-sql   # django-extensions
```

6.2 Settings Essentials

UNDERSTAND THE CONCEPT

`settings.py` is the **central configuration** of a Django project: which apps are installed, which database to use, middleware, templates, static files, security settings, time zone, and so on.

Important settings explained:

- **`SECRET_KEY` :** used to sign sessions, password-reset tokens and CSRF tokens. It must be secret and never committed to Git.

- **`DEBUG`:** shows detailed error pages. It must be `False` in production, because the debug pages leak sensitive information.
- **`ALLOWED_HOSTS`:** the domain names your site may be served from, which protects against host-header attacks.
- **`INSTALLED_APPS`:** every app (yours and third-party) that Django should load.
- **`DATABASES`:** connection details.
- **`MIDDLEWARE`:** the ordered list of middleware.
- **`AUTH_USER_MODEL`:** your custom user model. Set it **at the very start** of a project, because changing it later is painful.

Good practice: read secrets and environment-specific values from **environment variables**, and keep separate settings for development and production.

```python
# settings.py
import os
from pathlib import Path
BASE_DIR = Path(__file__).resolve().parent.parent

SECRET_KEY = os.environ["DJANGO_SECRET_KEY"]
DEBUG = os.getenv("DEBUG") == "1"
ALLOWED_HOSTS = os.getenv("ALLOWED_HOSTS", "").split(",")

INSTALLED_APPS = [
    "django.contrib.admin", "django.contrib.auth", "django.contrib.contenttypes",
    "django.contrib.sessions", "django.contrib.messages", "django.contrib.staticfiles",
    "rest_framework", "django_filters", "corsheaders",
    "apps.orders",
]

DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.postgresql",
        "NAME": os.environ["DB_NAME"], "USER": os.environ["DB_USER"],
        "PASSWORD": os.environ["DB_PASSWORD"], "HOST": os.environ["DB_HOST"], "PORT": 5432,
        "CONN_MAX_AGE": 60,          # persistent connections
        "ATOMIC_REQUESTS": False,
    }
}

AUTH_USER_MODEL = "users.User"      # set a custom user model at project start!
```

Split settings per environment (`base.py`, `dev.py`, `prod.py`) or use `django-environ`.

6.3 Models

UNDERSTAND THE CONCEPT

A **model** is a Python class that represents a **database table**. Each **attribute** (field) is a **column**, and each **instance** of the class is a **row**. You define your data once in Python, and Django creates the table and generates all the SQL for you.

```python
class Book(models.Model):                       # → table "app_book"
    title = models.CharField(max_length=200)    # → column VARCHAR(200)
    published = models.DateField()              # → column DATE
    price = models.DecimalField(max_digits=6, decimal_places=2)
# Django automatically adds:  id = primary key, auto-incrementing
```

Field types match data types: `CharField` (short text), `TextField` (long text), `IntegerField`, `DecimalField` (money), `BooleanField`, `DateTimeField`, `JSONField`, `EmailField`, and so on.
Field options add rules: `unique=True`, `default=...`, `null=True` (database allows NULL), `blank=True` (forms allow empty), `choices=...`, `db_index=True`.

Relationships between tables (the core of relational databases, see 7.16):

- `ForeignKey` is a **many-to-one** relationship. Many orders belong to one user. It stores the user's id in the order table. `on_delete` decides what happens to the orders when the user is deleted (delete them too, block the deletion, or set the field to NULL).
- `ManyToManyField` is **many-to-many**. A product has many tags and a tag has many products. Django creates a hidden linking ("junction") table. Use `through=` to define it yourself when you need extra columns (e.g. quantity in an order item).
- `OneToOneField` is **one-to-one**. Each user has exactly one profile.

`related_name` sets the name used to go backwards: `user.orders.all()`.

`Meta` **class options:** default ordering, database indexes, constraints (unique combinations, check rules), and the table name.

Managers and QuerySets: `Product.objects` is the **manager**, the entry point for database queries. You can add custom query methods (`Product.objects.active()`) so filtering rules live in one place.

```
from django.db import models
from django.conf import settings
from django.core.validators import MinValueValidator

class TimeStamped(models.Model):
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        abstract = True                         # no table; fields inherited


class Category(TimeStamped):
    name = models.CharField(max_length=100, unique=True)
    slug = models.SlugField(unique=True)

    def __str__(self):
        return self.name


class ProductQuerySet(models.QuerySet):
    def active(self):
        return self.filter(is_active=True)
    def in_stock(self):
        return self.filter(stock__gt=0)


class Product(TimeStamped):
    class Status(models.TextChoices):
        DRAFT = "draft", "Draft"
        PUBLISHED = "published", "Published"

    category = models.ForeignKey(Category, on_delete=models.PROTECT, related_name="products")
    name = models.CharField(max_length=200, db_index=True)
    price = models.DecimalField(max_digits=10, decimal_places=2, validators=
[MinValueValidator(0)])
```

```
    stock = models.PositiveIntegerField(default=0)
    status = models.CharField(max_length=20, choices=Status.choices, default=Status.DRAFT)
    is_active = models.BooleanField(default=True)
    tags = models.ManyToManyField("Tag", blank=True, related_name="products")
    metadata = models.JSONField(default=dict, blank=True)

    objects = ProductQuerySet.as_manager()      # Product.objects.active().in_stock()

    class Meta:
        ordering = ["-created_at"]
        indexes = [models.Index(fields=["category", "status"])]
        constraints = [
            models.CheckConstraint(condition=models.Q(price__gte=0), name="price_non_negative"),
            models.UniqueConstraint(fields=["category", "name"],
name="uniq_product_per_category"),
        ]

    def __str__(self):
        return self.name


class Tag(models.Model):
    name = models.CharField(max_length=50, unique=True)


class Order(TimeStamped):
    user = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE,
related_name="orders")
    products = models.ManyToManyField(Product, through="OrderItem")   # M2M with extra data
    total = models.DecimalField(max_digits=12, decimal_places=2, default=0)


class OrderItem(models.Model):
    order = models.ForeignKey(Order, on_delete=models.CASCADE, related_name="items")
    product = models.ForeignKey(Product, on_delete=models.PROTECT)
    quantity = models.PositiveIntegerField()
    unit_price = models.DecimalField(max_digits=10, decimal_places=2)


class Profile(models.Model):
    user = models.OneToOneField(settings.AUTH_USER_MODEL, on_delete=models.CASCADE)
    bio = models.TextField(blank=True)
```

`on_delete` options: `CASCADE` (delete children), `PROTECT` (block deletion), `RESTRICT`, `SET_NULL` (needs `null=True`), `SET_DEFAULT`, `SET(...)`, `DO_NOTHING`.

`null` vs `blank`: `null=True` → DB column allows NULL; `blank=True` → form/validation allows empty. For strings prefer `blank=True` with `""` instead of `null=True`.

Model inheritance

Type	Tables	Use
Abstract base (`abstract = True`)	Only children	Shared fields (timestamps)
Multi-table	Parent + child joined 1:1	Real "is-a" with queries on parent
Proxy (`proxy = True`)	Same table	Different Python behaviour / manager / admin

6.4 ORM — QuerySets

UNDERSTAND THE CONCEPT

The **ORM (Object-Relational Mapper)** lets you query the database with Python instead of SQL. Django translates your Python calls into SQL, runs them, and turns the resulting rows back into Python objects.

A **QuerySet** represents a database query. Key properties:

- **Lazy:** building a QuerySet (`Product.objects.filter(price__gt=100)`) does **not** touch the database. The SQL runs only when you actually need the data: when you loop over it, call `list()` , `len()` , print it, and so on. You can keep chaining `.filter().exclude().order_by()` for free.
- **Chainable:** each method returns a new QuerySet, so you build queries step by step.
- **Cached:** once evaluated, a QuerySet keeps its results, so looping over it twice doesn't hit the database twice.

Lookups use double underscores: `price__gte=100` (≥), `name__icontains="pro"` (case-insensitive contains), `category__name="Phones"` (follow a relationship, which creates a JOIN), `created_at__year=2026` .

- **`Q` objects** build complex conditions with OR (`|`) and NOT (`~`).
- **`F` expressions** refer to a column's value *inside the database*. For example, `stock=F("stock") - 1` is done by the database atomically, avoiding race conditions where two requests read the same old value.
- **`aggregate()`** computes one summary over the whole QuerySet (total, average). **`annotate()`** adds a computed value to **each** row (e.g. the number of products per category).

The N+1 query problem is the most common ORM performance bug. You fetch a list of N orders (1 query), then access `order.user` for each one, and Django runs **one more query per order** (N queries). With 100 orders that's 101 queries instead of 1 or 2. The fixes:

- **`select_related("user")`** for ForeignKey/OneToOne: Django fetches the related row in the **same query using a SQL JOIN**.
- **`prefetch_related("items")`** for many-to-many and reverse relations: Django runs **one extra query** for all related objects and matches them up in Python.

QuerySets are **lazy** — no SQL runs until evaluated (iteration, `list()` , slicing with step, `len()` , `bool()` , `repr()`). They are also **cached** after evaluation.

```
from django.db.models import Q, F, Count, Sum, Avg, Max, Value, Case, When, Subquery, OuterRef,
Exists
from django.db.models.functions import Coalesce, Lower, TruncMonth

Product.objects.all()
Product.objects.filter(price__gte=100, category__slug="phones")   # JOIN via __
Product.objects.exclude(status="draft")
Product.objects.get(pk=1)                    # DoesNotExist / MultipleObjectsReturned
Product.objects.filter(name__icontains="pro").order_by("-price")[:10]  # LIMIT 10
Product.objects.values("id", "name")         # dicts
```

```python
Product.objects.values_list("id", flat=True) # [1, 2, ...]
Product.objects.only("id", "name")           # defer other columns
Product.objects.exists(); Product.objects.count()
obj, created = Product.objects.get_or_create(name="X", defaults={"price": 1, "category": c})
Product.objects.update_or_create(name="X", defaults={"price": 2})

# Field lookups
# exact, iexact, contains, icontains, in, gt, gte, lt, lte, startswith, range,
# isnull, date, year, month, regex

# Q objects: OR / NOT
Product.objects.filter(Q(price__lt=10) | Q(stock=0), ~Q(status="draft"))

# F expressions: reference columns, atomic updates (avoid race conditions)
Product.objects.filter(stock__lt=F("reorder_level"))
Product.objects.filter(pk=1).update(stock=F("stock") - 1)

# Aggregation (whole table) vs annotation (per row)
Product.objects.aggregate(avg=Avg("price"), total=Sum("stock"))
Category.objects.annotate(n=Count("products")).filter(n__gt=5).order_by("-n")

# Group by month
(Order.objects.annotate(month=TruncMonth("created_at"))
     .values("month").annotate(revenue=Sum("total")).order_by("month"))

# Conditional expressions
Product.objects.annotate(
    tier=Case(When(price__gte=1000, then=Value("premium")), default=Value("standard"))
)

# Subqueries
latest = Order.objects.filter(user=OuterRef("pk")).order_by("-created_at")
User.objects.annotate(
    last_order_total=Subquery(latest.values("total")[:1]),
    has_orders=Exists(Order.objects.filter(user=OuterRef("pk"))),
)

# Bulk operations — far fewer queries
Product.objects.bulk_create([Product(name=f"p{i}", price=1, category=c) for i in range(1000)],
batch_size=500)
Product.objects.bulk_update(products, ["price"], batch_size=500)
Product.objects.filter(status="draft").delete()

# Raw SQL when needed (still parameterized)
Product.objects.raw("SELECT * FROM shop_product WHERE price > %s", [100])
from django.db import connection
with connection.cursor() as cur:
    cur.execute("SELECT count(*) FROM shop_product WHERE stock = %s", [0])
    cur.fetchone()

print(qs.query)            # see SQL
qs.explain(analyze=True)   # EXPLAIN on Postgres
```

The N+1 Problem: `select_related` vs `prefetch_related`

```python
# N+1: 1 query for orders + 1 query per order for user
for o in Order.objects.all():
    print(o.user.email)

# select_related → SQL JOIN — for ForeignKey / OneToOne (forward)
for o in Order.objects.select_related("user"):
    print(o.user.email)                # 1 query total

# prefetch_related → separate query + Python join — for M2M & reverse FK
for o in Order.objects.prefetch_related("items__product"):
    for item in o.items.all():         # 3 queries total, regardless of N
        print(item.product.name)

# Prefetch object for filtered/optimized prefetches
```

```python
from django.db.models import Prefetch
Order.objects.prefetch_related(
    Prefetch("items",
queryset=OrderItem.objects.select_related("product").filter(quantity__gt=1),
             to_attr="big_items")
)
```

Detect N+1 with `django-debug-toolbar`, `nplusone`, or `assertNumQueries` in tests.

6.5 Transactions and Concurrency

UNDERSTAND THE CONCEPT

A **transaction** groups several database operations into one **all-or-nothing** unit (see ACID in 7.15). If anything fails midway, everything is **rolled back** as if nothing happened. Placing an order, for example, creates the order, adds the items and decreases stock. You never want only half of that done.

- **`transaction.atomic`** (decorator or `with` block) makes the enclosed code one transaction. An exception inside means a rollback. Nested `atomic` blocks create **savepoints**, so an inner block can fail without undoing the outer one.
- **`transaction.on_commit(fn)`** runs `fn` only **after** the transaction has successfully committed. It is perfect for sending emails or starting background tasks, which should never happen if the data was rolled back.

Concurrency problems: two users buy the last item at the same moment. Both read `stock = 1`, both sell, and the stock becomes -1. Solutions:

- **`select_for_update()`** **locks** the selected rows until the transaction ends, so the second request waits until the first has finished.
- **`F()` expressions** do the arithmetic inside the database in a single atomic statement.
- **Database constraints** (e.g. `stock >= 0`) act as a final safety net.

```python
from django.db import transaction

@transaction.atomic
def place_order(user, cart):
    # SELECT ... FOR UPDATE — lock rows until the transaction ends
    products = Product.objects.select_for_update().filter(id__in=cart.keys())
    order = Order.objects.create(user=user)
    for p in products:
        qty = cart[p.id]
        if p.stock < qty:
            raise ValueError(f"{p.name} out of stock")    # rolls back everything
        p.stock = F("stock") - qty
        p.save(update_fields=["stock"])
        OrderItem.objects.create(order=order, product=p, quantity=qty, unit_price=p.price)

    # Run side effects only after a successful commit
    transaction.on_commit(lambda: send_order_email.delay(order.id))
    return order

# Savepoints with nested atomic blocks
with transaction.atomic():
    create_user()
    try:
```

```
        with transaction.atomic():      # savepoint
            risky()
    except IntegrityError:
        pass                            # outer transaction still alive
```

Race-condition tools: `F()` expressions, `select_for_update(nowait=True / skip_locked=True)`, unique constraints + `IntegrityError` handling, optimistic locking with a `version` column.

6.6 Migrations

UNDERSTAND THE CONCEPT

Your models change over time: new fields, renamed tables, new relationships. **Migrations** are Django's version control for the database structure (**schema**).

1. You change `models.py`.
2. **`makemigrations`** compares your models with the previous migrations and writes a new migration file describing the changes ("add column `phone` to `user`").
3. **`migrate`** applies all unapplied migrations to the database. Django records which ones have run in a `django_migrations` table.

Migration files are committed to Git, so every developer and every server builds the exact same database structure. **Data migrations** (`RunPython`) transform existing data, e.g. filling a new column. In production, be careful with large tables: some operations lock the table, so big changes are done in small, backwards-compatible steps.

```
python manage.py makemigrations orders
python manage.py migrate
python manage.py showmigrations
python manage.py sqlmigrate orders 0003      # show SQL
python manage.py migrate orders 0002         # roll back
python manage.py squashmigrations orders 0001 0010
```

```
# Data migration
from django.db import migrations

def forwards(apps, schema_editor):
    Product = apps.get_model("shop", "Product")       # use historical model!
    Product.objects.filter(status="").update(status="draft")

class Migration(migrations.Migration):
    dependencies = [("shop", "0004_product_status")]
    operations = [migrations.RunPython(forwards, migrations.RunPython.noop)]
```

Zero-downtime tips: add nullable columns first, backfill in batches, then add NOT NULL; create indexes concurrently on Postgres (`AddIndexConcurrently`, `atomic = False`); never rename/drop columns used by running code in the same deploy.

6.7 Views

UNDERSTAND THE CONCEPT

A **view** is a Python callable that **takes a request and returns a response**. That's the whole contract. Django supports two styles:

- **Function-Based Views (FBV):** a plain function `def my_view(request): ...`. They are explicit and easy to read, and good for unique or simple logic.
- **Class-Based Views (CBV):** a class with methods per HTTP method (`get()` , `post()`). Django ships **generic CBVs** for common patterns (`ListView` , `DetailView` , `CreateView` , `UpdateView` , `DeleteView`) that do most of the work for you: you configure attributes like `model` and `template_name` and override small hooks like `get_queryset()` . Behaviour is combined through **mixins** (`LoginRequiredMixin`).

URLconf (`urls.py`) maps URL patterns to views. **Path converters** like `<int:pk>` capture parts of the URL and pass them as arguments to the view. Give each URL a **`name`** so you can build links with `reverse()` or `{% url %}` instead of hard-coding paths.

Function-Based Views

```python
from django.shortcuts import render, get_object_or_404, redirect
from django.http import JsonResponse
from django.contrib.auth.decorators import login_required
from django.views.decorators.http import require_http_methods

@login_required
@require_http_methods(["GET", "POST"])
def product_detail(request, pk):
    product = get_object_or_404(Product, pk=pk)
    if request.method == "POST":
        form = ProductForm(request.POST, instance=product)
        if form.is_valid():
            form.save()
            return redirect("product-detail", pk=pk)
    else:
        form = ProductForm(instance=product)
    return render(request, "shop/detail.html", {"product": product, "form": form})
```

Class-Based Views

```python
from django.views.generic import ListView, DetailView, CreateView, UpdateView, DeleteView
from django.contrib.auth.mixins import LoginRequiredMixin, PermissionRequiredMixin
from django.urls import reverse_lazy

class ProductListView(LoginRequiredMixin, ListView):
    model = Product
    paginate_by = 20
    template_name = "shop/list.html"
    context_object_name = "products"

    def get_queryset(self):
        qs = super().get_queryset().select_related("category")
        if q := self.request.GET.get("q"):
            qs = qs.filter(name__icontains=q)
        return qs

class ProductCreateView(PermissionRequiredMixin, CreateView):
    model = Product
    fields = ["name", "price", "category"]
    permission_required = "shop.add_product"
    success_url = reverse_lazy("product-list")

    def form_valid(self, form):
```

```
        form.instance.created_by = self.request.user
        return super().form_valid(form)
```

CBV flow: `as_view()` → `dispatch()` → `get()/post()` → `get_queryset()` / `get_context_data()` → `render_to_response()`.

URLs

```
from django.urls import path, include
urlpatterns = [
    path("products/", ProductListView.as_view(), name="product-list"),
    path("products/<int:pk>/", product_detail, name="product-detail"),
    path("blog/<slug:slug>/", post_detail),
    path("api/", include("apps.api.urls")),
]
# reverse("product-detail", kwargs={"pk": 1}); in templates {% url 'product-detail' pk=1 %}
```

6.8 Templates and Forms (Quick)

UNDERSTAND THE CONCEPT

Templates are HTML files with special syntax that Django fills with data:

- `{{ variable }}` outputs a value, and **filters** transform it: `{{ name|upper }}`, `{{ price|floatformat:2 }}`.
- `{% tag %}` adds logic: `{% if %}`, `{% for %}`, `{% url %}`, `{% csrf_token %}`.
- **Template inheritance:** a `base.html` defines the common layout with `{% block %}` placeholders, and child templates `{% extends %}` it and fill in the blocks. You write the header and footer once.
- **Auto-escaping:** Django converts characters like `<` into `<` automatically, so user content can't inject scripts (XSS protection).

Forms handle the full cycle of user input: render HTML inputs, receive submitted data, **validate** it (`is_valid()`), give clean Python values (`cleaned_data`), and show error messages. A **ModelForm** builds a form automatically from a model and can save it directly.

When React is your frontend, you mostly use DRF serializers instead of templates and forms, but the concepts are the same.

```
{% extends "base.html" %}
{% load static %}
{% block content %}
  {% for p in products %}
    <h3>{{ p.name|title }}</h3> <p>{{ p.price|floatformat:2 }}</p>
  {% empty %}
    <p>No products.</p>
  {% endfor %}
  <form method="post">{% csrf_token %}{{ form.as_p }}<button>Save</button></form>
{% endblock %}
```

```
from django import forms
class ProductForm(forms.ModelForm):
    class Meta:
```

```
        model = Product
        fields = ["name", "price", "category"]
    def clean_price(self):
        price = self.cleaned_data["price"]
        if price > 1_000_000:
            raise forms.ValidationError("Too expensive")
        return price
    def clean(self):                        # cross-field validation
        data = super().clean()
        return data
```

Templates auto-escape HTML (XSS protection); `|safe` disables it — use carefully.

6.9 Middleware

UNDERSTAND THE CONCEPT

Middleware is a layer of code that wraps **every** request and response, like a chain of checkpoints. Each middleware can inspect or modify the request on the way **in**, stop it early by returning a response itself (for example, blocking a banned IP), and inspect or modify the response on the way **out**.

```
Request  → SecurityMW → SessionMW → AuthMW → [ your view ] → AuthMW → SessionMW →
SecurityMW → Response
```

Built-in middleware provides security headers, sessions, CSRF protection, authentication (`request.user`) and more. Custom middleware is for cross-cutting concerns that apply to all requests: logging, timing, request IDs, multi-tenant detection. **Order matters:** for example, authentication needs the session, so `SessionMiddleware` must come before `AuthenticationMiddleware` .

```
import time, logging, uuid
logger = logging.getLogger(__name__)

class RequestTimingMiddleware:
    def __init__(self, get_response):          # once at startup
        self.get_response = get_response

    def __call__(self, request):               # every request
        request.request_id = str(uuid.uuid4())
        start = time.monotonic()
        response = self.get_response(request)  # → next middleware / view
        response["X-Request-ID"] = request.request_id
        logger.info("%s %s %.1fms", request.method, request.path, (time.monotonic() - start) *
1000)
        return response

    def process_exception(self, request, exception):
        logger.exception("unhandled error id=%s", request.request_id)
        return None                            # let Django handle it
```

Other hooks: `process_view` , `process_template_response` . Order in `MIDDLEWARE` matters (e.g., `SecurityMiddleware` first, `SessionMiddleware` before `AuthenticationMiddleware` , `CorsMiddleware` high up).

6.10 Signals

UNDERSTAND THE CONCEPT

Signals let one part of the code **notify** other parts that something happened, without them being directly connected (the Observer pattern again). For example, *after a User is saved for the first time, automatically create a Profile*. You write a **receiver** function and connect it to a signal like `post_save` for a specific model.

Built-in signals fire around model saves and deletes, many-to-many changes, requests, and logins.

Use them sparingly. Signals make the flow of code hard to follow ("where did this record come from?"), and some bulk operations (`bulk_create` , `queryset.update()`) **don't** trigger them. For your own business logic, calling a function explicitly from a service is usually clearer.

```python
from django.db.models.signals import post_save, pre_delete
from django.dispatch import receiver

@receiver(post_save, sender=settings.AUTH_USER_MODEL)
def create_profile(sender, instance, created, **kwargs):
    if created:
        Profile.objects.create(user=instance)
# Register in AppConfig.ready(): import apps.users.signals
```

Common signals: `pre_save` , `post_save` , `pre_delete` , `post_delete` , `m2m_changed` , `request_started` , `user_logged_in` . **Caution:** signals hide control flow, are not fired by `bulk_create` / `update()` / `QuerySet.delete()` (for pre/post_save) — prefer explicit service functions for business logic.

6.11 Authentication and Permissions

UNDERSTAND THE CONCEPT

Django ships a complete **authentication system**:

- A **User** model (username, password, email, `is_staff` , `is_superuser` , ...). Most projects define a **custom user model** (e.g. logging in with email) at the start.
- **Password hashing:** passwords are never stored in plain text. Django stores a salted hash (PBKDF2 by default, Argon2 optional) and handles checking it.
- **Sessions:** after `login()` , Django remembers the user via a session cookie, and `request.user` is available in every view. Anonymous visitors get an `AnonymousUser` .
- **Permissions and groups:** every model automatically gets add/change/delete/view permissions. Assign them to users or to **groups** (roles like "Editors") and check them with `user.has_perm(...)` .
- Tools to protect views: `@login_required` , `LoginRequiredMixin` and `PermissionRequiredMixin` .

```python
from django.contrib.auth.models import AbstractUser

class User(AbstractUser):
    email = models.EmailField(unique=True)
    USERNAME_FIELD = "email"
    REQUIRED_FIELDS = ["username"]

# Usage
from django.contrib.auth import authenticate, login, logout
user = authenticate(request, username=email, password=pw)
if user: login(request, user)

user.has_perm("shop.change_product")
user.groups.add(Group.objects.get(name="editors"))
```

Passwords are hashed with PBKDF2 by default (Argon2 supported). Sessions stored in DB/cache; CSRF middleware protects cookie-authenticated POSTs.

6.12 Django REST Framework (DRF)

UNDERSTAND THE CONCEPT

Django REST Framework (DRF) turns Django into an API server that speaks JSON, which is what React and mobile apps consume. Its main building blocks:

1. Serializers convert between complex Python objects (model instances) and simple data types that can become JSON, in **both directions**:

- **Serialization (output):** model instance → dict → JSON response.
- **Deserialization (input):** JSON request body → validated Python data → saved model.

They also **validate** input: field types, required fields, and custom rules (`validate_<field>` for one field, `validate()` for rules involving several fields). A `ModelSerializer` builds its fields automatically from a model, much like a ModelForm.

2. Views handle requests. DRF offers increasing levels of automation:

- `APIView` : you write `get()` , `post()` and so on yourself.
- **Generic views** (`ListCreateAPIView` , `RetrieveUpdateDestroyAPIView`) implement common patterns from a `queryset` and `serializer_class` .
- **ViewSets** (`ModelViewSet`) put list, create, retrieve, update and delete in **one class**. Combined with a **Router**, which generates all the URLs automatically, you get a full CRUD API in a few lines.

3. Authentication identifies who is calling (session, token, JWT via `simplejwt`). **Permissions** decide if they may proceed: `has_permission` is checked for every request to the view, and `has_object_permission` for a specific object (e.g. "only the owner can edit this post").

4. Throttling limits how many requests a client can make (rate limiting). **Pagination** splits large lists into pages. **Filtering, search and ordering** let clients ask for specific data via query parameters.

Serializers

```python
from rest_framework import serializers

class CategorySerializer(serializers.ModelSerializer):
    class Meta:
        model = Category
        fields = ["id", "name", "slug"]


class ProductSerializer(serializers.ModelSerializer):
    category = CategorySerializer(read_only=True)
    category_id = serializers.PrimaryKeyRelatedField(
        queryset=Category.objects.all(), source="category", write_only=True
    )
    discounted_price = serializers.SerializerMethodField()

    class Meta:
        model = Product
        fields = ["id", "name", "price", "stock", "category", "category_id", "discounted_price"]
        read_only_fields = ["stock"]

    def get_discounted_price(self, obj):
        return obj.price * 90 / 100

    def validate_price(self, value):                    # field-level
        if value <= 0:
            raise serializers.ValidationError("Price must be positive")
        return value

    def validate(self, attrs):                          # object-level
        if attrs.get("name", "").lower() == "test":
            raise serializers.ValidationError({"name": "Reserved name"})
        return attrs


class OrderCreateSerializer(serializers.Serializer):    # nested writes
    items = serializers.ListField(child=serializers.DictField(), min_length=1)

    def create(self, validated_data):
        return place_order(self.context["request"].user, validated_data["items"])
```

Flow: `serializer = S(data=request.data)` → `is_valid(raise_exception=True)` → `validated_data` → `save()` calls `create()` or `update()` → `serializer.data` for output.

Views, ViewSets, Routers

```python
from rest_framework import viewsets, generics, status, permissions, filters
from rest_framework.decorators import action, api_view, permission_classes
from rest_framework.response import Response
from django_filters.rest_framework import DjangoFilterBackend

@api_view(["GET"])
@permission_classes([permissions.AllowAny])
def health(request):
    return Response({"ok": True})


class ProductViewSet(viewsets.ModelViewSet):
    serializer_class = ProductSerializer
    permission_classes = [permissions.IsAuthenticatedOrReadOnly]
    filter_backends = [DjangoFilterBackend, filters.SearchFilter, filters.OrderingFilter]
    filterset_fields = ["category", "status"]
    search_fields = ["name", "category__name"]
    ordering_fields = ["price", "created_at"]
```

```
    def get_queryset(self):
        return Product.objects.active().select_related("category").prefetch_related("tags")

    def get_serializer_class(self):
        return ProductListSerializer if self.action == "list" else ProductSerializer

    def perform_create(self, serializer):
        serializer.save(created_by=self.request.user)

    @action(detail=True, methods=["post"], url_path="restock")
    def restock(self, request, pk=None):
        product = self.get_object()
        qty = int(request.data.get("quantity", 0))
        Product.objects.filter(pk=product.pk).update(stock=F("stock") + qty)
        return Response({"status": "restocked"}, status=status.HTTP_200_OK)


# urls.py
from rest_framework.routers import DefaultRouter
router = DefaultRouter()
router.register("products", ProductViewSet, basename="product")
urlpatterns = router.urls
# GET/POST /products/, GET/PUT/PATCH/DELETE /products/{id}/, POST /products/{id}/restock/
```

Hierarchy: `APIView` → `GenericAPIView` (+ mixins: `ListModelMixin`, `CreateModelMixin`, ...) → concrete generics (`ListCreateAPIView`, `RetrieveUpdateDestroyAPIView`) → `ViewSet` / `ModelViewSet`.

Authentication, Permissions, Throttling, Pagination

```
REST_FRAMEWORK = {
    "DEFAULT_AUTHENTICATION_CLASSES": [
        "rest_framework_simplejwt.authentication.JWTAuthentication",
        "rest_framework.authentication.SessionAuthentication",
    ],
    "DEFAULT_PERMISSION_CLASSES": ["rest_framework.permissions.IsAuthenticated"],
    "DEFAULT_PAGINATION_CLASS": "rest_framework.pagination.PageNumberPagination",
    "PAGE_SIZE": 20,
    "DEFAULT_THROTTLE_CLASSES": ["rest_framework.throttling.UserRateThrottle",
                                 "rest_framework.throttling.AnonRateThrottle"],
    "DEFAULT_THROTTLE_RATES": {"user": "1000/day", "anon": "100/day"},
    "EXCEPTION_HANDLER": "apps.core.exceptions.custom_exception_handler",
}

# Custom object-level permission
from rest_framework.permissions import BasePermission, SAFE_METHODS

class IsOwnerOrReadOnly(BasePermission):
    def has_permission(self, request, view):            # view-level
        return request.user.is_authenticated
    def has_object_permission(self, request, view, obj): # called by get_object()
        return request.method in SAFE_METHODS or obj.owner_id == request.user.id

# Custom exception handler
from rest_framework.views import exception_handler
def custom_exception_handler(exc, context):
    response = exception_handler(exc, context)
    if response is not None:
        response.data = {"error": {"status": response.status_code, "detail": response.data}}
    return response
```

Pagination options: `PageNumberPagination`, `LimitOffsetPagination`, `CursorPagination` (stable & efficient for large/real-time datasets).

6.13 Caching

UNDERSTAND THE CONCEPT

Caching in Django means storing the results of expensive work so it doesn't have to be redone on every request (see the cache-aside explanation in 4.11). Django provides one caching API with pluggable backends (Redis is the most common in production) at several levels:

- **Low-level cache API:** `cache.get` / `cache.set` / `cache.get_or_set` for any value you choose. This gives the most control.
- **Per-view cache:** `@cache_page(seconds)` caches a view's entire response.
- **Template fragment cache:** caches part of a template.

Always think about **invalidation**: when the underlying data changes, delete or refresh the cached copy, or keep the timeout short.

```
CACHES = {"default": {"BACKEND": "django.core.cache.backends.redis.RedisCache",
                      "LOCATION": "redis://127.0.0.1:6379/1"}}

from django.core.cache import cache
from django.views.decorators.cache import cache_page

def get_dashboard(user_id):
    key = f"dashboard:{user_id}"
    return cache.get_or_set(key, lambda: build_dashboard(user_id), timeout=300)

cache.delete("dashboard:1")

@cache_page(60 * 15)                # per-view cache
def product_list(request): ...
# Template fragment cache: {% load cache %}{% cache 500 sidebar %}...{% endcache %}
```

6.14 Celery (Background Tasks)

UNDERSTAND THE CONCEPT

Celery is a **distributed task queue** for Python. It runs slow or scheduled work **in the background**, outside the request/response cycle. It is the Python equivalent of BullMQ (4.13).

How it works:

1. Your view calls `send_email.delay(user_id)` . This puts a **message** describing the task on a **broker** (Redis or RabbitMQ) and returns immediately. The user doesn't wait.
2. Separate **worker** processes listen to the broker, pick up tasks and execute them.
3. Failed tasks can be **retried** automatically with increasing delays.
4. **Celery Beat** is a scheduler that adds tasks on a timetable (e.g. "every night at 2 am, generate reports").

Best practices: pass **IDs** rather than model objects (the data may change before the task runs), make tasks **idempotent** (safe to run twice), and start tasks inside `transaction.on_commit` so they never run for data that was rolled back.

```
# config/celery.py
import os
from celery import Celery
os.environ.setdefault("DJANGO_SETTINGS_MODULE", "config.settings")
app = Celery("config")
app.config_from_object("django.conf:settings", namespace="CELERY")
app.autodiscover_tasks()

# apps/orders/tasks.py
from celery import shared_task

@shared_task(bind=True, autoretry_for=(ConnectionError,), retry_backoff=True, max_retries=5)
def send_order_email(self, order_id):
    order = Order.objects.select_related("user").get(pk=order_id)   # pass IDs, not objects
    send_mail("Order placed", f"#{order.id}", None, [order.user.email])

# Periodic tasks (celery beat)
CELERY_BEAT_SCHEDULE = {
    "nightly-report": {"task": "apps.reports.tasks.nightly", "schedule": crontab(hour=2,
minute=0)},
}
```

```
celery -A config worker -l info --concurrency=4
celery -A config beat -l info
```

Best practices: idempotent tasks, pass primitive IDs, call `.delay()` inside `transaction.on_commit`, set time limits, use separate queues for slow/fast tasks.

6.15 Async Django

UNDERSTAND THE CONCEPT

Traditionally Django is **synchronous**: each request occupies a worker until it finishes, including all the time spent waiting for external APIs. Modern Django (3.1+) also supports **async views** (`async def`), running under **ASGI** servers like Uvicorn. While one request awaits a slow network call, the same worker can serve other requests (see asyncio in 5.11).

The ORM offers async versions of its methods (`aget` , `acreate` , `acount` , `async for`). Sync-only code can be called from async code with `sync_to_async` . For **WebSockets** and real-time features, Django uses the **Channels** package.

Async helps most when views spend their time *waiting* on other services. It does not make CPU-heavy code faster.

Django supports async views (3.1+), async ORM methods (`aget` , `acreate` , `afilter` iteration via `async for`) and ASGI deployment.

```
from django.http import JsonResponse
import httpx
```

```python
async def weather(request):
    async with httpx.AsyncClient() as client:
        r = await client.get("https://api.example.com/weather")
    product = await Product.objects.aget(pk=1)
    count = await Product.objects.filter(stock=0).acount()
    return JsonResponse({"weather": r.json(), "out_of_stock": count})

from asgiref.sync import sync_to_async
result = await sync_to_async(legacy_sync_function)()
```

Real-time features (WebSockets) use **Django Channels**.

6.16 Testing Django

UNDERSTAND THE CONCEPT

Django has strong built-in testing support:

- Each test runs inside a **database transaction that is rolled back** afterwards, so tests don't affect each other. A separate test database is created automatically.
- The **test client** (or DRF's `APIClient`) sends fake requests to your views without a real server, and lets you force-authenticate a user.
- With **pytest-django** you write tests as simple functions and mark those that need the database with `@pytest.mark.django_db`.
- **factory_boy** generates realistic test objects with minimal code.
- `assertNumQueries` or `django_assert_num_queries` **catch N+1 regressions** by asserting how many SQL queries a view runs.

```python
import pytest
from django.urls import reverse
from rest_framework.test import APIClient

@pytest.fixture
def api_client(django_user_model):
    user = django_user_model.objects.create_user(username="a", email="a@x.com", password="pw")
    client = APIClient()
    client.force_authenticate(user)
    return client

@pytest.mark.django_db
def test_create_product(api_client, category):
    resp = api_client.post(reverse("product-list"),
                           {"name": "Phone", "price": "999.00", "category_id": category.id},
                           format="json")
    assert resp.status_code == 201
    assert Product.objects.filter(name="Phone").exists()

@pytest.mark.django_db
def test_list_has_no_n_plus_one(api_client, django_assert_num_queries, products):
    with django_assert_num_queries(3):
        api_client.get(reverse("product-list"))
```

Tools: `pytest-django`, `factory_boy` (test data), `freezegun` (time), `responses` / `respx` (mock HTTP), `TestCase` (wraps each test in a transaction), `TransactionTestCase` (for on_commit/real transactions), `override_settings`.

6.17 Security Checklist

UNDERSTAND THE CONCEPT

Django protects you from many common attacks **by default**. Understanding *how* is a frequent interview topic:

- **SQL injection:** the ORM always sends values separately from the SQL text (parameterization). Raw SQL must also use parameters.
- **XSS (Cross-Site Scripting):** templates auto-escape output.
- **CSRF (Cross-Site Request Forgery):** forms must include a secret `{% csrf_token %}` that other websites can't know, so they can't forge requests on behalf of your logged-in users.
- **Clickjacking:** an `X-Frame-Options` header prevents your site from being embedded invisibly in another site.
- **Passwords:** salted, slow hashing plus strength validators.

Your job is not to disable these protections and to configure production correctly: `DEBUG=False`, secret key from the environment, HTTPS-only cookies, HSTS. `manage.py check --deploy` lists what's missing.

- `DEBUG = False`, strong `SECRET_KEY` from env, correct `ALLOWED_HOSTS`.
- `SECURE_SSL_REDIRECT`, `SESSION_COOKIE_SECURE`, `CSRF_COOKIE_SECURE`, `SECURE_HSTS_SECONDS`.
- Built-in protections: CSRF tokens, template auto-escaping (XSS), ORM parameterization (SQL injection), clickjacking (`X-Frame-Options`), password hashing & validators.
- Never use `extra()` / raw SQL with string formatting.
- Validate uploads; serve media from a separate domain/bucket.
- `python manage.py check --deploy`.

6.18 Performance and Deployment

UNDERSTAND THE CONCEPT

Most Django performance problems come from the **database**: too many queries (N+1), fetching more columns or rows than needed, or missing indexes. So the checklist is about doing less database work, caching results, and moving slow work to background tasks.

Deployment basics: Django's `runserver` is for development only. In production, an application server (**Gunicorn** for WSGI, **Uvicorn** for ASGI) runs several worker processes, usually behind **Nginx** (which handles HTTPS and serves static files) or a cloud load balancer. **Static files** (CSS, JS, images) are collected with `collectstatic` and served by Nginx, WhiteNoise or a CDN. **Media files** (user uploads) go to object storage such as S3.

- Fix N+1 (`select_related` / `prefetch_related`), use `only()/defer()`, `values()` for read-only data, `iterator(chunk_size=…)` for huge loops, `bulk_*` for writes, `exists()` instead of `count() > 0`.

- Add DB indexes for filter/order columns; analyse with `EXPLAIN`.
- Cache aggressively (Redis), use CDN for static files (`collectstatic`, WhiteNoise/S3).
- Run with Gunicorn (`gunicorn config.wsgi -w 4 -k gthread`) or Uvicorn for ASGI, behind Nginx.
- Connection pooling (`CONN_MAX_AGE`, PgBouncer, Django 5.1 native pool option for psycopg3).
- Offload slow work to Celery; monitor with Sentry, Prometheus, APM.

6.19 Service Layer Pattern (Clean Django)

UNDERSTAND THE CONCEPT

As projects grow, putting all logic in views or models becomes messy. Business rules get duplicated between the API, the admin, management commands and Celery tasks. A common solution is a **service layer**:

- **Services** (`services.py`) are functions that perform *actions*, such as creating an order or charging a card. They contain the business rules, validation and transactions.
- **Selectors** (`selectors.py`) are functions that *fetch* data, with the right filters and query optimisations.
- **Views** stay thin: parse input → call a service or selector → return the serialized output.

Each piece of business logic then has exactly one home, can be reused from anywhere, and is easy to unit-test.

```python
# services.py — writes / business logic
def product_create(*, name: str, price: Decimal, category: Category, user) -> Product:
    product = Product(name=name, price=price, category=category, created_by=user)
    product.full_clean()
    product.save()
    audit_log(user, "product_created", product.id)
    return product

# selectors.py — reads
def product_list(*, filters: dict | None = None) -> QuerySet[Product]:
    qs = Product.objects.active().select_related("category")
    return ProductFilter(filters or {}, qs).qs

# Views stay thin: validate input → call service/selector → serialize output
```

6.20 Interview Questions Checklist

UNDERSTAND THE CONCEPT

For each question below, try to explain the answer **out loud in your own words**, as you would to an interviewer, then check the section above that covers it.

- Explain the request/response lifecycle and middleware order.

- `select_related` vs `prefetch_related`; how to detect N+1.
- QuerySet laziness and caching; when is a query executed?
- `F()` and `Q()` objects; `annotate` vs `aggregate`.
- How `transaction.atomic` and `select_for_update` work; `on_commit`.
- `null` vs `blank`; `on_delete` options; custom user model.
- Signals: pros/cons; why they don't fire on `bulk_create` / `update`.
- CBV vs FBV; mixins and MRO.
- DRF: serializer validation flow, ViewSets vs APIView, permissions (`has_permission` vs `has_object_permission`), throttling, pagination.
- Celery: idempotency, retries, beat; why pass IDs.
- WSGI vs ASGI; async views.
- How Django protects against CSRF, XSS, SQL injection, clickjacking.

Part 7 — SQL: Basics to Advanced

UNDERSTAND THE CONCEPT

What is a database? A database is an organised store of data that lives permanently, unlike variables, which disappear when a program stops. It lets many users read and change the data safely at the same time.

What is a relational database? It stores data in **tables**, like spreadsheets:

- A **table** holds one kind of thing (`employees` , `orders`).
- Each **row** (record) is one item (one employee).
- Each **column** (field) is one attribute with a fixed data type (`name` , `salary`).
- A **primary key** uniquely identifies each row (usually `id`).
- A **foreign key** is a column that points to a row in another table (an employee's `department_id` refers to `departments.id`). These links are the **relations**.

```
departments                 employees
+----+-------------+        +----+--------+---------------+--------+
| id | name        |        | id | name   | department_id | salary |
+----+-------------+        +----+--------+---------------+--------+
|  1 | Engineering | <------+  1 | Anish  |       1       | 120000 |
|  2 | Sales       | <---+  |  2 | Ravi   |       1       |  90000 |
+----+-------------+     +--+  3 | Priya  |       2       |  70000 |
                          |   +----+--------+---------------+--------+
                          +------ (Ravi & Priya also point to Engineering)
```

What is SQL? SQL (Structured Query Language) is the standard language for working with relational databases. It is **declarative**: you describe *what* data you want, and the database decides *how* to get it efficiently.

```sql
SELECT name, salary          -- which columns
FROM employees               -- which table
WHERE salary > 80000         -- which rows
ORDER BY salary DESC;        -- in what order
```

Popular relational databases are **PostgreSQL**, **MySQL**, **SQLite**, **SQL Server** and **Oracle**. Core SQL is the same in all of them, with small dialect differences. A **DBMS** (Database Management System) is the software that runs the database.

Examples use **PostgreSQL** syntax (differences with MySQL noted where important). Sample schema used throughout:

```sql
CREATE TABLE departments (
    id   SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE
);

CREATE TABLE employees (
    id            SERIAL PRIMARY KEY,
    name          VARCHAR(100) NOT NULL,
    email         VARCHAR(255) UNIQUE NOT NULL,
    department_id INT REFERENCES departments(id) ON DELETE SET NULL,
    manager_id    INT REFERENCES employees(id),
```

```
    salary        NUMERIC(10,2) CHECK (salary > 0),
    hired_at      DATE NOT NULL DEFAULT CURRENT_DATE,
    is_active     BOOLEAN DEFAULT TRUE
);

CREATE TABLE orders (
    id          BIGSERIAL PRIMARY KEY,
    customer_id INT NOT NULL,
    amount      NUMERIC(12,2) NOT NULL,
    status      VARCHAR(20) NOT NULL DEFAULT 'pending',
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

7.1 SQL Sub-languages

UNDERSTAND THE CONCEPT

SQL commands fall into families according to their purpose:

- **DDL (Data Definition Language)** defines the *structure*: create, change or delete tables, columns and indexes.
- **DML (Data Manipulation Language)** changes the *data*: add rows (`INSERT`), change them (`UPDATE`), remove them (`DELETE`).
- **DQL (Data Query Language)** *reads* data: `SELECT` . This is what you'll write most.
- **DCL (Data Control Language)** manages *permissions*: who can read or change what.
- **TCL (Transaction Control Language)** groups changes into all-or-nothing **transactions**.

Category	Commands
DDL (Definition)	`CREATE` , `ALTER` , `DROP` , `TRUNCATE` , `RENAME`
DML (Manipulation)	`INSERT` , `UPDATE` , `DELETE` , `MERGE`
DQL (Query)	`SELECT`
DCL (Control)	`GRANT` , `REVOKE`
TCL (Transaction)	`BEGIN` , `COMMIT` , `ROLLBACK` , `SAVEPOINT`

7.2 Data Types (Common)

UNDERSTAND THE CONCEPT

Every column has a **data type** that restricts what values it can hold. This protects data quality (you can't put "abc" into a number column) and lets the database store and compare values efficiently.

Choosing well:

- Use `INT` / `BIGINT` for IDs and counts. Use `BIGINT` for tables that may exceed about 2 billion rows.

- Use `NUMERIC` / `DECIMAL` for **money**. They are exact, while `FLOAT` can produce rounding errors like `0.1 + 0.2 = 0.30000000000000004`.
- Use `VARCHAR(n)` for text with a maximum length, and `TEXT` for unlimited text.
- Use `TIMESTAMPTZ` (timestamp with time zone) for moments in time, stored in UTC. Use `DATE` for calendar days.
- Use `BOOLEAN` for yes/no, `UUID` for globally unique IDs, and `JSONB` (PostgreSQL) for flexible semi-structured data.

NULL is a special marker meaning "**unknown / no value**". It is not zero and not an empty string (see 7.4).

- Numeric: `SMALLINT`, `INT`, `BIGINT`, `NUMERIC(p,s)` / `DECIMAL` (exact — use for money), `REAL` / `DOUBLE PRECISION` (approximate).
- Text: `CHAR(n)` (fixed), `VARCHAR(n)`, `TEXT`.
- Date/time: `DATE`, `TIME`, `TIMESTAMP`, `TIMESTAMPTZ` (store UTC — preferred), `INTERVAL`.
- Others: `BOOLEAN`, `UUID`, `JSON` / `JSONB` (binary, indexable), `ARRAY`, `ENUM`, `BYTEA`.

7.3 DDL and Constraints

UNDERSTAND THE CONCEPT

`CREATE TABLE` defines a table's columns and their types. `ALTER TABLE` changes an existing table. `DROP TABLE` deletes it entirely.

Constraints are rules the database **enforces automatically**, so bad data can never get in, whatever application or person writes to it:

- `PRIMARY KEY`: unique and never NULL. It identifies each row.
- `FOREIGN KEY` / `REFERENCES`: the value must exist in the other table (you can't assign an employee to department 99 if it doesn't exist). This is called **referential integrity**. `ON DELETE` decides what happens to the referencing rows when the referenced row is deleted: delete them too (`CASCADE`), set the column to NULL (`SET NULL`), or refuse the delete (`RESTRICT`).
- `UNIQUE`: no two rows may have the same value (e.g. email).
- `NOT NULL`: a value is required.
- `CHECK`: a custom rule, e.g. `salary > 0`.
- `DEFAULT`: the value used when none is provided.

```
ALTER TABLE employees ADD COLUMN phone VARCHAR(20);
ALTER TABLE employees ALTER COLUMN phone SET NOT NULL;
ALTER TABLE employees DROP COLUMN phone;
ALTER TABLE employees RENAME COLUMN name TO full_name;
ALTER TABLE employees ADD CONSTRAINT chk_email CHECK (email LIKE '%@%');

DROP TABLE IF EXISTS temp_data;          -- removes table + structure
TRUNCATE TABLE logs RESTART IDENTITY;    -- removes all rows fast, keeps structure
```

Constraints: `PRIMARY KEY` (unique + not null), `FOREIGN KEY` (referential integrity; `ON DELETE CASCADE | SET NULL | RESTRICT`), `UNIQUE`, `NOT NULL`, `CHECK`, `DEFAULT`.

DELETE vs TRUNCATE vs DROP:

	DELETE	TRUNCATE	DROP
Removes	Selected rows (`WHERE`)	All rows	Whole table
Speed	Slower (row by row, logs each)	Fast	Fast
Triggers fire	Yes	No	No
Rollback	Yes	Yes in Postgres (not in MySQL)	Yes in Postgres

7.4 Basic DML and Queries

UNDERSTAND THE CONCEPT

The four basic data operations are known as **CRUD**: **C**reate (`INSERT`), **R**ead (`SELECT`), **U**pdate (`UPDATE`), **D**elete (`DELETE`).

Always include a `WHERE` clause on `UPDATE` and `DELETE`. Without it, *every* row in the table is changed or deleted.

Anatomy of a `SELECT` :

- **`SELECT`** chooses the columns (or `*` for all). You can compute new columns and rename them with `AS` .
- **`FROM`** chooses the table.
- **`WHERE`** filters rows using conditions: comparisons (`=` , `<>` , `>`), `AND` / `OR` / `NOT` , `IN (list)` , `BETWEEN a AND b` , `LIKE 'pattern'` (`%` = any characters, `_` = one character), `IS NULL` .
- **`ORDER BY`** sorts (`ASC` ascending is the default, `DESC` descending).
- **`LIMIT` / `OFFSET`** return only some rows, which is used for pagination.
- **`DISTINCT`** removes duplicate rows.
- **`CASE WHEN ... THEN ... ELSE ... END`** is SQL's if/else, used inside queries.

NULL logic: because NULL means "unknown", any comparison with NULL gives *unknown* rather than true or false. So `WHERE manager_id = NULL` never matches anything. You must write `IS NULL` / `IS NOT NULL` . `COALESCE(a, b)` returns the first non-NULL value, which makes it handy for defaults.

```sql
INSERT INTO departments (name) VALUES ('Engineering'), ('Sales'), ('HR');

INSERT INTO employees (name, email, department_id, salary)
VALUES ('Anish', 'anish@x.com', 1, 120000)
RETURNING id;                                  -- Postgres

-- Upsert
INSERT INTO employees (email, name, salary) VALUES ('a@x.com', 'A', 50000)
ON CONFLICT (email) DO UPDATE SET salary = EXCLUDED.salary;
-- MySQL: INSERT ... ON DUPLICATE KEY UPDATE salary = VALUES(salary);

UPDATE employees SET salary = salary * 1.10 WHERE department_id = 1;
DELETE FROM employees WHERE is_active = FALSE;

SELECT DISTINCT department_id FROM employees;
```

```sql
SELECT name, salary
FROM employees
WHERE salary BETWEEN 50000 AND 150000
  AND name ILIKE 'a%'                -- LIKE: % any chars, _ single char; ILIKE = case-
insensitive
  AND department_id IN (1, 2)
  AND manager_id IS NOT NULL         -- never "= NULL"
ORDER BY salary DESC NULLS LAST, name
LIMIT 10 OFFSET 20;                  -- pagination (SQL Server: OFFSET ... FETCH NEXT)

SELECT name,
       CASE
         WHEN salary >= 150000 THEN 'High'
         WHEN salary >= 80000  THEN 'Mid'
         ELSE 'Low'
       END AS band,
       COALESCE(manager_id, 0) AS mgr,   -- first non-null
       NULLIF(salary, 0)                 -- NULL if equal
FROM employees;
```

NULL semantics (three-valued logic)

- `NULL = NULL` → `NULL` (unknown), not true. Use `IS NULL` / `IS DISTINCT FROM`.
- Aggregates ignore NULLs (except `COUNT(*)`).
- `NOT IN (subquery)` returns no rows if the subquery contains a NULL → prefer `NOT EXISTS`.

7.5 Logical Order of Execution

UNDERSTAND THE CONCEPT

You *write* a query starting with `SELECT`, but the database *processes* the clauses in a different, **logical order**:

1. **FROM / JOIN**: gather the tables and combine them.
2. **WHERE**: filter individual rows.
3. **GROUP BY**: form groups.
4. **HAVING**: filter groups.
5. **SELECT**: compute the output columns (and aliases).
6. **DISTINCT**: remove duplicates.
7. **ORDER BY**: sort.
8. **LIMIT / OFFSET**: cut the result.

Knowing this explains many errors. You can't use a column alias from `SELECT` inside `WHERE` (it hasn't been computed yet), and you can't use aggregates like `COUNT()` in `WHERE` (groups don't exist yet), which is why `HAVING` exists.

```
FROM / JOIN  →  WHERE  →  GROUP BY  →  HAVING  →  SELECT  →  DISTINCT  →  ORDER BY  →
LIMIT/OFFSET
```

That is why you cannot use a `SELECT` alias inside `WHERE` (it doesn't exist yet), but you can in `ORDER BY`. Window functions are computed after `HAVING`, before `ORDER BY`.

7.6 Aggregation: GROUP BY and HAVING

UNDERSTAND THE CONCEPT

Aggregate functions compute **one value from many rows**: `COUNT` (how many), `SUM` (total), `AVG` (average), `MIN`, `MAX`.

```
SELECT COUNT(*), AVG(salary) FROM employees;   -- one row for the whole table
```

`GROUP BY` splits rows into **groups** that share the same value(s) and computes the aggregates **per group**, for example "average salary *per department*". Picture sorting employees into piles by department and then counting each pile. Every column in `SELECT` must either be in the `GROUP BY` or be inside an aggregate function.

`WHERE` vs `HAVING`: `WHERE` filters **rows before** grouping ("only active employees"). `HAVING` filters **groups after** aggregation ("only departments with more than 5 people").

Conditional aggregation counts or sums only the rows that meet a condition (`FILTER (WHERE ...)` or `SUM(CASE WHEN ...)`). It turns rows into columns, which is called a **pivot**, and is very common in reports.

```
SELECT department_id,
       COUNT(*)            AS headcount,
       COUNT(manager_id)   AS with_manager,   -- counts non-null
       AVG(salary)::INT    AS avg_salary,
       MIN(salary), MAX(salary), SUM(salary),
       STRING_AGG(name, ', ' ORDER BY name) AS names  -- MySQL: GROUP_CONCAT
FROM employees
WHERE is_active
GROUP BY department_id
HAVING COUNT(*) > 5             -- filter groups (WHERE filters rows)
ORDER BY avg_salary DESC;

-- Conditional aggregation (pivot)
SELECT DATE_TRUNC('month', created_at) AS month,
       COUNT(*) FILTER (WHERE status = 'paid')     AS paid,
       COUNT(*) FILTER (WHERE status = 'refunded') AS refunded,
       SUM(CASE WHEN status = 'paid' THEN amount ELSE 0 END) AS revenue
FROM orders
GROUP BY 1
ORDER BY 1;

-- Subtotals
SELECT department_id, status, SUM(amount)
FROM orders GROUP BY ROLLUP (department_id, status);  -- also CUBE, GROUPING SETS
```

7.7 Joins

UNDERSTAND THE CONCEPT

A **JOIN** combines rows from two (or more) tables based on a related column, usually a foreign key matching a primary key. Joins are how you put normalised data back together ("show each employee *with their department name*").

Think of two circles that overlap (a Venn diagram):

- `INNER JOIN` (or just `JOIN`) returns only rows that have a match in **both** tables. Employees without a department are dropped.
- `LEFT JOIN` returns **all** rows from the left table, plus matching data from the right. Where there is no match, the right-side columns are `NULL`. Use it for "all departments, *even those with no employees*".
- `RIGHT JOIN` is the mirror image of LEFT JOIN, and is rarely used.
- `FULL OUTER JOIN` returns all rows from both sides, matched where possible.
- `CROSS JOIN` returns every combination of rows (rows in A × rows in B).
- **Self join** joins a table to itself, e.g. matching employees with their managers, who are also in the `employees` table. Table **aliases** (`e`, `m`) tell the two copies apart.

```
INNER:  A ∩ B          LEFT:  all of A (+ B where it matches)          FULL: A ∪ B
```

Anti-join pattern: `LEFT JOIN ... WHERE right.id IS NULL` finds rows in A that have **no** match in B ("customers who never ordered").

```sql
-- INNER: only matching rows
SELECT e.name, d.name AS dept
FROM employees e
JOIN departments d ON d.id = e.department_id;

-- LEFT: all rows from left, NULLs when no match
SELECT d.name, COUNT(e.id) AS headcount        -- COUNT(e.id) not COUNT(*) → 0 for empty depts
FROM departments d
LEFT JOIN employees e ON e.department_id = d.id
GROUP BY d.name;

-- Anti-join: departments with NO employees
SELECT d.* FROM departments d
LEFT JOIN employees e ON e.department_id = d.id
WHERE e.id IS NULL;

-- RIGHT JOIN (mirror of LEFT), FULL OUTER JOIN (all rows from both)
SELECT * FROM a FULL OUTER JOIN b ON a.id = b.a_id;

-- CROSS JOIN: Cartesian product (every combination)
SELECT s.size, c.color FROM sizes s CROSS JOIN colors c;

-- SELF JOIN: employee with manager name
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON m.id = e.manager_id;

-- Join with extra condition: put it in ON (not WHERE) for LEFT joins to keep unmatched rows
SELECT c.id, o.id
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id AND o.status = 'paid';
```

Join algorithms (chosen by the planner): Nested Loop (small inner/indexed), Hash Join (large unsorted equi-joins), Merge Join (both inputs sorted).

7.8 Set Operations

UNDERSTAND THE CONCEPT

Set operations stack the results of two queries **vertically** (one under the other), while joins combine tables side by side. Both queries must return the same number of columns with compatible types.

- `UNION` : all rows from both, with duplicates removed.
- `UNION ALL` : all rows from both, keeping duplicates. It is faster because it doesn't check for duplicates.
- `INTERSECT` : only rows appearing in both.
- `EXCEPT` : rows in the first but not in the second.

```
SELECT email FROM customers
UNION                -- removes duplicates (sort/hash cost)
SELECT email FROM newsletter;

SELECT email FROM customers UNION ALL SELECT email FROM newsletter;  -- keeps dupes, faster
SELECT id FROM a INTERSECT SELECT id FROM b;
SELECT id FROM a EXCEPT SELECT id FROM b;       -- Oracle: MINUS
```

Column count and compatible types must match.

7.9 Subqueries

UNDERSTAND THE CONCEPT

A **subquery** is a query nested inside another query. You use the result of one question to answer another, e.g. "employees earning more than **the average salary**", where the average is itself a query.

- **Scalar subquery:** returns a single value, usable anywhere a value is expected.
- `IN (subquery)` **:** returns a list of values to match against.
- **Correlated subquery:** refers to the outer query's current row, so it conceptually runs **once per outer row** ("above the average *of their own* department"). Powerful, but it can be slow on large tables.
- `EXISTS (subquery)` **:** true if the subquery returns at least one row. It stops as soon as it finds one, and it handles NULLs correctly. Prefer `NOT EXISTS` over `NOT IN` .
- **Derived table:** a subquery in `FROM` , treated like a temporary table.
- `LATERAL` **:** a subquery in `FROM` that can refer to columns of tables before it, e.g. "the top 3 orders *for each* customer".

```
-- Scalar subquery
SELECT name, salary, (SELECT AVG(salary) FROM employees) AS company_avg FROM employees;

-- IN subquery
SELECT * FROM employees
WHERE department_id IN (SELECT id FROM departments WHERE name IN ('Engineering', 'Sales'));

-- Correlated subquery: runs per outer row — employees earning above their dept average
SELECT e.name, e.salary
FROM employees e
WHERE e.salary > (SELECT AVG(salary) FROM employees WHERE department_id = e.department_id);

-- EXISTS / NOT EXISTS (stops at first match; NULL-safe)
```

```sql
SELECT c.* FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.id);

-- Derived table
SELECT dept, avg_sal FROM (
    SELECT department_id AS dept, AVG(salary) AS avg_sal FROM employees GROUP BY department_id
) t WHERE avg_sal > 100000;

-- LATERAL: top 3 orders per customer
SELECT c.id, o.*
FROM customers c
CROSS JOIN LATERAL (
    SELECT * FROM orders WHERE customer_id = c.id ORDER BY amount DESC LIMIT 3
) o;
```

7.10 Common Table Expressions (CTEs)

UNDERSTAND THE CONCEPT

A **CTE (Common Table Expression)**, written with `WITH name AS (...)`, gives a name to a subquery so you can refer to it like a table in the main query. CTEs make complex queries **readable**: you can build the logic step by step, like intermediate variables in code, instead of nesting subqueries inside subqueries.

A **recursive CTE** refers to *itself*, which lets SQL walk through **hierarchies and graphs** such as org charts (employee → manager → manager's manager), category trees, folder structures and threaded comments. It has two parts joined by `UNION ALL`:

1. The **anchor** member is the starting rows (e.g. the CEO, who has no manager).
2. The **recursive** member joins the table to the rows found in the previous step (e.g. the employees whose manager is in the previous level).

It repeats until a step finds no new rows.

```sql
WITH dept_stats AS (
    SELECT department_id, AVG(salary) AS avg_sal
    FROM employees GROUP BY department_id
),
high_paid AS (
    SELECT e.* FROM employees e
    JOIN dept_stats s USING (department_id)
    WHERE e.salary > s.avg_sal * 1.2
)
SELECT * FROM high_paid;
```

Recursive CTE — hierarchy / org chart

```sql
WITH RECURSIVE org AS (
    SELECT id, name, manager_id, 1 AS level, name::TEXT AS path
    FROM employees WHERE manager_id IS NULL                  -- anchor: CEO
    UNION ALL
    SELECT e.id, e.name, e.manager_id, o.level + 1, o.path || ' > ' || e.name
    FROM employees e
    JOIN org o ON e.manager_id = o.id                        -- recursive step
)
SELECT * FROM org ORDER BY path;
```

```sql
-- Generate a date series (useful to fill gaps in reports)
WITH RECURSIVE days AS (
    SELECT DATE '2026-01-01' AS d
    UNION ALL SELECT d + 1 FROM days WHERE d < DATE '2026-01-31'
)
SELECT d FROM days;
-- Postgres shortcut: SELECT generate_series('2026-01-01'::date, '2026-01-31', '1 day');
```

7.11 Window Functions

UNDERSTAND THE CONCEPT

Window functions are among the most powerful and most-asked-about SQL features.

They calculate a value **for each row using a group of related rows (a "window")**, while **keeping every row** in the result. `GROUP BY`, in contrast, collapses each group into one row.

Example: show every employee **and** their department's average salary next to them. With `GROUP BY` you'd lose the individual employees. With a window function you keep them:

```sql
SELECT name, department_id, salary,
       AVG(salary) OVER (PARTITION BY department_id) AS dept_avg
FROM employees;
```

The `OVER (...)` clause defines the window:

- `PARTITION BY` splits rows into groups. The function restarts for each group, like GROUP BY but without collapsing rows.
- `ORDER BY` sets the order of rows inside each partition. It is needed for rankings, running totals and "previous/next row" functions.
- **Frame** (`ROWS BETWEEN ...`) sets exactly which rows around the current one to include, e.g. "the 6 previous rows plus this one" for a 7-day moving average.

Families of window functions:

- **Ranking:** `ROW_NUMBER()` (1, 2, 3, 4, always unique), `RANK()` (1, 2, 2, 4, ties share a rank and leave a gap), `DENSE_RANK()` (1, 2, 2, 3, no gaps), `NTILE(n)` (split into n buckets).
- **Aggregates as windows:** `SUM`, `AVG`, `COUNT` with `OVER` give running totals, percentages of a total, and group averages.
- **Offset functions:** `LAG()` reads a value from the **previous** row and `LEAD()` from the **next** row. They are perfect for "change compared to last month".
- `FIRST_VALUE` / `LAST_VALUE`: the first or last value in the window.

Because window functions are computed *after* `WHERE`, you can't filter on them directly. Wrap the query in a subquery or CTE and filter outside (that is the "top N per group" pattern).

Window functions compute a value **per row** over a set of related rows **without collapsing** them (unlike `GROUP BY`).

```
function() OVER (PARTITION BY ... ORDER BY ... ROWS/RANGE BETWEEN ... )
```

```
-- Ranking
SELECT name, department_id, salary,
       ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY salary DESC) AS rn,   -- 1,2,3,4
       RANK()       OVER (PARTITION BY department_id ORDER BY salary DESC) AS rnk,  -- 1,2,2,4
       DENSE_RANK() OVER (PARTITION BY department_id ORDER BY salary DESC) AS drnk, -- 1,2,2,3
       NTILE(4)     OVER (ORDER BY salary) AS quartile,
       PERCENT_RANK() OVER (ORDER BY salary) AS pct
FROM employees;

-- Top-N per group (classic interview question)
SELECT * FROM (
    SELECT e.*, DENSE_RANK() OVER (PARTITION BY department_id ORDER BY salary DESC) AS r
    FROM employees e
) t WHERE r <= 3;

-- Aggregates as windows
SELECT name, department_id, salary,
       AVG(salary) OVER (PARTITION BY department_id) AS dept_avg,
       salary - AVG(salary) OVER (PARTITION BY department_id) AS diff_from_avg,
       SUM(salary) OVER () AS company_total,
       ROUND(100.0 * salary / SUM(salary) OVER (PARTITION BY department_id), 2) AS pct_of_dept
FROM employees;

-- Running total & moving average
SELECT created_at::date AS day, amount,
       SUM(amount) OVER (ORDER BY created_at ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
AS running_total,
       AVG(amount) OVER (ORDER BY created_at ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS
moving_avg_7
FROM orders;

-- LAG / LEAD: compare with previous / next row
SELECT month, revenue,
       LAG(revenue)  OVER (ORDER BY month) AS prev_month,
       revenue - LAG(revenue) OVER (ORDER BY month) AS mom_change,
       ROUND(100.0 * (revenue - LAG(revenue) OVER (ORDER BY month))
             / NULLIF(LAG(revenue) OVER (ORDER BY month), 0), 2) AS mom_pct,
       LEAD(revenue) OVER (ORDER BY month) AS next_month
FROM monthly_revenue;

-- FIRST_VALUE / LAST_VALUE / NTH_VALUE
SELECT name, department_id,
       FIRST_VALUE(name) OVER w AS top_earner,
       LAST_VALUE(name)  OVER (w ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS
lowest
FROM employees
WINDOW w AS (PARTITION BY department_id ORDER BY salary DESC);
```

ROWS vs RANGE: `ROWS` counts physical rows; `RANGE` groups peers with equal `ORDER BY` values. The default frame with `ORDER BY` is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` — the reason `LAST_VALUE` looks "wrong" without an explicit frame.

7.12 Classic SQL Interview Problems

UNDERSTAND THE CONCEPT

These are **classic interview problems**. Each one combines the building blocks above. Before reading a solution, try to answer three questions: *What does each output row represent? Which tables do I need? Do I need grouping, a window, or a self-join?*

The "**gaps and islands**" trick in problem 6 deserves an explanation. For consecutive dates, the date minus its row number stays **constant**, so that difference can be used as a group identifier for each streak.

```sql
-- 1. Second highest salary
SELECT MAX(salary) FROM employees WHERE salary < (SELECT MAX(salary) FROM employees);
-- Nth highest (N = 3)
SELECT DISTINCT salary FROM employees ORDER BY salary DESC OFFSET 2 LIMIT 1;

-- 2. Find duplicates
SELECT email, COUNT(*) FROM employees GROUP BY email HAVING COUNT(*) > 1;

-- 3. Delete duplicates, keep the lowest id
DELETE FROM employees e
USING employees e2
WHERE e.email = e2.email AND e.id > e2.id;
-- or with ROW_NUMBER():
DELETE FROM employees WHERE id IN (
  SELECT id FROM (SELECT id, ROW_NUMBER() OVER (PARTITION BY email ORDER BY id) rn FROM
employees) t
  WHERE rn > 1
);

-- 4. Employees earning more than their manager
SELECT e.name FROM employees e JOIN employees m ON e.manager_id = m.id WHERE e.salary >
m.salary;

-- 5. Customers who ordered in every month of 2026 so far
SELECT customer_id FROM orders
WHERE created_at >= '2026-01-01'
GROUP BY customer_id
HAVING COUNT(DISTINCT DATE_TRUNC('month', created_at)) = EXTRACT(MONTH FROM now());

-- 6. Consecutive days login (gaps & islands)
WITH d AS (
  SELECT DISTINCT user_id, login_date::date AS day FROM logins
), g AS (
  SELECT user_id, day,
         day - (ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY day))::INT AS grp
  FROM d
)
SELECT user_id, MIN(day) AS start_day, MAX(day) AS end_day, COUNT(*) AS streak
FROM g GROUP BY user_id, grp HAVING COUNT(*) >= 3;

-- 7. Cumulative % of revenue per customer (Pareto)
SELECT customer_id, total,
       SUM(total) OVER (ORDER BY total DESC) / SUM(total) OVER () AS cumulative_share
FROM (SELECT customer_id, SUM(amount) total FROM orders GROUP BY customer_id) t;

-- 8. Day-1 retention
SELECT f.first_day,
       COUNT(DISTINCT f.user_id) AS new_users,
       COUNT(DISTINCT l.user_id) AS returned_next_day
FROM (SELECT user_id, MIN(login_date) first_day FROM logins GROUP BY user_id) f
LEFT JOIN logins l ON l.user_id = f.user_id AND l.login_date = f.first_day + 1
GROUP BY f.first_day;
```

7.13 Indexes

UNDERSTAND THE CONCEPT

An **index** is a separate data structure that helps the database **find rows quickly without reading the whole table**. It is just like the index at the back of a book: instead of reading every page to find "closures", you look up the word and jump straight to page 42.

- **Without an index,** finding `WHERE email = 'a@x.com'` means a **full table scan**, checking every row. That's fine for 100 rows and painfully slow for 100 million.
- **With an index** on `email`, the database searches a sorted tree structure (a **B-tree**) and finds the row in a few steps, in O(log n) time.

The trade-off: indexes take extra disk space, and **every INSERT, UPDATE and DELETE must also update every index**. So index the columns you frequently **filter on (`WHERE`), join on (`JOIN`) and sort by (`ORDER BY`)**, not every column.

Key ideas:

- **Primary keys and UNIQUE constraints** get an index automatically.
- A **composite index** covers several columns, e.g. `(customer_id, created_at)`. Column order matters, like a phone book sorted by last name and then first name: it helps you search by last name, or last + first name, but not by first name alone. This is the **leftmost-prefix rule**.
- A **covering index** contains all the columns a query needs, so the database never has to visit the table itself.
- A **partial index** indexes only some rows (e.g. only `status = 'pending'`).
- A **clustered index** decides the *physical order* of the table's rows on disk. A table can have only one, and in MySQL InnoDB it is the primary key.

An index is an auxiliary data structure that speeds up lookups at the cost of extra storage and slower writes.

Index type	Good for
B-Tree (default)	`=`, `<`, `>`, `BETWEEN`, `ORDER BY`, prefix `LIKE 'abc%'`
Hash	Equality only
GIN	JSONB, arrays, full-text search (`@>`, `?`, `@@`)
GiST / SP-GiST	Geometric, ranges, nearest-neighbour
BRIN	Huge, naturally ordered tables (time-series)

```
CREATE INDEX idx_emp_dept ON employees (department_id);
CREATE UNIQUE INDEX idx_emp_email ON employees (lower(email));       -- expression index
CREATE INDEX idx_orders_cust_date ON orders (customer_id, created_at DESC); -- composite
CREATE INDEX idx_orders_pending ON orders (created_at) WHERE status = 'pending'; -- partial
CREATE INDEX idx_cover ON orders (customer_id) INCLUDE (amount);      -- covering (index-only
scans)
CREATE INDEX CONCURRENTLY idx_x ON big_table (col);                   -- no write lock
CREATE INDEX idx_meta ON products USING GIN (metadata jsonb_path_ops);
```

Clustered vs non-clustered: a clustered index defines the physical order of rows (InnoDB primary key; only one per table). Non-clustered (secondary) indexes store keys + a pointer (or PK in InnoDB). Postgres tables are heaps; `CLUSTER` is a one-time reorder.

Composite index — leftmost prefix rule: an index on `(a, b, c)` helps queries filtering on `a`, `a,b`, `a,b,c` — but not `b` alone or `c` alone. Put equality columns first, range columns last.

When an index is NOT used:

- Function/expression on the column: `WHERE LOWER(email) = ...` (create an expression index), `WHERE created_at::date = ...`.
- Leading wildcard: `LIKE '%abc'` (use trigram GIN index `pg_trgm`).
- Implicit type casts (`varchar_col = 123`).
- Low selectivity (e.g., boolean column, most rows match) — sequential scan is cheaper.
- `OR` across different columns (sometimes; may use bitmap OR).
- Stale statistics (`ANALYZE`).

7.14 Query Optimization and EXPLAIN

UNDERSTAND THE CONCEPT

`EXPLAIN` shows the **query plan**: the step-by-step strategy the database's **query planner (optimizer)** chose to run your query. That includes which indexes it will use, the order it joins tables in, and how many rows it expects at each step. `EXPLAIN ANALYZE` actually runs the query and shows **real times and row counts** next to the estimates.

How to read it at a basic level:

- **Seq Scan** means the whole table was read. On a big table with a selective `WHERE`, this usually means a missing index.
- **Index Scan / Index Only Scan** means an index was used. That is good.
- Look for the step with the **largest time**, and for **estimated rows far from actual rows** (outdated statistics, so run `ANALYZE`).

Sargable ("Search ARGument ABLE") means a condition is written so an index *can* be used. Wrapping the column in a function (`WHERE YEAR(created_at) = 2026`) usually prevents index use. Rewrite it as a range on the raw column (`created_at >= '2026-01-01' AND created_at < '2027-01-01'`).

Keyset pagination: `OFFSET 100000` still makes the database walk past 100,000 rows. Instead, remember the last row you showed and ask for rows "after" it, which is fast at any depth.

```sql
EXPLAIN ANALYZE
SELECT o.* FROM orders o
WHERE o.customer_id = 42 AND o.created_at > now() - INTERVAL '30 days'
ORDER BY o.created_at DESC
LIMIT 20;
```

Reading the plan:

- **Seq Scan** — full table scan (fine for small tables).
- **Index Scan** — uses index, fetches rows from the heap.
- **Index Only Scan** — all needed columns in the index (best).

- **Bitmap Heap Scan** — many rows via index, read pages in order.
- Compare **estimated rows vs actual rows** — big mismatches mean stale stats.
- Look for expensive **Sort**, **Hash**, nested loops over big sets, and high `Buffers` reads.

Optimization checklist:

1. Select only needed columns (avoid `SELECT *`).
2. Index columns used in `WHERE`, `JOIN`, `ORDER BY`; use composite/covering indexes.
3. Make predicates **sargable**: `created_at >= '2026-01-01' AND created_at < '2026-02-01'` instead of `EXTRACT(MONTH FROM created_at) = 1`.
4. Replace correlated subqueries with joins/window functions when slow.
5. Prefer `EXISTS` over `IN` for large subqueries; `UNION ALL` over `UNION` when duplicates are impossible.
6. **Keyset (cursor) pagination** instead of large `OFFSET`:

```
-- OFFSET 100000 must scan and discard 100k rows
SELECT * FROM orders WHERE (created_at, id) < ($last_created_at, $last_id)
ORDER BY created_at DESC, id DESC LIMIT 20;
```

7. Batch writes; avoid N+1 queries from application code.
8. Keep statistics fresh (`ANALYZE`), vacuum (Postgres MVCC creates dead tuples).
9. Partition very large tables; consider materialized views for heavy reports.

7.15 Transactions and ACID

UNDERSTAND THE CONCEPT

A **transaction** is a group of SQL statements treated as **one indivisible unit of work**: either **all** of them take effect (`COMMIT`) or **none** do (`ROLLBACK`). The classic example is a bank transfer. Subtracting from account A and adding to account B must happen together. If the server crashes in between, neither change should remain.

ACID describes the guarantees of a reliable transaction:

- **Atomicity:** all or nothing.
- **Consistency:** the data moves from one valid state to another. All constraints still hold.
- **Isolation:** transactions running at the same time don't see each other's half-finished work. How strictly is controlled by the **isolation level**.
- **Durability:** once committed, the data survives crashes and power loss, because it is written to a log on disk first.

Concurrency anomalies (problems that happen when transactions run at the same time):

- **Dirty read:** you read another transaction's changes before it commits, and it may still roll back.
- **Non-repeatable read:** you read the same row twice in one transaction and get different values, because someone committed in between.
- **Phantom read:** you run the same query twice and *new rows* appear.

- **Lost update:** two transactions read the same value, both modify it, and the second overwrites the first's change.

Higher **isolation levels** prevent more anomalies but allow less concurrency (more waiting or retries). **Locks** (`SELECT ... FOR UPDATE`) make other transactions wait for a row. **Optimistic locking** uses a version number and retries if someone else changed the row first. A **deadlock** is two transactions each waiting for a lock the other holds. The database detects it and cancels one of them.

MVCC (Multi-Version Concurrency Control), used by PostgreSQL and InnoDB, keeps multiple versions of rows, so **readers never block writers and writers never block readers**. Each transaction sees a consistent snapshot.

- **Atomicity** — all or nothing.
- **Consistency** — constraints always hold.
- **Isolation** — concurrent transactions don't interfere (to the configured level).
- **Durability** — committed data survives crashes (write-ahead log).

```
BEGIN;
UPDATE accounts SET balance = balance - 500 WHERE id = 1;
SAVEPOINT before_credit;
UPDATE accounts SET balance = balance + 500 WHERE id = 2;
-- ROLLBACK TO SAVEPOINT before_credit;
COMMIT;
```

Isolation Levels and Anomalies

Level	Dirty read	Non-repeatable read	Phantom read	Notes
Read Uncommitted	Possible	Possible	Possible	Postgres treats as Read Committed
Read Committed	No	Possible	Possible	Postgres default
Repeatable Read	No	No	Possible (No in PG)	MySQL InnoDB default
Serializable	No	No	No	May abort with serialization errors → retry

- **Dirty read:** reading uncommitted data of another transaction.
- **Non-repeatable read:** same row read twice returns different values.
- **Phantom read:** same query returns new rows inserted by another transaction.
- **Lost update:** two transactions read-modify-write the same row; one overwrites the other. Prevent with `UPDATE ... SET x = x - 1` , `SELECT ... FOR UPDATE` , or optimistic locking.

```
-- Pessimistic locking
BEGIN;
SELECT stock FROM products WHERE id = 7 FOR UPDATE;      -- others wait
UPDATE products SET stock = stock - 1 WHERE id = 7;
COMMIT;

-- Job queue pattern: workers grab different rows without blocking
SELECT id FROM jobs WHERE status = 'queued'
```

```sql
ORDER BY id LIMIT 10 FOR UPDATE SKIP LOCKED;

-- Optimistic locking
UPDATE products SET stock = 9, version = version + 1
WHERE id = 7 AND version = 3;          -- 0 rows updated → someone else changed it → retry
```

Deadlocks occur when two transactions wait on each other's locks; the DB aborts one. Avoid by locking rows in a consistent order and keeping transactions short.

MVCC (Postgres, InnoDB): readers don't block writers and writers don't block readers; each transaction sees a snapshot.

7.16 Normalization

UNDERSTAND THE CONCEPT

Normalization is the process of organising tables to **avoid duplicated data**. Duplication causes **anomalies**:

- **Update anomaly:** a customer's address is stored in 50 order rows, and you update 49 of them by mistake. Now the data contradicts itself.
- **Insert anomaly:** you can't record a new product until someone orders it, because products only exist inside order rows.
- **Delete anomaly:** deleting the last order of a customer also deletes the only record of that customer.

The cure is to store **each fact in exactly one place** and link tables with foreign keys. The normal forms are steps towards that:

- **1NF:** each cell holds a single value (no comma-separated lists), and each row is unique.
- **2NF:** every non-key column depends on the **whole** primary key (this matters for composite keys).
- **3NF:** non-key columns depend **only on the key**, not on other non-key columns. For example, `department_name` doesn't belong in `employees`. It belongs in `departments`.

A popular summary of 3NF: every column must depend on "*the key, the whole key, and nothing but the key*".

Denormalization deliberately adds some duplication back (summary tables, cached totals) to make reads faster, accepting extra work to keep the copies in sync. It is common in analytics and reporting.

Normal form	Rule
1NF	Atomic values; no repeating groups/arrays in a column; each row unique
2NF	1NF + no partial dependency (non-key column depends on whole composite key)
3NF	2NF + no transitive dependency (non-key → non-key)
BCNF	Every determinant is a candidate key

Example: `orders(order_id, customer_id, customer_name, product_id, product_name, qty)` violates 2NF/3NF → split into `customers`, `products`, `orders`, `order_items`.

Denormalization (duplicating data, summary tables, materialized views) trades write complexity for read speed — common in reporting/analytics (OLAP) systems.

Relationships: one-to-one (FK + UNIQUE), one-to-many (FK on the "many" side), many-to-many (junction table with two FKs and a composite PK/unique).

7.17 Views, Materialized Views, Functions, Triggers

UNDERSTAND THE CONCEPT

- A **view** is a **saved query** that you can use like a table. It stores no data. It just runs its query each time you use it. Use views to simplify complex queries or to expose only certain columns to certain users.
- A **materialized view** stores the query **result** physically, so it is fast to read. It must be **refreshed** to pick up changes. It suits heavy reports and dashboards.
- **Functions** and **stored procedures** are code stored and run **inside the database** (in PostgreSQL, using the PL/pgSQL language). They are useful for logic that must run close to the data.
- A **trigger** automatically runs a function **when an event happens** on a table (before or after INSERT/UPDATE/DELETE). Examples are setting `updated_at` automatically or writing audit logs.

Use database-side logic thoughtfully. It's powerful, but it is harder to version, test and debug than application code.

```sql
CREATE VIEW active_employees AS
SELECT id, name, department_id FROM employees WHERE is_active;

CREATE MATERIALIZED VIEW monthly_revenue AS
SELECT DATE_TRUNC('month', created_at) AS month, SUM(amount) AS revenue
FROM orders WHERE status = 'paid' GROUP BY 1;
REFRESH MATERIALIZED VIEW CONCURRENTLY monthly_revenue;  -- needs a unique index

-- Function (PL/pgSQL)
CREATE OR REPLACE FUNCTION dept_headcount(p_dept INT) RETURNS INT AS $$
BEGIN
  RETURN (SELECT COUNT(*) FROM employees WHERE department_id = p_dept);
END;
$$ LANGUAGE plpgsql STABLE;

-- Trigger: maintain updated_at
CREATE OR REPLACE FUNCTION set_updated_at() RETURNS TRIGGER AS $$
BEGIN
  NEW.updated_at := now();
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_orders_updated
BEFORE UPDATE ON orders
FOR EACH ROW EXECUTE FUNCTION set_updated_at();

-- Stored procedure (can manage transactions)
```

```
CREATE PROCEDURE transfer(src INT, dst INT, amt NUMERIC) LANGUAGE plpgsql AS $$
BEGIN
  UPDATE accounts SET balance = balance - amt WHERE id = src;
  UPDATE accounts SET balance = balance + amt WHERE id = dst;
  COMMIT;
END; $$;
CALL transfer(1, 2, 100);
```

7.18 Working with JSON, Strings and Dates (PostgreSQL)

UNDERSTAND THE CONCEPT

Modern databases offer many built-in **functions** for working with specific kinds of data:

- **JSON / JSONB** (PostgreSQL): store flexible, semi-structured data in a column, and still query inside it (`->` returns JSON, `->>` returns text). GIN indexes make it fast to search. This gives you some NoSQL-style flexibility inside a relational database.
- **String functions:** change case, cut, search, replace, split and concatenate text.
- **Date/time functions:** current time, add or subtract intervals, truncate to a day or month (`DATE_TRUNC` , very common for reports), extract parts like the year or weekday, and format dates.
- **Full-text search:** search natural-language text by words, ignoring word forms ("running" matches "run"), with ranking. That is far better than `LIKE '%word%'` .

```
-- JSONB
SELECT metadata->>'color' AS color,              -- text
       metadata->'dims'->>'w' AS width,
       metadata #>> '{dims,h}' AS height
FROM products
WHERE metadata @> '{"brand": "acme"}'            -- containment (GIN-indexable)
  AND metadata ? 'warranty';                     -- key exists

UPDATE products SET metadata = jsonb_set(metadata, '{color}', '"red"');
SELECT jsonb_agg(row_to_json(e)) FROM employees e;          -- rows → JSON array
SELECT * FROM jsonb_to_recordset('[{"a":1},{"a":2}]') AS x(a INT);

-- Strings
SELECT UPPER(name), LENGTH(name), SUBSTRING(name FROM 1 FOR 3), TRIM(name),
       CONCAT(first, ' ', last), SPLIT_PART(email, '@', 2), REPLACE(name, 'a', 'A'),
       REGEXP_REPLACE(phone, '\D', '', 'g')
FROM employees;

-- Dates
SELECT now(), CURRENT_DATE, now() - INTERVAL '7 days',
       DATE_TRUNC('week', created_at), EXTRACT(DOW FROM created_at),
       AGE(now(), hired_at), TO_CHAR(created_at, 'YYYY-MM-DD HH24:MI')
FROM orders;

-- Full-text search
SELECT * FROM articles
WHERE to_tsvector('english', title || ' ' || body) @@ plainto_tsquery('english', 'django orm');
```

7.19 Security, Roles and SQL Injection

UNDERSTAND THE CONCEPT

Database security has two sides:

1. **Access control:** create separate database **roles/users** with the **minimum permissions** they need (the *principle of least privilege*). For example, a reporting tool gets read-only access. **Row-Level Security** can restrict which rows each user sees, which is useful in multi-tenant apps.
2. **SQL injection**, the most famous web vulnerability. It happens when user input is **pasted directly into SQL text**. An attacker types SQL code into a form field and changes the meaning of your query. For example, they can bypass login or delete tables:

```
Code:   "SELECT * FROM users WHERE email = '" + input + "'"
Input:  ' OR '1'='1
Result: SELECT * FROM users WHERE email = '' OR '1'='1'   ← returns every user!
```

The fix is always parameterized queries (prepared statements). The SQL text and the values are sent to the database **separately**, so input is always treated as data, never as code. ORMs like Django's and Prisma do this automatically.

```sql
CREATE ROLE app_readonly LOGIN PASSWORD '...';
GRANT CONNECT ON DATABASE shop TO app_readonly;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO app_readonly;
REVOKE DELETE ON orders FROM app_user;

-- Row-Level Security (multi-tenant)
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON orders
  USING (tenant_id = current_setting('app.tenant_id')::INT);
```

SQL injection: never build SQL with string concatenation of user input.

```python
# BAD
cursor.execute(f"SELECT * FROM users WHERE email = '{email}'")   # email = "' OR 1=1 --"
# GOOD (parameterized)
cursor.execute("SELECT * FROM users WHERE email = %s", [email])
```

```javascript
// BAD:  pool.query(`SELECT * FROM users WHERE id = ${req.params.id}`)
// GOOD:
pool.query('SELECT * FROM users WHERE id = $1', [req.params.id]);
```

7.20 Scaling Databases

UNDERSTAND THE CONCEPT

When an application grows, a single database server can become the bottleneck. Common strategies, roughly in the order teams adopt them:

1. **Optimise first:** indexes, better queries, caching (often the cheapest and most effective step).
2. **Vertical scaling:** a bigger server with more CPU, RAM and faster disks. It is simple but has limits.

3. **Connection pooling:** reuse database connections efficiently.
4. **Read replicas:** copies of the database that receive all changes from the main (primary) server. Reads are spread across replicas, and writes still go to the primary. Replicas may lag slightly behind.
5. **Partitioning:** split one huge table into smaller pieces (e.g. one per month) inside the same database. Queries touch only the relevant pieces.
6. **Sharding:** split data across **multiple database servers** by a **shard key** (e.g. customer ID). It is very scalable but complex, because cross-shard queries and transactions are hard.

CAP theorem: when the network between servers fails (a Partition), a distributed system must choose between **C**onsistency (everyone sees the same latest data) and **A**vailability (every request gets an answer).

SQL vs NoSQL: relational (SQL) databases offer strong structure, relationships, joins and ACID transactions. NoSQL databases (document: MongoDB; key-value: Redis; wide-column: Cassandra) trade some of that for flexible schemas and easier horizontal scaling. Choose based on the shape of your data and how you'll query it.

- **Vertical scaling** (bigger machine) vs **horizontal** (more machines).
- **Read replicas** — offload reads; beware replication lag (read-your-writes).
- **Connection pooling** — PgBouncer / RDS Proxy; DB connections are expensive.
- **Partitioning** (single DB, split table):

```
CREATE TABLE events (id BIGINT, created_at TIMESTAMPTZ, payload JSONB)
PARTITION BY RANGE (created_at);
CREATE TABLE events_2026_10 PARTITION OF events
  FOR VALUES FROM ('2026-10-01') TO ('2026-11-01');
-- Partition pruning: queries filtering on created_at only scan relevant partitions.
```

- **Sharding** (split across DBs by shard key, e.g., `tenant_id`) — complex cross-shard joins/transactions.
- **CAP theorem:** in a network partition choose Consistency or Availability. **OLTP** (transactions, normalized) vs **OLAP** (analytics, columnar, denormalized).
- **SQL vs NoSQL:** SQL — schema, joins, ACID, complex queries; NoSQL (MongoDB, DynamoDB, Redis, Cassandra) — flexible schema, horizontal scale, access-pattern-driven design, often eventual consistency.

7.21 Quick Reference — Postgres vs MySQL

UNDERSTAND THE CONCEPT

PostgreSQL and MySQL are the two most popular open-source relational databases. The SQL you write is about 90% identical. This table lists the differences you are most likely to run into when switching between them or answering interview questions.

Feature	PostgreSQL	MySQL (InnoDB)
Auto-increment	`SERIAL` / `GENERATED ALWAYS AS IDENTITY`	`AUTO_INCREMENT`
Upsert	`ON CONFLICT ... DO UPDATE`	`ON DUPLICATE KEY UPDATE`
Returning rows	`RETURNING`	Not supported (use `LAST_INSERT_ID()`)
String aggregate	`STRING_AGG`	`GROUP_CONCAT`
Case-insensitive LIKE	`ILIKE`	Depends on collation (default CI)
JSON	`JSONB` with GIN indexes	`JSON` with generated-column indexes
Default isolation	Read Committed	Repeatable Read
Full outer join	Yes	No (emulate with UNION)